

ΠΑΝΕΠΙΣΤΗΜΙΟ ΠΑΤΡΩΝ - ΠΟΛΥΤΕΧΝΙΚΗ ΣΧΟΛΗ
ΤΜΗΜΑ ΜΗΧΑΝΙΚΩΝ Η/Υ
ΚΑΙ ΠΛΗΡΟΦΟΡΙΚΗΣ



ΠΑΝΕΠΙΣΤΗΜΙΟ
ΠΑΤΡΩΝ
UNIVERSITY OF PATRAS

Διπλωματική Εργασία

του φοιτητή του Τμήματος Μηχανικών Η/Υ και Πληροφορικής
της Πολυτεχνικής Σχολής του Πανεπιστημίου Πατρών

Άγγελου Δήμογλη

A.M.: 1059603

Θέμα

**Υλοποίηση Communication Controller σε FPGA, με βάση το πρωτόκολλο
FlexRay**

Επιβλέπων

Αναπληρωτής Καθηγητής Νικόλαος Σκλάβος, Πανεπιστήμιο Πατρών

Αριθμός Διπλωματικής Εργασίας:

Πάτρα, Ιούνιος 2022

ΠΙΣΤΟΠΟΙΗΣΗ

Πιστοποιείται ότι η διπλωματική εργασία με θέμα

**Υλοποίηση Communication Controller σε FPGA, με βάση το πρωτόκολλο
FlexRay**

του φοιτητή του Τμήματος Μηχανικών Η/Υ και Πληροφορικής

Άγγελου Δήμογλη

(Α.Μ.: 1059603)

παρουσιάστηκε δημόσια και εξετάστηκε στο τμήμα Μηχανικών Η/Υ και
Πληροφορικής στις

__/__/__

Ο Επιβλέπων

Νικόλαος Σκλάβος
Αναπληρωτής Καθηγητής

Στοιχεία διπλωματικής εργασίας

**Θέμα: Υλοποίηση Communication Controller σε FPGA, με βάση το
πρωτόκολλο FlexRay**

Φοιτητής: Άγγελος Δήμογλης

**Ομάδα επίβλεψης
Αναπληρωτής Καθηγητής Νικόλαος Σκλάβος, Πανεπιστήμιο Πατρών**

Περίληψη

Στις μέρες μας, τα σύγχρονα οχήματα ενσωματώνουν όλο και περισσότερες λειτουργίες, όπως αυτοματοποιημένο παρκάρισμα, εντοπισμός πεζών, κ.λ.π. Το πρωτόκολλο επικοινωνίας FlexRay είναι πια ένα από τα ευρέως χρησιμοποιούμενα πρωτόκολλα στο χώρο της αυτοκινητοβιομηχανίας, για την πραγματοποίηση των προαναφερθέντων λειτουργιών. Αυτό συμβαίνει γιατί, το συγκεκριμένο πρωτόκολλο προσφέρει ταχύτερη μετάδοση δεδομένων και μεγαλύτερη αξιοπιστία. Παράλληλα όμως, το πρωτόκολλο FlexRay χρησιμοποιεί περισσότερο εύρος ζώνης, και καταναλώνει περισσότερη ενέργεια σε σχέση με τα προγενέστερα πρωτόκολλα, με αποτέλεσμα την υποβάθμιση της συνολικής απόδοσης του συστήματος. Γι' αυτό το λόγο, προτείνεται η επέκταση των δυνατοτήτων των communication controllers, σε κάθε κόμβο του δικτύου, όπου διενεργεί το πρωτόκολλο FlexRay. Έτσι, θα βελτιωθεί η συνολική ποιότητα και αποτελεσματικότητα του συστήματος. Οι περισσότερες συσκευές στο εμπόριο δεν επιτρέπουν αυτή την επέκταση, αλλά αυτή θα ήταν εφικτή με τη χρήση προγραμματιζόμενης λογικής, και πιο συγκεκριμένα Field-Programmable Gate Array (FPGA). Στόχος λοιπόν της παρούσας διπλωματικής εργασίας είναι ο σχεδιασμός και η υλοποίηση ενός αντίστοιχου Communication Controller με τη χρήση της γλώσσας περιγραφής υλικού VHDL.

Abstract

Nowadays, the synchronous vehicles incorporate more and more features, such as automated parking, pedestrian detection, etc. The communication protocol FlexRay is now one of the widely used protocols in the automotive industry, for the operation of the aforementioned features. That is because the specific protocol offers faster data transmission and higher reliability. At the same time though, the FlexRay protocol requires higher bandwidth and consumes more energy, opposed to previously used protocols, resulting in the degradation of the overall system performance. For this reason, the extension of the communication controllers' capabilities is suggested, in each network's node, where the FlexRay protocol operates. In this way, the quality and the efficiency of the system would be increased. Most of the off-the-shelf devices don't allow such an extension, but it would be possible by using reconfigurable logic, and specifically Field-Programmable Gate Array (FPGA). So, the purpose of the present Diploma Thesis, is the design and the implementation of a corresponding Communication Controller, by using the hardware description language VHDL.

Ευχαριστίες

Αρχικά, θα ήθελα να ευχαριστήσω τον επιβλέποντα καθηγητή μου, κ. Νικόλαο Σκλάβο, ο οποίος μου υπέδειξε τις κατάλληλες πηγές έρευνας για το θέμα, μου έδωσε σημαντικές συμβουλές, και μου έκφρασε την άποψή του για κάθε απόφασή μου. Επίσης, οφείλω να ευχαριστήσω τους καθηγητές του τμήματος Μηχανικών Η/Υ & Πληροφορικής, οι οποίοι με εφοδίασαν με τις κατάλληλες γνώσεις, για να ολοκληρώσω επιτυχώς τη διπλωματική μου εργασία. Τέλος, θα ήθελα να ευχαριστήσω την οικογένειά μου, που μου παρείχε την κατάλληλη υποστήριξη καθ' όλη τη διάρκεια των σπουδών μου.

Περιεχόμενα

Κατάλογος σχημάτων	xvii
Κατάλογος πινάκων	xix
1 Εισαγωγή	1
1.1 Το πρόβλημα προς επίλυση	1
1.2 Στόχος της Εργασίας	2
1.3 Διάρθρωση της Διπλωματικής Εργασίας	2
2 Ενσωματωμένα Συστήματα	5
2.1 Χρησιμοποιούμενοι Όροι	5
2.2 Εφαρμογές	6
2.3 Προκλήσεις	8
2.4 Κοινά Χαρακτηριστικά	10
3 Το πρωτόκολλο FlexRay	13
3.1 Η δημιουργία του πρωτοκόλλου FlexRay	13
3.1.1 Απαραίτητες Προδιαγραφές	14
3.2 Γενικά Χαρακτηριστικά	16
3.3 Κανάλια και Κύκλοι Επικοινωνίας	19
3.3.1 Κανάλι Επικοινωνίας	19
3.3.2 Κύκλος Επικοινωνίας	19
3.4 Τμήματα	20
3.4.1 Slots και Minislots	20
3.4.2 Στατικά Τμήματα και Slots	21
3.4.3 Δυναμικά Τμήματα και Minislots	21
3.4.4 Το Τμήμα Symbol Window	22
3.4.5 Το Τμήμα Network Idle Time	22
3.5 Πλαίσια Επικοινωνίας	23
3.5.1 Συστατικά Μέρη των Πλαισίων	23
3.6 Η Αρχιτεκτονική ενός Κόμβου	25
4 Προσέγγιση στη λύση του προβλήματος	29
4.1 Βελτιώσεις στο πρωτόκολλο FlexRay	29
4.1.1 Τοπολογία	29
4.1.2 Χρονοπρογραμματισμός	30
4.1.3 Βελτιστοποιήσεις σε επίπεδο δικτύου	30
4.2 Το Communication Controller	30
4.2.1 Αρχιτεκτονική	31
4.2.2 Υλοποίηση και Βελτιστοποιήσεις	31
4.2.3 Επεκτάσεις στο Data Path	34

4.3	Πειραματικά Αποτελέσματα	35
4.3.1	Περίπτωση Χρήσης: Εντοπισμός Σφαλμάτων & Εφεδρική Λειτουργία . . .	36
4.4	Συμπεράσματα	37
5	Υλοποίηση Communication Controller	39
5.1	Γενική Αρχιτεκτονική	39
5.2	Η δομή του πλαισίου δεδομένων	40
5.3	Protocol Management Module	41
5.3.1	Οι εντολές από τον Host/Microcontroller	42
5.3.2	Η μηχανή πεπερασμένων καταστάσεων	42
5.3.3	Testbench	44
5.4	Controller Host Interface	45
5.4.1	Testbench	45
5.5	Medium Interface Layer	46
5.5.1	Output Layer	46
5.5.2	Input Layer	48
5.5.3	Testbench	49
5.6	Wakeup Pattern Generator	51
5.6.1	Testbench	52
5.7	Wakeup Pattern Recognition	52
5.7.1	Testbench	53
5.8	Clock Synchronization	53
5.8.1	Τρόπος Λειτουργίας	54
5.8.2	Πλαισίωση Δεδομένων	54
5.8.3	Testbench	56
5.9	Δοκιμές Συστήματος	57
5.9.1	Testbench	58
5.10	Σύνθεση Συστήματος	61
5.10.1	Γενικές Πληροφορίες	61
5.10.2	Αποτελέσματα Σύνθεσης	62
6	Συμπεράσματα & Μελλοντική Εργασία	65
6.1	Συμπεράσματα	65
6.2	Μελλοντική Εργασία	65
	Παραρτήματα	67
A	Κώδικας	69
A.1	Protocol Management Module	69
A.2	Controller Host Interface	73
A.3	Medium Interface Layer	74
A.4	Wakeup Pattern Generator	85
A.5	Wakeup Pattern Recognition	89
A.6	Clock Synchronization	92
A.7	Communication Controller (Ενοποίηση των components)	98
B	Σχόλια σχετικά με τον κώδικα	105
B.1	Οι μηχανές πεπερασμένων καταστάσεων	105
B.2	Αντιστοιχίες σημάτων στο σύστημα	106
B.3	Η εγκυρότητα του σήματος sync_status	107

Γ Τεχνολογία FPGA	109
ΓΠ.1 Τί είναι FPGA;	109
ΓΠ.2 Η κατάταξη των FPGAs	109
ΓΠ.3 Η δομή ενός FPGA	110
Δ Κωδικοποίηση ανά μπλοκ	113
ΔΠ.1 Κωδικοποίηση 4B/5B	113
Ε Το μοντέλο OSI	115
ΕΠ.1 Εισαγωγή	115
ΕΠ.2 Περιγραφή των Επιπέδων	116
ΕΠ.2.1 Φυσικό Επίπεδο (Physical Layer)	116
ΕΠ.2.2 Επίπεδο Συνδέσμου Δεδομένων (Data Link Layer)	116
ΕΠ.2.3 Επίπεδο Δικτύου (Network Layer)	117
ΕΠ.2.4 Επίπεδο Μεταφοράς (Transport Layer)	117
ΕΠ.2.5 Επίπεδο Συνόδου (Session Layer)	117
ΕΠ.2.6 Επίπεδο Παρουσίασης (Presentation Layer)	117
ΕΠ.2.7 Επίπεδο Εφαρμογής (Application Layer)	118
Βιβλιογραφία	119

Κατάλογος σχημάτων

2.1	Η σχέση μεταξύ των Ενσωματωμένων και των Κυβερνοφυσικών Συστημάτων	6
2.2	Ενσωματωμένα συστήματα στα αυτοκίνητα Πηγή: Modern Car Technology	7
2.3	Διαφορές μεταξύ προσωπικών υπολογιστών και ενσωματωμένων συστημάτων	11
3.1	Εύρος προβλημάτων προς επίλυση	14
3.2	Διάρθρωση κύκλου σε στατικό και δυναμικό τμήμα	17
3.3	Η διπλή παράλληλη ιεραρχία του πρωτοκόλλου FlexRay	17
3.4	Τα στοιχεία του πρωτοκόλλου FlexRay	18
3.5	Η δομή των κύκλων επικοινωνίας	19
3.6	Slots και Minislots	20
3.7	Τεματισμός του Symbol Window	22
3.8	Διόρθωση φάσης τοπικού ρολογιού	23
3.9	Μετάδοση πλαισίων επικοινωνίας	23
3.10	Συστατικά μέρη πλαισίων επικοινωνίας	24
3.11	Οι διασυνδέσεις εντός ενός κόμβου FlexRay	25
3.12	Οι λογικές σχέσεις των συστατικών μερών του κόμβου FlexRay	26
3.13	Εσωτερική αρχιτεκτονική του Protocol Controller	27
4.1	Κύκλος επικοινωνίας του FlexRay	29
4.2	Τοπολογία ενεργού αστέρα	30
4.3	Τοπολογία μεταγωγέα	30
4.4	Αρχιτεκτονική τροποποιημένου Communication Controller	31
4.5	Καταστάσεις λειτουργίας του Communication Controller	32
4.6	Εντολές του host	32
4.7	Αρχιτεκτονική του Clock Synchronization	33
4.8	Αρχιτεκτονική του Medium Interface Layer	34
4.9	Επεκτάσεις στο Data Path του Communication Controller	35
4.10	Καθυστερήση για την επεξεργασία κρίσιμων δεδομένων	36
5.1	Γενική Αρχιτεκτονική Communication Controller	39
5.2	Χρησιμοποιούμενο πλαίσιο δεδομένων	40
5.3	Αρχιτεκτονική του Protocol Management Module	41
5.4	Διάγραμμα καταστάσεων της FSM	43
5.5	Testbench της υπομονάδας Next-state logic (FSM), Αρχείο: pmm/fsm_tb.vhd	44
5.6	Testbench του Protocol Management Module, Αρχείο: pmm/pmm_tb.vhd	44
5.7	Αρχιτεκτονική του Controller Host Interface	45
5.8	Testbench του Controller Host Interface, Αρχείο: chi/main_tb.vhd	45
5.9	Αρχιτεκτονική του Medium Interface Layer	46
5.10	Κύκλωμα χρονοπρογραμματισμού πλαισίων	47
5.11	Αρχιτεκτονική υπομονάδας εντοπισμού σφαλμάτων	48
5.12	Testbench για το Scheduling (Output Layer), Αρχείο: mil/scheduler_tb.vhd	49

5.13	Testbench για το Frame ID Generator (Output Layer), Αρχείο: mil/fid_gen_tb.vhd . .	49
5.14	Testbench για το MIL (Output Layer), Αρχείο: mil/mil_main_tb.vhd	49
5.15	Testbench #1 για το MIL (Input Layer), Αρχείο: mil/mil_main_tb.vhd	50
5.16	Testbench #2 για το MIL (Input Layer), Αρχείο: mil/mil_main_tb.vhd	50
5.17	Αρχιτεκτονική του Wakeup Pattern Generator	51
5.18	Πλαίσιο δεδομένων του Wakeup Pattern Generator	52
5.19	Testbench του Wakeup Pattern Generator, Αρχείο: wupg/wupg_main_tb.vhd	52
5.20	Αρχιτεκτονική του Wakeup Pattern Recognition	52
5.21	Διάγραμμα καταστάσεων της FSM στο WUPR	53
5.22	Testbench του WUPR, Αρχείο: wup_recognition/wupr_main_tb.vhd	53
5.23	Αρχιτεκτονική του Microtick Counter	54
5.24	Πλαίσιο δεδομένων του Clock Synchronization	55
5.25	Μηχανή πεπερασμένων καταστάσεων για το ψηφίο εγκυρότητας	55
5.26	Testbench #1 του Microtick Counter, Αρχείο: microtick_counter_tb.vhd	56
5.27	Testbench #2 του Microtick Counter (changed global_value), Αρχείο: microtick_counter_tb.vhd	56
5.28	Ροή εκτέλεσης κατά τη δοκιμή του συστήματος	57
5.29	Testbench του Συστήματος (1/5)	58
5.30	Testbench του Συστήματος (2/5)	58
5.31	Testbench του Συστήματος (3/5)	59
5.32	Πλαίσιο δεδομένων του Wakeup Pattern Generator	59
5.33	Testbench του Συστήματος (4/5)	60
5.34	Testbench του Συστήματος (5/5)	60
5.35	Πλαίσιο δεδομένων του Clock Synchronization	61
5.36	Λογική Σύνθεση & Χαρτογράφηση Τεχνολογίας από περιγραφή RTL	62
5.37	Κατανάλωση Ισχύος	63
5.38	Κατανάλωση Δυναμικής Ισχύος	63
BΠ.1	Κυκλωματικό διάγραμμα στυλ σχεδιασμού #1	105
BΠ.2	Κυκλωματικό διάγραμμα στυλ σχεδιασμού #2	106
ΓΠ.1	Η θέση των FPGAs στις συσκευές ημιαγωγών	109
ΓΠ.2	Δομή παραδοσιακού FPGA	110
ΔΠ.1	Κωδικοποίηση ανά μπλοκ	113
ΕΠ.1	Το μοντέλο αναφοράς OSI	115

Κατάλογος πινάκων

5.1	Πίνακας καταστάσεων της FSM στο PMM	42
5.2	Κωδικοποίηση Καταστάσεων	44
5.3	Κωδικοποίηση Commands από τον Host	44
5.4	Πίνακας αληθείας του κυκλώματος χρονοπρογραμματισμού πλαισίων	47
5.5	Πίνακας καταστάσεων της FSM στο WUPR	53
5.6	Χρησιμοποίηση πόρων στην ιεραρχία	62
5.7	Σύνοψη χρησιμοποίησης πόρων	62
BΠ.1	Αντιστοιχία σημάτων CHI-PMM	106
BΠ.2	Αντιστοιχία σημάτων CHI-CS	106
BΠ.3	Αντιστοιχία σημάτων PMM-CS	106
BΠ.4	Αντιστοιχία σημάτων CHI-MIL	106
BΠ.5	Αντιστοιχία σημάτων CS-MIL	106
BΠ.6	Αντιστοιχία σημάτων WUPR-MIL	106
BΠ.7	Αντιστοιχία σημάτων WUPG-MIL	106
BΠ.8	Αντιστοιχία σημάτων WUPG-PMM	106
ΔΠ.1	Κωδικοποίηση 4B/5B: Αντιστοιχίες ακολουθιών δεδομένων	114
ΔΠ.2	Κωδικοποίηση 4B/5B: Αντιστοιχίες ακολουθιών ελέγχου	114

1. Εισαγωγή

1.1 Το πρόβλημα προς επίλυση

Τα τελευταία χρόνια, τα ενσωματωμένα συστήματα έχουν γνωρίσει τρομερή ανάπτυξη, χάρη στη σμίκρυνση του τρανζίστορ και στο χαμηλότερο κόστος κατασκευής. Ως ενσωματωμένο σύστημα ορίζεται η συσκευή εκείνη, η οποία συνδυάζει υλικό και λογισμικό, και πιθανόν κι άλλα ηλεκτρικά και μηχανικά μέρη, για την πραγματοποίηση μιας συγκεκριμένης λειτουργίας. Αυτό είναι και το κύριο σημείο, που τα διαφοροποιεί απ' τους υπολογιστές γενικού σκοπού, οι οποίοι μπορούν να χρησιμοποιηθούν για οποιαδήποτε εφαρμογή, ανάλογα με τις επιθυμίες του τελικού χρήστη.

Τα ενσωματωμένα συστήματα βασίζονται στην επιτυχία τους στους μικροεπεξεργαστές. Ο πρώτος μικροεπεξεργαστής κατασκευάστηκε απ' την Intel το 1971. Στην πραγματικότητα, η παραγγελία αφορούσε μια σειρά ολοκληρωμένων κυκλωμάτων για την επιτέλεση των λειτουργιών μιας αριθμομηχανής. Η Intel όμως παρέδωσε ένα και μόνο chip, έναν μικροεπεξεργαστή, ο οποίος ενσωμάτωνε όλες τις επιθυμητές λειτουργίες.[1]

Ο μικροεπεξεργαστής γνώρισε μεγάλη επιτυχία στην επόμενη δεκαετία, καθώς τα πλεονεκτήματά του δεν άργησαν να αναγνωριστούν. Εκτός από την ενσωμάτωση των λειτουργιών σε ένα και μόνο chip, διευκολύνονται οι λειτουργικές αναβαθμίσεις και η συντήρηση, χάρη στη χρήση του λογισμικού. Επίσης, βελτιώνεται η προστασία των πνευματικών δικαιωμάτων. Συγκεκριμένα, το υλικό μπορεί να παραμένει ευάλωτο σε επιθέσεις, αλλά το λογισμικό μπορεί να αποκρυφθεί από κακόβουλους τελικούς χρήστες.[2]

Οι παραπάνω λόγοι αιτιολογούν το γεγονός ότι, τα ενσωματωμένα συστήματα συναντώνται σε πολυάριθμες εφαρμογές στην καθημερινότητά μας, ακόμη κι αν αυτό δε γίνεται άμεσα αντιληπτό από εμάς. Χαρακτηριστικά παραδείγματα αποτελούν ο φούρνος μικροκυμάτων, η τηλεόραση, ο εκτυπωτής, οι αναγνώστες πιστωτικών καρτών (POS), και οι ιατρικές συσκευές παρακολούθησης κρίσιμων παραμέτρων.

Συνήθως, ένα ενσωματωμένο σύστημα αποτελεί συστατικό ενός μεγαλύτερου συστήματος. Για παράδειγμα, τα σύγχρονα οχήματα περιέχουν πολλά και διαφορετικής φύσεως ενσωματωμένα συστήματα, τα οποία είναι γνωστά ως Electronic Control Units (ECUs). Όπως κι ένα κοινό ενσωματωμένο σύστημα, κάθε ECU σε ένα αυτοκίνητο, είναι υπεύθυνο για την εκτέλεση μιας συγκεκριμένης λειτουργίας. Για παράδειγμα, κάποιες κοινές λειτουργίες στα σύγχρονα οχήματα περιλαμβάνουν τον έλεγχο του κινητήρα, τον έλεγχο φρένων, το αυτοματοποιημένο παρκάρισμα, τον εντοπισμό πεζών με αυτοματοποιημένο φρενάρισμα, κ.λ.π.

Όμως, τα περισσότερα απ' τα ECUs είναι αλληλοεξαρτώμενα, άρα χρειάζεται να επικοινωνούν μεταξύ τους, προκειμένου να πραγματοποιήσουν επιτυχώς τη λειτουργία τους. Αυτό επιτυγχάνεται με τη χρήση ενός πρωτοκόλλου επικοινωνίας. Συνήθη χρησιμοποιούμενα πρωτόκολλα αποτελούν εκείνα, που είναι πυροδοτούμενα από γεγονότα (event-triggered), όπως το Controller Area Network (CAN). Όμως, η ανάγκη για ενσωμάτωση περισσότερων εφαρμογών, αλλά και εφαρμογών με κρίσιμο παράγοντα την ασφάλεια αύξησαν τις απαιτήσεις για περισσότερη αιτιοκρατία/προβλεψιμότητα και αξιοπιστία. Αυτό οδήγησε στην υιοθέτηση χρονοπυροδοτούμενων (time-triggered) πρωτοκόλλων, όπως είναι το Ethernet και το FlexRay.

Όσο ελκυστικά κι αν είναι τα νέα πρωτόκολλα, η υιοθέτησή τους έρχεται και με ένα κόστος. Συγκεκριμένα, τα χρονο-πυροδοτούμενα πρωτόκολλα χρησιμοποιούν μεγαλύτερο εύρος ζώνης και καταναλώνουν περισσότερη ενέργεια, με αποτέλεσμα να υποβαθμίζεται σημαντικά η συνολική απόδοση του συστήματος.

Μια λύση στο πρόβλημα αυτό θα ήταν η επέκταση των δυνατοτήτων του Communication Controller σε κάθε κόμβο ECU, προκειμένου να βελτιωθεί συνολικά η ποιότητα και η αποτελεσματικότητα του συστήματος. Με αυτόν τον τρόπο, το Communication Controller θα είχε τη δυνατότητα να εκτελέσει λειτουργίες, οι οποίες κατά κανόνα θα εκτελούνταν στον μικροεπεξεργαστή του αντίστοιχου κόμβου. Έτσι, το Communication Controller απελευθερώνει μέρος φόρτου εργασίας απ' την επεξεργαστική λογική, με αποτέλεσμα την απαλοιφή αρκετών σημείων συμφόρησης. Εκτός αυτού, η συγκεκριμένη λύση ανοίγει πόρτες για επιπλέον δυνατότητες, οι οποίες δεν υπήρχαν στο πρότυπο του πρωτοκόλλου FlexRay, όπως η εφεδρική λειτουργία και η ανεκτικότητα σφαλμάτων.

Το κύριο εμπόδιο της παραπάνω προσέγγισης είναι το γεγονός ότι, οι περισσότερες συσκευές στο εμπόριο, ειδικά αυτές που βασίζονται σε κάποια πλατφόρμα, δεν έχουν την ευελιξία, ώστε να δεχτούν αρκετή παραμετροποίηση για την επέκταση των δυνατοτήτων του Communication Controller. Γίνεται λοιπόν αναγκαία η χρήση μιας προγραμματιζόμενης λογικής, π.χ. με τη χρήση της τεχνολογίας Field-Programmable Gate Array (FPGA).[3]

1.2 Στόχος της Εργασίας

Ο στόχος της παρούσας διπλωματικής εργασίας είναι ο σχεδιασμός και η ανάπτυξη ενός Communication Controller, σύμφωνα με τις προδιαγραφές που αναφέρθηκαν παραπάνω. Η ανάπτυξη του συστήματος βασίστηκε στην τεχνολογία FPGA, και πραγματοποιήθηκε με τη χρήση της γλώσσας περιγραφής υλικού VHDL. Το συγκεκριμένο Communication Controller υπακούει στο πρωτόκολλο FlexRay και ενσωματώνει επιπλέον λειτουργίες για τη βελτιστοποίηση της απόδοσης και αποτελεσματικότητας του συστήματος. Μερικές από αυτές τις λειτουργίες είναι ο συγχρονισμός με τους υπόλοιπους κόμβους (ECUs) στο δίκτυο, ο εντοπισμός σφαλμάτων, και η κωδικοποίηση/αποκωδικοποίηση δεδομένων.

1.3 Διάρθρωση της Διπλωματικής Εργασίας

Κεφάλαιο 2: Σε αυτό το κεφάλαιο παρουσιάζονται τα ενσωματωμένα συστήματα. Συγκεκριμένα, επεξηγούνται χρησιμοποιούμενοι όροι (π.χ. Κυβερνοφυσικά Συστήματα, ΔΤΠ, κλπ.), αναφέρονται πιθανές εφαρμογές, προκλήσεις στη σχεδίαση, και τα κοινά χαρακτηριστικά ενσωματωμένων συστημάτων.

Κεφάλαιο 3: Εδώ παρουσιάζεται το πρωτόκολλο FlexRay, εστιάζοντας στους λόγους δημιουργίας του, στα κύρια χαρακτηριστικά του, και στις χρησιμοποιούμενες δομές. Ακόμη, απεικονίζεται η γενική αρχιτεκτονική ενός κόμβου, εντός ενός δικτύου FlexRay.

Κεφάλαιο 4: Εδώ παρουσιάζεται πιο αναλυτικά η προσέγγιση στη λύση του προβλήματος, που παρουσιάστηκε στην ενότητα 1.1, με βάση το paper [3]. Προτείνονται κάποιες βελτιώσεις όσον αφορά τη λειτουργικότητα του πρωτοκόλλου FlexRay, παρουσιάζεται μια πιθανή υλοποίηση Communication Controller με βάση την προσέγγιση στη λύση του προβλήματος, και ακολουθούν δοκιμές για την αξιολόγηση της συγκεκριμένης προσέγγισης.

Κεφάλαιο 5: Το συγκεκριμένο κεφάλαιο είναι το κύριο αντικείμενο της διπλωματικής εργασίας, δηλαδή η υλοποίηση ενός Communication Controller, με βάση τις προδιαγραφές

που παρουσιάζονται στο κεφάλαιο 4. Περιλαμβάνονται μια γενική αρχιτεκτονική του συστήματος, οι αρχιτεκτονικές των επιμέρους υπομονάδων, και περιγράφεται συνοπτικά η λειτουργία τους. Εκτός αυτού ακολουθούν και δοκιμές, για την επιβεβαίωση της ορθής λειτουργίας των υπομονάδων, αλλά και ολόκληρου του συστήματος. Το κεφάλαιο ολοκληρώνεται με τη σύνθεση του συστήματος και μια ανασκόπηση χρησιμοποίησης των πόρων του FPGA.

Κεφάλαιο 6: Σε αυτό το κεφάλαιο διατυπώνονται τα συμπεράσματα μετά την εκπόνηση της διπλωματικής εργασίας, και στη συνέχεια δίνονται πιθανές μελλοντικές βελτιώσεις για τη συνέχεια της ανάπτυξης του Communication Controller.

Παράρτημα Α: Εδώ περιέχεται ο κώδικας VHDL, που χρησιμοποιήθηκε για την περιγραφή των υπομονάδων και του συστήματος, αλλά και αντίστοιχα testbenches για την εξομοίωσή τους.

Παράρτημα Β: Στο συγκεκριμένο παράρτημα αναγράφονται ορισμένα αξιοσημείωτα σχόλια σχετικά με τον κώδικα, που παρουσιάστηκε στο παράρτημα Α.

Παράρτημα Γ: Σε αυτό το παράρτημα παρουσιάζονται τα βασικά της τεχνολογίας FPGA, μιας και το FPGA είναι η χρησιμοποιούμενη πλατφόρμα για την υλοποίηση της παρούσας διπλωματικής εργασίας.

Παράρτημα Δ: Εδώ αναφέρεται η κωδικοποίηση ανά μπλοκ, και η κωδικοποίηση 4B/5B, η οποία χρησιμοποιείται στην υλοποίηση του Communication Controller.

Παράρτημα Ε: Σε αυτό το παράρτημα παρουσιάζεται το μοντέλο OSI, για τοποίο γίνεται μια αναφορά στο κεφάλαιο 3. Εξηγείται ο σημαντικός του ρόλος στο χώρο των δικτύων, περιγράφονται τα αποτελούμενα επίπεδα, και πώς προέκυψαν αυτά.

2. Ενσωματωμένα Συστήματα

2.1 Χρησιμοποιούμενοι Όροι

Μέχρι τα τέλη του 1980, η επεξεργασία της πληροφορίας πραγματοποιούνταν κυρίως από υπολογιστές mainframe και ογκώδεις οδηγούς μαγνητικής ταινίας. Αργότερα, ο Weiser επινόησε τον όρο «Διάχυτος Υπολογισμός», που σημαίνει η δυνατότητα πραγματοποίησης υπολογισμών οπουδήποτε και οποιαδήποτε χρονική στιγμή. Επίσης, πρόβλεψε την ενσωμάτωση των υπολογιστών σε προϊόντα, που θα τους επέτρεπε να γίνουν αόρατοι στο περιβάλλον.

Εξαιτίας της ευρείας χρήσης μικρών συσκευών σε συνδυασμό με το διαδίκτυο, πολλά απ' αυτά τα οράματα έχουν γίνει ήδη πραγματικότητα. Η χρήση αυτών των συσκευών είναι πια διάχυτη με την έννοια ότι, παίζει σημαντικό ρόλο σε πολλές πτυχές της καθημερινότητάς μας. Το ίδιο θα μπορούσε να ειπωθεί και για την τεχνητή νοημοσύνη.

Οι ραγδαίες εξελίξεις στον τομέα της μικρογραφίας επέτρεψαν την επεξεργασία των πληροφοριών με το περιβάλλον, με τη χρήση των υπολογιστών. Αυτό το είδος επεξεργασίας των πληροφοριών αποκαλείται και *Ενσωματωμένο Σύστημα* (Embedded System). Σύμφωνα με τον Marwedel, τα Ενσωματωμένα Συστήματα ορίζονται ως εκείνα τα συστήματα επεξεργασίας πληροφοριών, τα οποία είναι ενσωματωμένα σε προϊόντα.

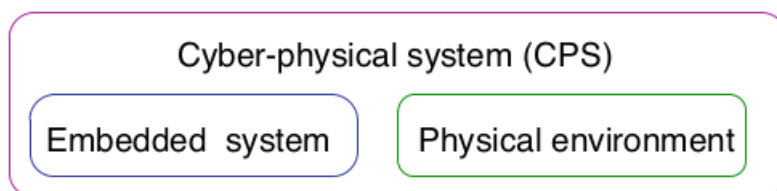
Χαρακτηριστικά παραδείγματα, που περιέχουν ενσωματωμένα συστήματα, αποτελούν τα αυτοκίνητα, τα τρέινα, τα αεροπλάνα και ο εξοπλισμός τηλεπικοινωνιών. Αξίζει να σημειωθεί ότι, όλα τα ενσωματωμένα συστήματα, ανεξάρτητα της εφαρμογής τους, έχουν ένα μεγάλο αριθμό κοινών χαρακτηριστικών, όπως περιορισμοί πραγματικού χρόνου, βαθμός αξιοπιστίας, και απαιτήσεις αποτελεσματικότητας.

Στα ενσωματωμένα συστήματα, μεγάλο ρόλο παίζει η *σύνδεση* με τα φυσικά συστήματα, η οποία θα μπορούσε να αποδοθεί απ' την ακόλουθη παραπομπή:

«Το Ενσωματωμένο Λογισμικό είναι ενσωματωμένο με τις φυσικές διεργασίες. Το τεχνικό πρόβλημα είναι η διαχείριση του χρόνου και ο ταυτοχρονισμός στα υπολογιστικά συστήματα.»

Παρ' όλα αυτά, αυτή η σύνδεση έχει τονιστεί ακόμη περισσότερο τα τελευταία χρόνια, χάρη στον όρο *Κυβερνοφυσικά Συστήματα* (Cyber-Physical Systems). Τα Κυβερνοφυσικά Συστήματα ορίζονται ως εκείνα τα συστήματα, που επιτυγχάνουν την ενσωμάτωση των υπολογισμών με τις φυσικές διεργασίες.

Η έμφαση αυτής της σύνδεσης έχει μεγάλη σημασία, καθώς πολλές φορές αγνοείται στον κόσμο των εφαρμογών, που τρέχουν σε διακομιστές, προσωπικούς υπολογιστές, και κινητά τηλέφωνα. Ο όρος Κυβερνοφυσικό Σύστημα αποτελείται από ένα ενσωματωμένο σύστημα (το τμήμα επεξεργασίας πληροφοριών) και ένα (δυναμικό) φυσικό περιβάλλον. Αυτό φαίνεται καλύτερα στην παρακάτω εικόνα.



Σχήμα 2.1: Η σχέση μεταξύ των Ενσωματωμένων και των Κυβερνοφυσικών Συστημάτων

Η διασύνδεση και η συνεργασία είναι δύο όροι, οι οποίοι έχουν τονιστεί ρητά απ' την Ευρωπαϊκή Επιτροπή:

«Τα Κυβερνοφυσικά Συστήματα αφορούν την επόμενη γενιά των ενσωματωμένων συστημάτων πληροφορίας και τηλεπικοινωνιών, τα οποία είναι διασυνδεδεμένα, και συνεργάζονται μέσω του Διαδικτύου των Πραγμάτων (Internet of Things), παρέχοντας στους πολίτες και τις επιχειρήσεις ένα μεγάλο εύρος καινοτόμων εφαρμογών και υπηρεσιών.»

Απ' την παραπάνω παραπομπή συνεπάγεται ότι, εκτός απ' τη συσχέτιση του κυβερνοκόσμου με τον φυσικό, μεγάλη αξία έχει και η έννοια της επικοινωνίας. Αυτή η έννοια τονίζεται καλύτερα απ' τον ορισμό του *Διαδικτύου των Πραγμάτων* (Internet of Things).

Συγκεκριμένα, το Διαδίκτυο των Πραγμάτων περιγράφει τη διάχυτη παρουσία της ποικιλίας των συσκευών (π.χ. αισθητήρες, ενεργοποιητές, και κινητά τηλέφωνα), τα οποία μέσω ειδικών πρωτοκόλλων, είναι σε θέση να αλληλεπιδράσουν και να συνεργαστούν για την επίτευξη κοινών στόχων.

Το Διαδίκτυο των Πραγμάτων είναι ο όρος, που γεφυρώνει τους αισθητήρες (οι οποίοι είναι σε θέση να αντιληφθούν πληροφορίες στο διαδίκτυο) με τους ενεργοποιητές (οι οποίοι ελέγχονται μέσω του διαδικτύου).

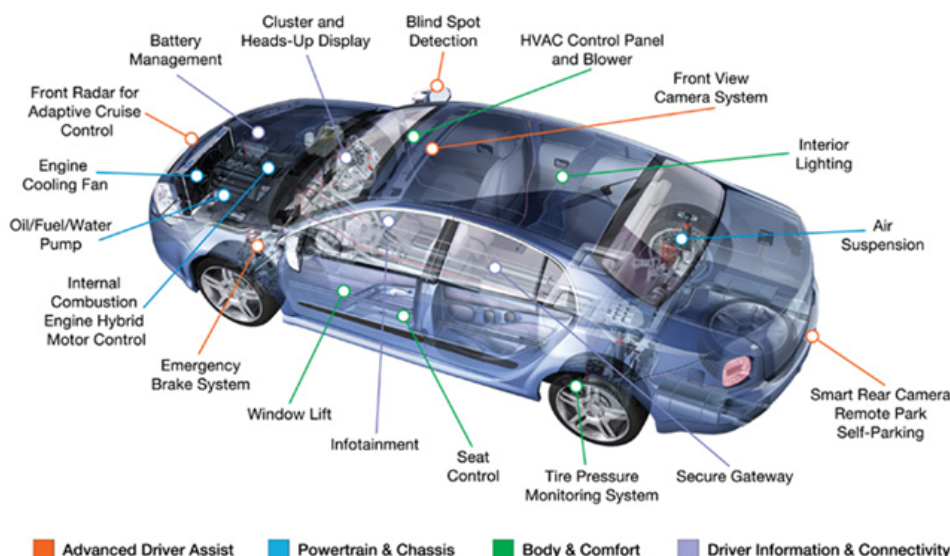
Επιπλέον, η εξέλιξη της τεχνολογίας του Διαδικτύου των Πραγμάτων έχει οδηγήσει σε αυτό, που αποκαλούμε *Industry 4.0*. Το Industry 4.0 αποσκοπεί σε μια πιο ευέλικτη παραγωγή, όπου ο κύκλος ζωής του προϊόντος υποστηρίζεται απ' το ΔτΠ, ξεκινώντας απ' τη φάση της σχεδιάσής του.

2.2 Εφαρμογές

Υπάρχει μια πληθώρα εφαρμογών επεξεργασίας πληροφοριών, όπου θα μπορούσαν να χρησιμοποιηθούν τα Κυβερνοφυσικά Συστήματα και το Διαδίκτυο των Πραγμάτων. Μιας και μια αναλυτική λίστα θα ήταν αρκετά εξαντλητική, εδώ αναφέρονται ορισμένα χαρακτηριστικά παραδείγματα ανά τομέα εφαρμογής.

Μεταφορές και Κινητικότητα:

- **Αυτοκινητοβιομηχανία:** Η πώληση αυτοκινήτων στις ανεπτυγμένες χώρες προϋποθέτει αυτά να περιέχουν ένα μεγάλο αριθμό ηλεκτρονικών συστημάτων. Μερικά απ' αυτά είναι τα συστήματα έλεγχου αερόσακου, έλεγχου μηχανής, πλοήγησης, κλιματισμού, και αντικλεπτικής προστασίας. Επίσης, μεγάλο επίτευγμα αποτελεί και η αυτόνομη οδήγηση, η οποία συναντάται όλο και πιο συχνά σε αυτοκινητόδρομους. Τα ενσωματωμένα συστήματα έχουν τη δυνατότητα να βελτιώσουν την άνεση, να αποτρέψουν ατυχήματα, και να μετριάσουν τις επιπτώσεις στο περιβάλλον.



Σχήμα 2.2: Ενσωματωμένα συστήματα στα αυτοκίνητα

Πηγή: [Modern Car Technology](#)

- **Αεροηλεκτρονική:** Η συνεχώς αυξανόμενη τιμή των αεροπλάνων οφείλεται στο γεγονός ότι, ενσωματώνουν όλο και περισσότερο εξοπλισμό επεξεργασίας πληροφορίας, όπως συστήματα ελέγχου πτήσης, αποτροπής σύγκρουσης, αυτόματου πιλότου κλπ. Σε αυτή την περίπτωση ο σημαντικότερος παράγοντας, που λαμβάνεται υπόψη, είναι η αξιοπιστία των ενσωματωμένων συστημάτων.
- **Καινοτομίες στην κινητικότητα:** Η χρήση της τεχνολογίας πληροφοριών και τηλεπικοινωνιών επιτρέπει κι άλλους τρόπους μετακίνησης. Χαρακτηριστικό παράδειγμα κυβερνοφυσικού συστήματος αποτελούν τα ηλεκτρικά scooter (e-scooters). Αυτός ο τρόπος μετακίνησης επιτρέπει τη διάνυση πολλών χιλιομέτρων, χωρίς την απαίτηση ιδιαίτερων δεξιοτήτων. Ακόμη, το Διαδίκτυο των Πραγμάτων και τα Κυβερνοφυσικά Συστήματα αποτελούν σημαντικά συστατικά για τις υπηρεσίες ταξί.

Μηχανολογία: Ο μηχανικός εξοπλισμός και ο εξοπλισμός κατασκευής έχουν συνδυαστεί με ενσωματωμένα συστήματα εδώ και δεκαετίες. Η χρήση της τεχνολογίας του Διαδικτύου των Πραγμάτων ή/και των Κυβερνοφυσικών Συστημάτων επιτρέπει τη βελτιστοποίηση της γραμμής παραγωγής ακόμη περισσότερο. Ο συντονισμός της γραμμής παραγωγής (logistics) καθιστά εφικτή την αυτοματοποίηση πολλών διεργασιών στο εργοστάσιο. Για παράδειγμα, η ταυτοποίηση μέσω ραδιοσυχνοτήτων (RFID) σε συνδυασμό με το διαδίκτυο διευκολύνει την αναγνώριση των προϊόντων παγκοσμίως.

Έξυπνα Κτήρια: Η επεξεργασία της πληροφορίας μπορεί να βελτιώσει την άνεση στα κτήρια, να μειώσει την κατανάλωση ενέργειας, και να αυξήσει το επίπεδο ασφαλείας. Υπάρχει η τάση για την ενσωμάτωση λειτουργιών πολλαπλών διαφορετικών συσκευών (π.χ. κλιματισμός, φωτισμός, έλεγχος πρόσβασης, κλπ.) σε ένα και μόνο σύστημα για κεντρικοποιημένο έλεγχο. Ορισμένες βελτιστοποιήσεις, που θα μπορούσαν να εφαρμοστούν, αποτελούν ο αυτοματοποιημένος έλεγχος του φωτισμού και του κλιματισμού, ανάλογα με την παρουσία ή όχι ανθρώπων στο εκάστοτε δωμάτιο. Ακόμη, λίστες άδειων δωματίων θα μπορούσαν να εμφανίζονται στην είσοδο του κτηρίου σε περιπτώσεις επείγουσών καταστάσεων. Με αυτό τον τρόπο, θα μπορούσε να εξοικονομηθεί αρκετή ενέργεια. Ένας κύριος στόχος λοιπόν, είναι η κατασκευή κτηρίων μηδενικής ενέργειας, που θα συμβάλλουν στη μείωση του διοξειδίου του άνθρακα στην ατμόσφαιρα και τη μείωση της υπερθέρμανσης του πλανήτη.

Υγεία: Η σπουδαιότητα των ιατρικών συσκευών αυξάνεται όλο και περισσότερο, ιδιαίτερα στις κοινωνίες ηλικιωμένων ανθρώπων. Οι εξελιγμένοι αισθητήρες σε συνδυασμό με τη μηχανική μάθηση έχουν τη δυνατότητα να παρέχουν ακριβή παρακολούθηση κρίσιμων τιμών, και να προβλέπουν ή/και να εντοπίζουν πιθανές ασθένειες. Εκτός αυτού, οι θεραπείες μπορούν να βελτιωθούν με τη χρήση της τεχνητής νοημοσύνης, παρέχοντας εξατομικευμένη αγωγή.

Τηλεπικοινωνίες: Τα κινητά τηλέφωνα αποτελούν μια από τις πιο ταχέως αναπτυσσόμενες αγορές τα τελευταία χρόνια. Το σήμα ραδιοσυχνότητας, η ψηφιακή επεξεργασία σήματος, και η χαμηλή κατανάλωση ενέργειας αποτελούν κεντρικά σημεία έρευνας. Οι τηλεπικοινωνίες είναι αναμφισβήτητα ένας τομέας του Διαδικτύου των Πραγμάτων.

Καταναλωτικά Προϊόντα: Ο εξοπλισμός αναπαραγωγής ήχου και βίντεο αποτελεί έναν κύριο τομέα της βιομηχανίας ηλεκτρικών ειδών. Όλο και περισσότερη επεξεργασία πληροφορίας ενσωματώνεται σε τέτοιου είδους συσκευές, καθώς οι εξελιγμένες τεχνικές ψηφιακής επεξεργασίας σήματος προσφέρουν καινούργιες υπηρεσίες και βελτιωμένη ποιότητα. Τα κινητά τηλέφωνα, οι τηλεοράσεις, και οι παιχνιδοκονσόλες αποτελούν ειδικές περιπτώσεις ενσωματωμένων συστημάτων. Σε τέτοιου είδους συσκευές θα πρέπει να εκπληρωθούν απαιτήσεις πραγματικού χρόνου και η επαρκής λειτουργία με περιορισμένους πόρους (π.χ. ηλεκτρική ενέργεια, και ρυθμό μετάδοσης).

2.3 Προκλήσεις

Δυστυχώς, ο σχεδιασμός των ενσωματωμένων συστημάτων και η ενσωμάτωσή τους σε Κυβερνοφυσικά Συστήματα και στο Διαδίκτυο των Πραγμάτων αντιμετωπίζει αρκετές προκλήσεις. Το είδος αυτών των προκλήσεων προσδιορίζεται απ' το είδος της εφαρμογής. Παρακάτω αναγράφονται συνήθεις δυσκολίες, που μπορεί να εμφανιστούν, κατά το σχεδιασμό ενός ενσωματωμένου συστήματος.

- Τα Κυβερνοφυσικά Συστήματα και τα συστήματα του Διαδικτύου των Πραγμάτων οφείλουν να είναι **άξια εμπιστοσύνης**. Ένα σύστημα θεωρείται άξιο εμπιστοσύνης, εφόσον προσφέρει την προσδοκώμενη υπηρεσία με υψηλή πιθανότητα, και δεν προκαλεί ζημιές. Ο κύριος λόγος για την ανάγκη εγγύησης της αξιοπιστίας είναι ότι, αυτά τα συστήματα είναι άμεσα συνδεδεμένα με το περιβάλλον, και γι' αυτό το λόγο η λειτουργία τους μπορεί να έχει άμεσες επιπτώσεις σε αυτό. Η εξασφάλιση της αξιοπιστίας προϋποθέτει την ύπαρξη των παρακάτω:

Ασφάλεια (Security): Ως ασφάλεια της πληροφορίας μπορεί να οριστεί η διατήρηση της εμπιστευτικότητας, της ακεραιότητας, και της διαθεσιμότητας της πληροφορίας. Η διασύνδεση των συστατικών ενός συστήματος Διαδικτύου των Πραγμάτων επιτρέπει αρκετά επικίνδυνες επιθέσεις, όπως η κλοπή, η παραποίηση, και η καταστροφή. Αυτές οι επιθέσεις αποτελούν σοβαρό ζήτημα για το σχεδιασμό και την ανάπτυξη συστημάτων ΔτΠ. Η επίτευξη της ασφάλειας ενός συστήματος προϋποθέτει την ύπαρξη των ακόλουθων:

- Ταυτοποίηση οντότητας
- Ασφαλή πρόσβαση στο διαδίκτυο
- Ασφαλή επικοινωνία
- Ασφαλή αποθήκευση (εμπιστευτικότητα και ακεραιότητα των δεδομένων)
- Ασφάλεια περιεχομένου (περιορισμός πρόσβασης)

Εμπιστευτικότητα (Confidentiality): Είναι η ιδιότητα ότι, η πληροφορία δεν έχει αποκαλυφθεί σε κάποια μη εξουσιοδοτημένη οντότητα ή διεργασία. Συνήθως, η εμπιστευτικότητα εξασφαλίζεται με τη χρήση κρυπτογράφησης.

Σιγουριά (Safety): Ως σιγουριά ορίζεται η απουσία του μη αποδεκτού κινδύνου, δηλαδή πρόκληση ζημιάς στην υγεία των ανθρώπων. Η ζημιά μπορεί να προκύψει άμεσα ή έμμεσα ως αποτέλεσμα καταστροφής ιδιοκτησίας ή του περιβάλλοντος.

Αξιοπιστία (Reliability): Αυτός ο όρος αναφέρεται στη δυσλειτουργία του συστήματος, ως αποτέλεσμα μη ορθής λειτουργίας, με βάση τις προδιαγραφές, που ορίστηκαν κατά τη φάση του σχεδιασμού. Η αξιοπιστία είναι η πιθανότητα μη αποτυχίας του συστήματος εντός ενός συγκεκριμένου χρονικού διαστήματος. Για την αξιολόγηση της αξιοπιστίας, δε λαμβάνονται υπόψη κακόβουλες επιθέσεις, αλλά μόνο συμβάντα που προκύπτουν κατά την κανονική λειτουργία του συστήματος.

Επισκευασιμότητα (Repairability): Είναι η πιθανότητα επιδιόρθωσης ενός αποτυχημένου συστήματος σε ένα ορισμένο χρονικό διάστημα.

Διαθεσιμότητα (Availability): Είναι η πιθανότητα ότι το σύστημα θα είναι διαθέσιμο. Για να επιτευχθεί υψηλή διαθεσιμότητα, απαιτείται η εξασφάλιση υψηλής αξιοπιστίας και επισκευασιμότητας.

- Πολλές φορές παρατηρείται μια **αναντιστοιχία μεταξύ των μοντέλων στο φυσικό χώρο και των μοντέλων στον κυβερνοχώρο**. Ακολουθούν μερικά παραδείγματα:
 - Αρκετά κυβερνοφυσικά συστήματα οφείλουν να πληρούν **περιορισμούς πραγματικού χρόνου**. Η μη εκπλήρωση αυτών των περιορισμών θα μπορούσε να οδηγήσει σε υποβάθμιση της ποιότητας, που παρέχεται απ' το σύστημα (π.χ. ποιότητα του ήχου), ή σε πρόκληση ζημιάς στον χρήστη (π.χ. αν κάποιο όχημα δε λειτουργεί εντός προδιαγραφών).
 - Πολλά ενσωματωμένα συστήματα είναι **υβριδικά**, δηλαδή περιλαμβάνουν επεξεργασία αναλογικού και ψηφιακού σήματος. Δυστυχώς, οι αναλογικές ποσότητες δεν μπορούν να αναπαρασταθούν με ακρίβεια. Γι' αυτό το λόγο, χρησιμοποιείται μια προσέγγιση, ευελπιστώντας στη λήψη ενός επιθυμητού αποτελέσματος.
 - Πληθώρα κυβερνοφυσικών συστημάτων αποτελούν **βρόχους ελέγχου**, δηλαδή λειτουργούν με ανάδραση. Όμως, έχει αποδειχθεί ότι η συνεχής δειγματοληψία σήματος δεν είναι και τόσο αποτελεσματική, και μπορεί να οδηγήσει σε μεγάλη σπατάλη ενέργειας.
 - Οι **παραδοσιακές γλώσσες προγραμματισμού** δεν είναι ο καλύτερος τρόπος περιγραφής ταυτόχρονων και χρονισμένων συστημάτων.
- Ένα άλλο σημαντικό ζήτημα είναι η **αποτελεσματική χρήση των πόρων**. Αρχικά, αυτή προϋποθέτει την αναγνώριση των πόρων που χρειάζονται. Οι μετρικές που θα πρέπει να ληφθούν υπόψη, κατά την αξιολόγηση της αποτελεσματικότητας χρησιμοποίησης των πόρων, είναι οι ακόλουθες:
 - Ενέργεια
 - Χρόνος Εκτέλεσης
 - Μέγεθος Κώδικα
 - Βάρος
 - Κόστος

- Αξίζει να σημειωθεί ότι, προκύπτουν και αρκετές **επιπτώσεις πέρα απ' τα τεχνικά ζητήματα**, που θα πρέπει να ληφθούν επίσης υπόψη. Αυτές αφορούν την κοινωνία, τη νομιμότητα, την ανθρωπότητα, και το περιβάλλον.
 - Η ενσωμάτωση πολλαπλών μονάδων από διαφορετικούς κατασκευαστές εκφέρει το ζήτημα της ευθύνης. Καταβάλλονται ήδη προσπάθειες για τη διευθέτηση τέτοιου είδους ζητημάτων, όσον αφορά τα αυτόνομα (self-driving) αυτοκίνητα.
 - Τα κοινωνικά ζητήματα αποτελούνται απ' τις επιπτώσεις που έχουν οι συσκευές πληροφοριών στην κοινωνία. Γι' αυτό το λόγο, επινοήθηκε και ο όρος *Κυβερνο-φυσικο-κοινωνικά Συστήματα* (Cyber-Physical-Social Systems). Δυστυχώς, αυτά τα ζητήματα εντοπίζονται πολύ καιρό αργότερα απ' την άφιξη μιας νέας τεχνολογίας.
 - Τα ανθρώπινα ζητήματα αφορούν το σχεδιασμό διεπαφών φιλικών προς τον χρήστη, λαμβάνοντας υπόψη τις αρχές αλληλεπίδρασης ανθρώπου-υπολογιστή (Human Computer Interaction).
 - Οι επιπτώσεις στην υπερθέρμανση του πλανήτη και η παραγωγή απορριμμάτων θα πρέπει να βρίσκονται σε αποδεκτά όρια. Το ίδιο ισχύει και για τη χρησιμοποίηση των πόρων.
- Τα κυβερνοφυσικά συστήματα και τα συστήματα του ΔτΠ συνήθως αποτελούνται από ετερογενή συστατικά υλικού και λογισμικού από διαφορετικούς παρόχους, τα οποία θα πρέπει να λειτουργούν σε μεταβαλλόμενα περιβάλλοντα. Όμως, αυτή η **ετερογένεια** εκφέρει αρκετές προκλήσεις για τη σωστή συνεργασία μεταξύ αυτών των συστατικών.

2.4 Κοινά Χαρακτηριστικά

Στα ενσωματωμένα συστήματα, στα κυβερνοφυσικά συστήματα, και στα συστήματα του Διαδικτύου των Πραγμάτων, εντοπίζονται αρκετά κοινά χαρακτηριστικά, άσχετα απ' τον τομέα εφαρμογής τους. Μερικά απ' αυτά αναγράφονται παρακάτω:

- Τα κυβερνοφυσικά συστήματα και τα συστήματα του ΔτΠ χρησιμοποιούν **αισθητήρες** και **ενεργοποιητές**, για να συνδέσουν τα ενσωματωμένα συστήματα με το περιβάλλον. Όσον αφορά τα συστήματα του ΔτΠ, αυτά συνδέονται και στο διαδίκτυο.
- Τυπικά, τα ενσωματωμένα συστήματα είναι **αντιδραστικά**. Αυτό σημαίνει ότι, βρίσκονται σε συνεχή αλληλεπίδραση με το περιβάλλον και εκτελούν ενέργειες με ρυθμό, που καθορίζεται απ' το περιβάλλον. Τα αυτόματα θα ήταν ένας ιδανικός τρόπος μοντελοποίησης αυτών των συστημάτων. Αντίθετα, η χρήση μαθηματικών συναρτήσεων θα ήταν ένα μη κατάλληλο μοντέλο περιγραφής.
- Τα ενσωματωμένα συστήματα είναι συνήθως σχεδιασμένα για την εκτέλεση μιας συγκεκριμένης εφαρμογής. Για παράδειγμα, οι επεξεργαστές που τρέχουν ένα λογισμικό ελέγχου σε ένα αυτοκίνητο, θα τρέχουν πάντα το ίδιο λογισμικό, και δεν είναι δυνατή η τροποποίησή του. Αυτό συμβαίνει για δύο λόγους:
 1. Η εκτέλεση επιπρόσθετων προγραμμάτων θα καθιστούσε εκείνα τα συστήματα λιγότερο αξιόπιστα.
 2. Η εκτέλεση επιπρόσθετων προγραμμάτων είναι εφικτή, μόνο αν υπάρχουν μη χρησιμοποιούμενοι πόροι (π.χ. μνήμη). Κανονικά δε θα πρέπει να υπάρχουν μη χρησιμοποιούμενοι πόροι σε ένα αποτελεσματικό σύστημα.

- Τα περισσότερα ενσωματωμένα συστήματα δε χρησιμοποιούν ποντίκια, πληκτρολόγια, και μεγάλες οθόνες ως διεπαφές χρήστη, αλλά μια **αφιερωμένη διεπαφή χρήστη**, όπως πλήκτρα, πετάλια κλπ. Γι' αυτό το λόγο, ο χρήστης αδυνατεί να αναγνωρίσει την επεξεργασία της πληροφορίας που εμπλέκεται. Αυτό είναι συνεπές με την έννοια του **αόρατου υπολογιστή**.

Το παρακάτω σχήμα απεικονίζει τις διαφορές μεταξύ προσωπικών υπολογιστών ή κέντρων δεδομένων και των ενσωματωμένων συστημάτων.

	Embedded	PC-/server-like
	Frequently heterogeneous very compact	Mostly homogeneous not compact (x86, etc.)
Architectures		
x86 compatibility	Less relevant	Very relevant
Architecture fixed?	Rarely	Yes
Models of computation (MoCs)	C+multiple models (data flow, discrete events, ...)	Mostly von Neumann (C, C++, Java)
Optimization objectives	Multiple (energy, size, ...)	Average performance dominates
Safety-critical?	Possibly	Usually not
Real-time relevant	Frequently	Hardly
Apps. known at design time	Yes, for real-time systems	Only some (e.g., WORD)

Σχήμα 2.3: Διαφορές μεταξύ προσωπικών υπολογιστών και ενσωματωμένων συστημάτων

Στην πράξη, κανένα ενσωματωμένο σύστημα δεν κατέχει όλα τα χαρακτηριστικά, που προαναφέρθηκαν. Οπότε, θα μπορούσε να δοθεί ένας ανανεωμένος ορισμός για τα ενσωματωμένα συστήματα:

Τα συστήματα επεξεργασίας πληροφοριών, τα οποία κατέχουν τα περισσότερα απ' τα κοινά χαρακτηριστικά που αναφέρθηκαν, αποκαλούνται ενσωματωμένα συστήματα.[4]

3. Το πρωτόκολλο FlexRay

3.1 Η δημιουργία του πρωτοκόλλου FlexRay

Η γέννηση του πρωτοκόλλου FlexRay ξεκίνησε από μια έρευνα, που διεξάχθηκε απ' τις αυτοκινητοβιομηχανίες. Αυτή η έρευνα αποσκοπούσε στη διερεύνηση του αν τα τωρινά πρωτόκολλα επικοινωνίας (π.χ. CAN, TTCAN, TCN, κλπ.), θα είχαν τη δυνατότητα να ικανοποιήσουν τις απαιτήσεις μελλοντικών εφαρμογών στις επόμενες δεκαετίες. Το πόρισμα της έρευνας ήταν ότι, τα χρησιμοποιούμενα πρωτόκολλα δε θα ήταν σε θέση να ικανοποιήσουν τις προδιαγραφές των εφαρμογών. Παρακάτω αναγράφονται οι λόγοι ανά πρωτόκολλο επικοινωνίας.

- **CAN**
 - Δε διαθέτει αρκετό ρυθμό μετάδοσης για νέες εφαρμογές.
 - Η παροχή ντετερμινισμού και πλεονασμού στη μετάδοση είναι αρκετά δύσκολη.
- **TTCAN**
 - Έχει τους ίδιους περιορισμούς ρυθμοαπόδοσης με το CAN.
 - Έλλειψη υποστήριξης για οπτική μετάδοση, καναλιού μετάδοσης με πλεονασμό, ανεκτικότητας σφαλμάτων, και bus guardian.
- **TTP/C**
 - Η χωρητικότητα της πληροφορίας εντός του πλαισίου είναι αρκετά μικρή.
 - Δεν παρέχει αρκετή ευελιξία, όπως το συνδυασμό σύγχρονων και ασύγχρονων τμημάτων στη μετάδοση, τη χρήση πολλαπλών slots για την αποστολή δεδομένων από κόμβο εντός σύγχρονου τμήματος, δραστηριοποίηση των κόμβων σε μονά, διπλά ή μικτά κανάλια, κλπ.
- **Byteflight**
 - Έχει ελάχιστες λειτουργίες.

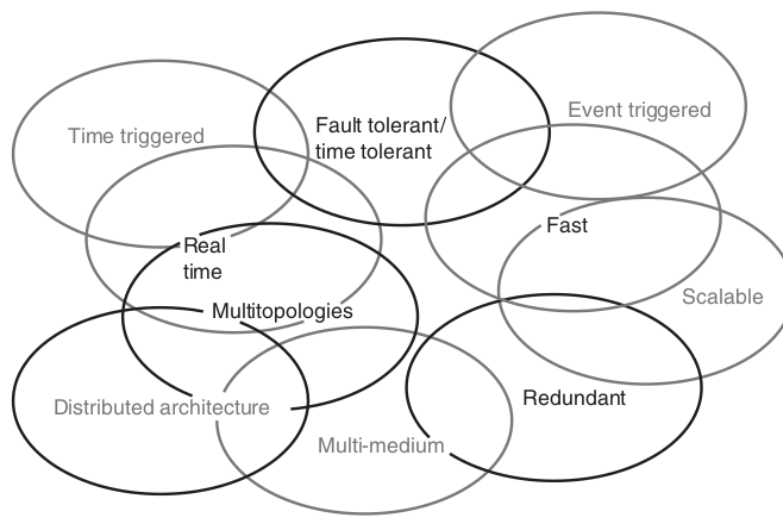
Για τους παραπάνω λόγους, κατασκευαστές αυτοκινήτων, εξοπλισμού, και ηλεκτρονικών συγκρότησαν την κοινοπραξία Flexray. Αυτή είχε ως στόχο να δημιουργήσει ένα πρωτόκολλο επικοινωνίας, το οποίο θα είχε μια ευέλικτη διαμόρφωση, δηλαδή τη δυνατότητα ελέγχου των εφαρμογών σε τέσσερα διαφορετικά επίπεδα:

- Υψηλή ρυθμοαπόδοση σε ψηφιακή μετάδοση δεδομένων, με στόχο τη βελτίωση, ολοκλήρωση ή/και συμπλήρωση των εφαρμογών, οι οποίες περιορίζονταν απ' το ρυθμό μετάδοσης του πρωτοκόλλου CAN.
- Δυνατότητα υλοποίησης λύσεων τύπου «X-by-Wire».
- Παροχή λύσεων με τη χρήση πλεονασμού:

- Επαναλαμβανόμενη αποστολή των ίδιων μηνυμάτων, χάρη στη μεγαλύτερη ρυθμοαπόδοση.
 - Ύπαρξη δύο διαφορετικών καναλιών επικοινωνίας για τη μετάδοση της ίδιας πληροφορίας παράλληλα.
 - Ύπαρξη δύο καναλιών επικοινωνίας για τη βελτιστοποίηση του φυσικού ρυθμού μετάδοσης δεδομένων, που ορίζει το πρωτόκολλο.
- Ικανότητα πραγματοποίησης όλων των μελλοντικών λειτουργιών σε μηχανοκίνητα οχήματα.

3.1.1 Απαραίτητες Προδιαγραφές

Στο παρακάτω σχήμα απεικονίζεται το εύρος των προβλημάτων, που καλείται να επιλύσει το πρωτόκολλο FlexRay.



Σχήμα 3.1: Εύρος προβλημάτων προς επίλυση

Για να διευθετηθούν τα ζητήματα, που παρουσιάζονται στο σχήμα 3.1, η κοινοπραξία έθεσε τις κατάλληλες προδιαγραφές για την επίλυση αυτών. Αυτές οι προδιαγραφές παρουσιάζονται παρακάτω, και αφορούν την επικοινωνία, την τοπολογία, την ασφάλεια και τις εφαρμογές.

Επικοινωνία

- Υψηλή ρυθμοαπόδοση για μετάδοση ψηφιακών δεδομένων (10Mbit/s).
- Μετάδοση δεδομένων σύγχρονα και ασύγχρονα.
- Η μετάδοση των δεδομένων θα πρέπει να είναι ντετερμινιστική, δηλαδή οι χρόνοι καθυστέρησης παράδοσης των μηνυμάτων να είναι γνωστοί και εγγυημένοι.
- Διαφορετικοί και ταυτόχρονοι ρυθμοί μετάδοσης δεδομένων σε διαφορετικούς κόμβους, με εύκολη ανάθεση ζώνης διέλευσης σε κάθε κόμβο του δικτύου.
- Δυνατότητα μεταφοράς στατικού και δυναμικού τμήματος στη μετάδοση δεδομένων.
- Διαμορφώσιμος αριθμός χρονικών slots ανά κόμβο αποστολής και ανά κύκλο ρολογιού.
- Πλεονασμός υλικού και λογισμικού.

- Δυνατότητα μετάβασης συμμετεχόντων σε κατάσταση sleep, και αφύπνισή τους μέσω του δικτύου.
- Διαχείριση κατανάλωσης πόρων απ' τους συμμετέχοντες στο δίκτυο.
- Γρήγορος εντοπισμός σφαλμάτων και επισήμανσή τους.
- Εντοπισμός σφαλμάτων συγχρονισμού με το καθολικό ρολόι (Global Clock).
- Παροχή ανεκτικότητας σε σφάλματα και χρονο-πυροδοτούμενων (time-triggered) υπηρεσιών, μέσω υλικού.
- Περιορισμός σφαλμάτων στο φυσικό επίπεδο μέσω του bus guardian.
- Υποστήριξη αποκεντριοποιημένου bus guardian στο φυσικό επίπεδο.
- Ικανότητα προσθήκης νέων κόμβων, χωρίς να απαιτείται επιπλέον παραμετροποίηση.

Τοπολογία

Προκειμένου να διατεθεί αρκετή ευελιξία στο σχεδιασμό των οχημάτων, το FlexRay θα πρέπει να είναι σε θέση να υποστηρίξει επικοινωνία στις ακόλουθες τοπολογίες:

- Μονού καναλιού
- Διπλού καναλιού
- bus, bus με stubs
- παθητικού και ενεργού αστέρα
- πολλαπλού αστέρα, με δυνατότητα sub-buses
- συνδυασμό όλων των παραπάνω

Ασφάλεια

Για όλα τα συστήματα του τύπου «όλος ο έλεγχος μέσω καλωδίου» (X-by-Wire), το FlexRay θα πρέπει να παρέχει τα παρακάτω:

- Δυνατότητα διαχείρισης πλεονάζουσας επικοινωνίας.
- Ντετερμινιστική πρόσβαση στο δίκτυο.
- Αποφυγή συγκρούσεων πρόσβασης στο bus.
- Πλεονασμός με μεταβλητή γεωμετρία (μονό και διπλό κανάλι, μικτό σύστημα).
- Η επανεκκίνηση ενός κόμβου δεν αρκείται μόνο στην επανεγκατάσταση της επικοινωνίας.
- Διατήρηση της επικοινωνίας με τους άλλους κόμβους, εφόσον αυτή δεν έχει τεθεί σε κίνδυνο.
- Εξασφάλιση ευρωστίας του συστήματος σε παροδικά σφάλματα και εξωτερική ραδιενέργεια.
- Ελαχιστοποίηση έκθεσης του συστήματος σε ραδιενέργεια, ηλεκτρομαγνητική προστασία (EMC), κλπ.

Εφαρμογή

Το FlexRay οφείλει να διαβεβαιώνεται ότι η εφαρμογή:

- είναι πάντα υπεύθυνη για όλες τις αποφάσεις, όσον αφορά την ασφάλεια και τη διαθεσιμότητα του δικτύου.
- λαμβάνει πάντα την τελική απόφαση.
- έχει πάντα το σύστημα επικοινωνίας υπό τον έλεγχό της, κι όχι το αντίθετο.
- διατηρεί τη λήψη δεδομένων όσο το δυνατόν περισσότερο, γιατί κάποιες πληροφορίες είναι κρίσιμες για τη λήψη της σωστής απόφασης.
- παρέχει διαφορετικές καταστάσεις λειτουργίας για κανονική, συνεχή, και περιορισμένη λειτουργία (π.χ. σε περίπτωση παρουσίας σφαλμάτων).
- είναι τέτοια, ώστε οι κόμβοι να μπορούν να παραμετροποιηθούν, για να παραμείνουν ενεργοί για κάποιους κύκλους, χωρίς να λάβουν πλαίσια επικοινωνίας (ή δεδομένων).

3.2 Γενικά Χαρακτηριστικά

Προκειμένου να δοθεί στο σύστημα η κατάλληλη ευελιξία εφαρμογής, αυτό θα πρέπει να είναι ικανό να κάνει τα ακόλουθα:

1. Λειτουργία σε πραγματικό χρόνο, δηλαδή εντός συγκεκριμένων και γνωστών χρονικών διαστημάτων. Με αυτό τον τρόπο, εξασφαλίζεται ότι το φυσικό μέσο επικοινωνίας θα χρησιμοποιείται μόνο από μια οντότητα τη φορά, και δε θα υπάρχει πιθανότητα σύγκρουσης.
2. Επικοινωνία ανά γεγονός, με μεταβλητό ρυθμό πληροφοριών κατ' απαίτηση. Έτσι, χρόνος επικοινωνίας είναι άγνωστος, αλλά κάτω από ένα προκαθορισμένο όριο.

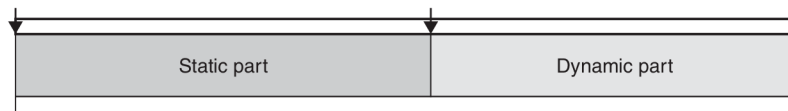
Στο πρωτόκολλο FlexRay, η επικοινωνία πραγματοποιείται σε κύκλους, οι οποίοι ονομάζονται *κύκλοι επικοινωνίας*. Σε αυτούς του κύκλους, η επικοινωνία λαμβάνει μέρος σύγχρονα και ασύγχρονα, εντός χρονοθυρίδων (time slots), οι οποίες είναι προκαθορισμένες απ' τον σχεδιαστή του συστήματος. Συγκεκριμένα, οι κύκλοι επικοινωνίας διαιρούνται σε slots και minislots ίσου χρόνου, στα οποία έχουν αποκλειστική πρόσβαση Κεντρικές Μονάδες Επεξεργασίας (CPUS), για τη μετάδοση δεδομένων.

Ο σχεδιασμός και η διαχείριση του δικτύου είναι ευθύνη του σχεδιαστή, καθώς αυτός καθορίζει τις τιμές, που αφορούν του κύκλους και τις χρονοθυρίδες. Αυτή η αρχή πρόσβασης στο δίκτυο, με βάση προκαθορισμένες χρονοθυρίδες (TDMA), αποτρέπει συγκρούσεις μηνυμάτων στο φυσικό μέσο μετάδοσης.

Το πρωτόκολλο FlexRay περιλαμβάνει δύο μοντέλα σύζευξης:

1. Αρχικοποίηση/Ανάθεση, με βάση το χρόνο (time-triggered).
2. Καθοδήγηση από εξωτερικά γεγονότα (event-triggered).

Προκειμένου να υποστηριχθούν και τα δύο αυτά μοντέλα εντός ενός κύκλου επικοινωνίας, απαιτείται η διαίρεση του κύκλου σε δύο τμήματα, το *στατικό* και το *δυναμικό* (για τα (1) και (2) αντίστοιχα), όπως φαίνεται στο παρακάτω σχήμα.

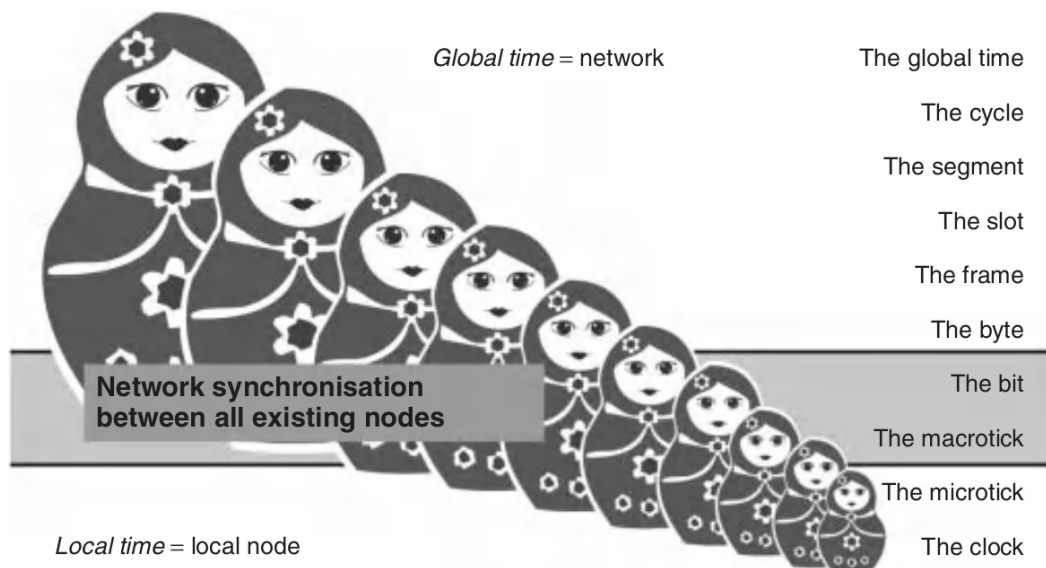


Σχήμα 3.2: Διαίρεση κύκλου σε στατικό και δυναμικό τμήμα

Το FlexRay είναι βασισμένο σε μια διπλή παράλληλη ιεραρχία, καθώς:

- Από άποψη λειτουργικότητας, τα δεδομένα ενθυλακώνονται σε πλαίσια επικοινωνίας, τα οποία στη συνέχεια περιλαμβάνονται σε χρονοθυρίδες. Επίσης, αυτές οι χρονοθυρίδες περιλαμβάνονται σε τμήμα κύκλων επικοινωνίας, και αυτοί οι κύκλοι με τη σειρά τους αποτελούν ένα κανάλι επικοινωνίας.
- Από άποψη χρονισμού, εμπλέκονται Καθολικό Ρολόι (Global Time), Macroticks, Microticks, τοπικό ρολόι, κλπ.

Αυτή η δομή λοιπόν, θα μπορούσε να παρομοιαστεί με την ιεραρχία μιας Ρώσικης μπαμπούσκας, όπως φαίνεται στο παρακάτω σχήμα.



Σχήμα 3.3: Η διπλή παράλληλη ιεραρχία του πρωτοκόλλου FlexRay

Στην επόμενη σελίδα απεικονίζονται τα περισσότερα στοιχεία του πρωτοκόλλου FlexRay. Τα στοιχεία αφορούν όλα τα επίπεδα, απ' το πιο υψηλό (επίπεδο δικτύου) μέχρι και το πιο χαμηλό (επίπεδο τοπικού μικροελεγκτή).

3.3 Κανάλια και Κύκλοι Επικοινωνίας

3.3.1 Κανάλι Επικοινωνίας

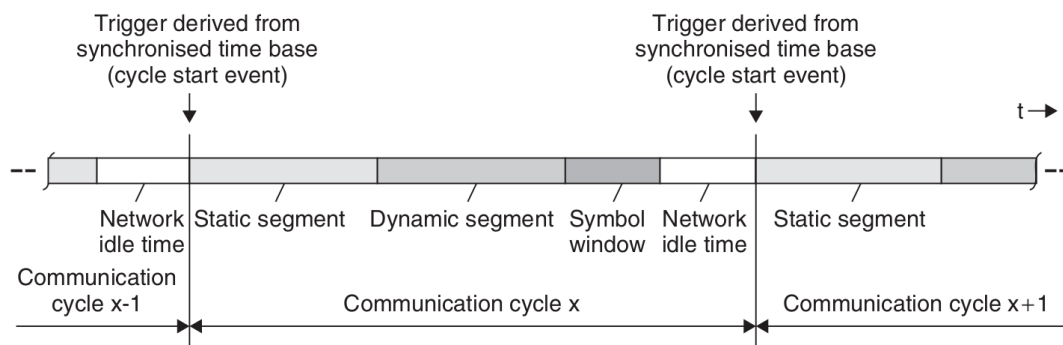
Το κανάλι επικοινωνίας αντιπροσωπεύει το σύνδεσμο μεταξύ των διάφορων συμμετεχόντων στο δίκτυο. Το FlexRay υποστηρίζει δύο κανάλια επικοινωνίας, Α και Β, τα οποία έχουν συγκεκριμένους περιορισμούς εφαρμογής και ευελιξίες. Παρ' όλα αυτά, το σύστημα είναι δυνατόν να λειτουργήσει με ένα και μόνο κανάλι επικοινωνίας. Η σχεδιαστική απόφαση για την ύπαρξη δύο καναλιών πάρθηκε για δύο λόγους:

1. Δυνατότητα συμπλήρωσης, πλεονασμού ή απόσυρσης πληροφορίας απ' το ένα κανάλι στο άλλο.
2. Δυνατότητα προσθήκης μελλοντικών εφαρμογών (π.χ. X-by-Wire).

3.3.2 Κύκλος Επικοινωνίας

Η επικοινωνία οργανώνεται σε κύκλους επικοινωνίας, οι οποίοι είναι επαναλαμβανόμενοι και ίσης χρονικής διάρκειας. Κατά κανόνα, η χρονική διάρκεια ενός κύκλου είναι μια σταθερά, που συνδέεται με την «καθολική ώρα» (Global Time).

Στην παρακάτω εικόνα απεικονίζεται η δομή των κύκλων επικοινωνίας.



Σχήμα 3.5: Η δομή των κύκλων επικοινωνίας

Απ' το σχήμα 3.5 παρατηρούμε ότι, οι κύκλοι επικοινωνίας διαιρούνται σε τέσσερα διαφορετικά μέρη, που λέγονται τμήματα, και επαναλαμβάνονται κυκλικά ανά σταθερά χρονικά διαστήματα. Τα τμήματα αυτά είναι:

- Το **στατικό τμήμα** (Static Segment ή ST).
- Το **δυναμικό τμήμα** (Dynamic Segment ή DT), το οποίο είναι προαιρετικό.
- Το **παράθυρο συμβόλου** (Symbol Window ή SW), το οποίο είναι προαιρετικό.
- Ο **αδρανής χρόνος δικτύου** (Network Idle Time ή NIT), που τερματίζει τον κύκλο. Αυτό είναι ένα χρονικό διάστημα, κατά το οποίο το δίκτυο βρίσκεται σε κατάσταση αδράνειας.

Εξαιτίας των απαιτήσεων εφαρμογών πραγματικού χρόνου, συνήθως η χρονική τιμή για τον κύκλο επικοινωνίας, τίθεται μεταξύ $10\mu s$ και $16ms$.

3.4.2 Στατικά Τμήματα και Slots

Ο σκοπός του στατικού τμήματος είναι η εξασφάλιση ντετερμινιστικής επικοινωνίας υψηλής απόδοσης, ο καθορισμός της σημασίας των μηνυμάτων, και η διαχείριση των κατανεμημένων συστημάτων και του ελέγχου σε κλειστούς βρόχους. Επίσης, η πρόσβαση του στατικού τμήματος στο μέσο ελέγχεται με βάση την αρχή Global Time Division Multiple Access (GTDMA).

Παρακάτω αναγράφονται ορισμένοι κανόνες, οι οποίοι διέπουν τη διαίρεση των στατικών τμημάτων σε slots.

- Όλα τα slots πρέπει να έχουν την ίδια χρονική διάρκεια.
- Όλοι οι κύκλοι επικοινωνίας θα πρέπει να περιλαμβάνουν τον ίδιο αριθμό slots.
- Κάθε slot αναγνωρίζεται από ένα μοναδικό αριθμό (ID).
- Σε περίπτωση επικοινωνίας τύπου διπλού καναλιού, η μορφή των slots και στα δύο κανάλια θα πρέπει να είναι η ίδια (διάρκεια, πλήθος, κλπ.).
- Η έναρξη ενός slot αποφασίζεται απ' την καθολική ώρα (Global Time) του δικτύου.
- Μέσω του Global Time, καθορίζεται η χρονική διάρκεια του slot, με βάση τις ομάδες των συμμετεχόντων (clusters).
- Η έξοδος επιτρέπεται μόνο σε έναν κόμβο ανά slot.
- Τα μοναδικά αναγνωριστικά (IDs) αντιπροσωπεύουν μια μοναδική διεργασία, η οποία ανατίθεται με βάση τους κόμβους εξόδου.
- Είναι απαραίτητη η χρήση τουλάχιστον δύο slots του στατικού τμήματος, κατά τη διάρκεια του συγχρονισμού.

3.4.3 Δυναμικά Τμήματα και Minislots

Αρχικά, η πρόσβαση του δυναμικού τμήματος στο μέσο ελέγχεται με βάση την αρχή Flexible Time Division Multiple Access (FTDMA). Προτού χρησιμοποιηθούν τα δυναμικά τμήματα, αυτά διαιρούνται προκαταβολικά (offline) σε minislots, απ' τον σχεδιαστή του συστήματος. Τα minislots είναι μικρής και ίσης χρονικής διάρκειας. Οι χρονικές θέσεις των minislots απαριθμούνται εντός του δυναμικού τμήματος, και σε σχέση με τον πλήρη κύκλο επικοινωνίας.

Επίσης, τα minislots αναπαριστούν «πιθανές στιγμές» (possible instants), και εξαρτώνται απ' την εκάστοτε εφαρμογή. Δηλαδή, όταν προκύψει ένα μη προσδοκώμενο γεγονός (π.χ. εκτός χρονισμού) προς μετάδοση, τότε μπορεί να χρησιμοποιηθεί ένα minislot, χάρη στη δυναμική φύση του.

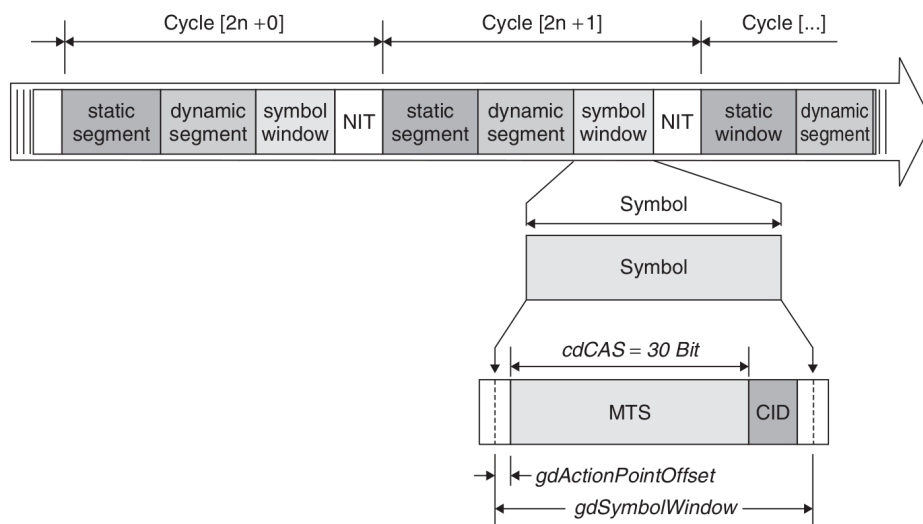
Ακολουθούν κάποιοι κανόνες, που αφορούν τον καθορισμό και τη διαχείριση των minislots:

- Όλοι οι κύκλοι επικοινωνίας περιλαμβάνουν τον ίδιο αριθμό minislots.
- Η χρονική τους διάρκεια θα πρέπει να είναι ίδια σε δύο κανάλια μετάδοσης.
- Η ακολουθία πρόσβασης, κατά τη διάρκεια ενός δυναμικού τμήματος, μπορεί να διαφέρει μεταξύ δύο καναλιών επικοινωνίας.
- Η μοναδική λειτουργία των μηνυμάτων εξόδου είναι στενά συνδεδεμένη με τον κόμβο, δεδομένων το δυναμικό slot και τις παραμέτρους του καναλιού.
- Η έναρξη του δυναμικού τμήματος μπορεί να συνδέεται με διαφορετικές καταστάσεις λειτουργίας, οι οποίες είναι:

- Μικτή Κατάσταση Λειτουργίας, όπου συνυπάρχουν στατικά και δυναμικά τμήματα.
 - Η κατάσταση λειτουργίας, όπου υπάρχουν μόνο δυναμικά τμήματα.
- Όταν ένα στοιχείο επικοινωνίας έχει εκκινηθεί κατά τη διάρκεια ενός προκαθορισμένου minislot, τότε αυτό αποκαλείται δυναμικό τμήμα (dynamic slot).

3.4.4 Το Τμήμα Symbol Window

Το τμήμα Symbol Window είναι προαιρετικό, και χρησιμοποιείται για τη συμπερίληψη του Συμβόλου Ελέγχου Πρόσβασης στο Μέσο (Medium Access Test Symbol ή MTS). Το MTS χρησιμοποιείται για την επιβεβαίωση της ορθής λειτουργίας του bus guardian. Επίσης, έχει την ίδια δομή με το Σύμβολο Αποφυγής Σύγκρουσης (Collision Avoidance Symbol ή CAS). Το μήκος και των δύο συμβόλων είναι 30 bits. Για τον τερματισμό του τμήματος Symbol Window, το MTS ακολουθείται από έναν οριοθέτη αδρανούς καναλιού (CID), όπως φαίνεται και στο παρακάτω σχήμα.

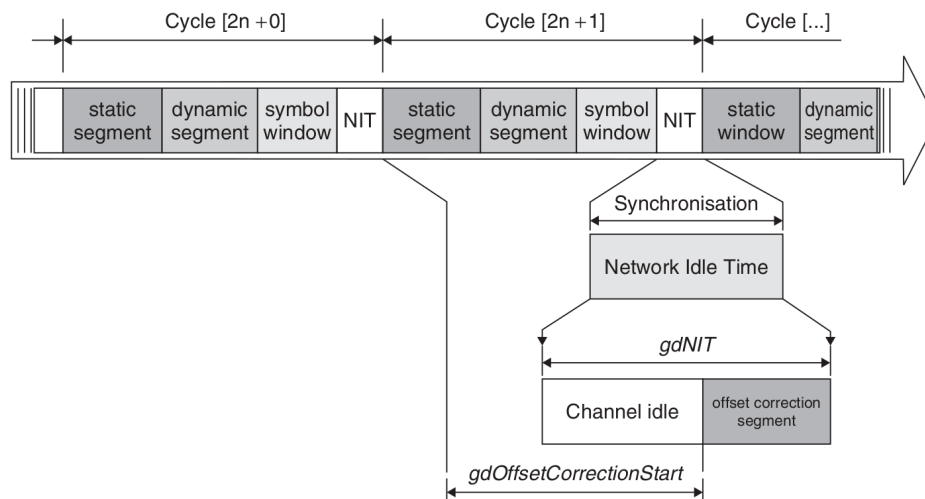


Σχήμα 3.7: Τερματισμός του Symbol Window

3.4.5 Το Τμήμα Network Idle Time

Το συγκεκριμένο τμήμα επισημαίνει τον τερματισμό κάθε κύκλου επικοινωνίας. Κατά τη διάρκεια εκείνου του τμήματος, δεν παρατηρείται καμία δραστηριότητα στο δίκτυο, αποστολής ή λήψης μηνυμάτων, καθώς το δίκτυο βρίσκεται σε κατάσταση αδράνειας.

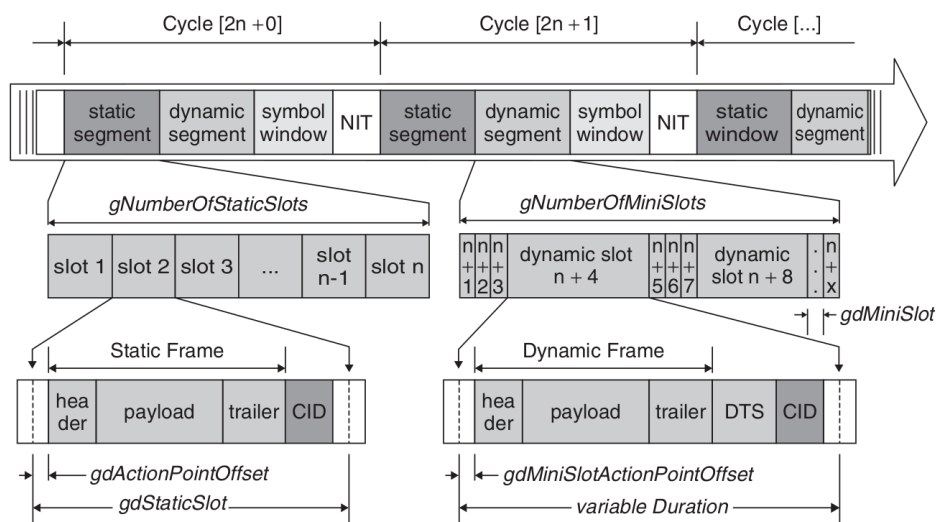
Στην πραγματικότητα όμως, υπάρχει έντονη δραστηριότητα εντός των κόμβων, η οποία δικαιολογεί την ύπαρξη του τμήματος Network Idle Time. Συγκεκριμένα, οι κόμβοι εκμεταλλεύονται το χρόνο απ' το NIT, για να πραγματοποιήσουν απαραίτητους υπολογισμούς συγχρονισμού, και στη συνέχεια να διορθώσουν τη φάση του τοπικού ρολογιού. Αυτές οι ενέργειες φαίνονται στο σχήμα 3.8. Ακόμη, να σημειωθεί ότι, οι διορθώσεις φάσης επηρεάζουν όλους τους συμμετέχοντες του δικτύου. Γι' αυτό το λόγο, η χρονική διάρκεια του τμήματος NIT εξαρτάται από το μέγεθος της συστάδας των κόμβων (cluster).



Σχήμα 3.8: Διόρθωση φάσης τοπικού ρολογιού

3.5 Πλαίσια Επικοινωνίας

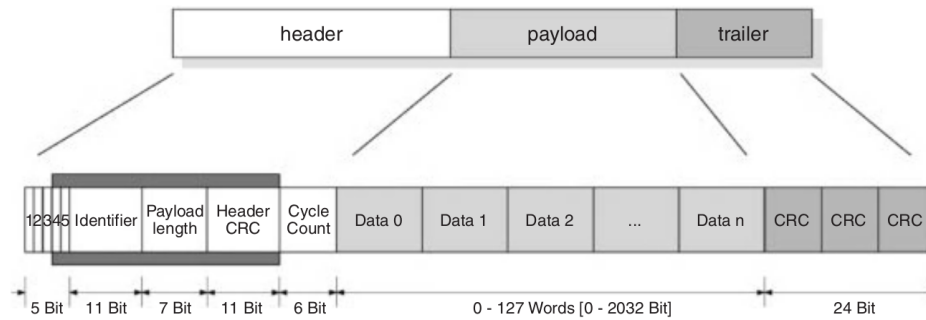
Τα Πλαίσια Επικοινωνίας (Communication Frames) περιέχουν το λογικό φορτίο, και μεταδίδονται σε στατικά ή/και δυναμικά τμήματα, εντός στατικών και δυναμικών slots αντίστοιχα, όπως φαίνεται στο σχήμα 3.9. Τα slots των στατικών τμημάτων και τα minislots των δυναμικών τμημάτων δεσμεύονται από στατικά και δυναμικά πλαίσια επικοινωνίας αντίστοιχα.



Σχήμα 3.9: Μετάδοση πλαισίων επικοινωνίας

3.5.1 Συστατικά Μέρη των Πλαισίων

Τα στατικά και δυναμικά πλαίσια αποτελούνται απ' το *Πεδίο Κεφαλίδας* (Header Field), το *Ωφέλιμο Φορτίο* (Payload), δηλαδή τα δεδομένα, και το *Trailer*, που επισημαίνει το τέλος του πλαισίου. Αυτά τα πεδία απεικονίζονται στο παρακάτω σχήμα.



Σχήμα 3.10: Συστατικά μέρη πλαισίων επικοινωνίας

Πεδίο Κεφαλίδας

Το Πεδίο Κεφαλίδας (Header Field) αποτελείται απ' τα ακόλουθα μέρη:

- **Leading Indicator (5 bits):** Τα δυαδικά ψηφία εντός του Leading Indicator ερμηνεύονται ως εξής:
 - Bit 1:** Δεσμευμένο για μελλοντική χρήση (προς το παρόν είναι 1).
 - Bit 2:** Υποδηλώνει το είδος του περιεχομένου εντός του πεδίου Payload. Το '0' αντιπροσωπεύει κλασικά περιεχόμενα, ενώ το '1' αντιπροσωπεύει ειδικά περιεχόμενα. Στη δεύτερη περίπτωση, υπάρχουν δύο επιλογές, ανάλογα με το αν το πλαίσιο είναι έξοδος στατικού ή δυναμικού τμήματος.
 - Bit 3:** Δηλώνει αν το πλαίσιο επικοινωνίας είναι μηδενικό (null). Αυτό σημαίνει ότι, ο αποστολέας του κόμβου δεν έχει τίποτα να στείλει απ' τον προηγούμενο κύκλο.
 - Bit 4:** Δηλώνει αν το πλαίσιο επικοινωνίας είναι πλαίσιο συγχρονισμού. Σε αυτή την περίπτωση, το πλαίσιο θα πρέπει να χρησιμοποιηθεί για το συγχρονισμό των ρολογιών των κόμβων.
 - Bit 5:** Δηλώνει την έναρξη του πλαισίου.
- **Slot/Frame ID (11 bits):** Αποτελεί το μοναδικό αναγνωριστικό του πλαισίου, και χρησιμοποιείται για να δηλώσει τη θέση του slot, όπου θα μεταδοθεί το πλαίσιο, εντός του στατικού ή του δυναμικού τμήματος. Επίσης, δύο ελεγκτές (controllers) δεν επιτρέπεται να αποστείλουν πλαίσια με το ίδιο αναγνωριστικό (Frame ID) στο ίδιο κανάλι επικοινωνίας.
- **Payload Length (7 bits):** Δηλώνει τον αριθμό των ψηφιολέξεων (ή μήκος των δεδομένων), που περιέχονται εντός του ωφέλιμου φορτίου (payload). Να σημειωθεί ότι, το μήκος έχει διαιρεθεί με το 2, καθώς μια ψηφιολέξη αποτελείται από 2 bytes. Οπότε, το μέγιστο μήκος δεδομένων δεν είναι $2^7 = 128$ λέξεις, αλλά 256. Επίσης, οποιαδήποτε τιμή μεγαλύτερη του 123 θεωρείται σφάλμα.
- **Header CRC (11 bits):** Χρησιμοποιείται για τον έλεγχο της συνέπειας μεταξύ του αριθμού slot και του Frame ID, που λήφθηκαν απ' τον κόμβο-αποστολέα. Έτσι, ο παραλήπτης δε θα προχωρήσει στην επεξεργασία δεδομένων, τα οποία δεν έχουν καμία αξία.
- **Cycle Count (6 bits):** Αναπαριστά τον αριθμό του κύκλου επικοινωνίας για την εκάστοτε μετάδοση. Ακόμη, μπορεί να χρησιμοποιηθεί και ως «δείκτης συνέχειας» της επικοινωνίας. Ο ελεγκτής (controller), που αποστέλλει το πλαίσιο, αυξάνει αυτήν την τιμή αυτόματα σε κάθε κύκλο επικοινωνίας.

Ωφέλιμο Φορτίο

Το συγκεκριμένο πεδίο χρησιμοποιείται για τη μεταφορά χρήσιμων δεδομένων. Παρέχει τη δυνατότητα μεταφοράς έως και 127 λέξεων των 16 δυαδικών ψηφίων, δηλαδή 2023 bits το μέγιστο.

Trailer/CRC

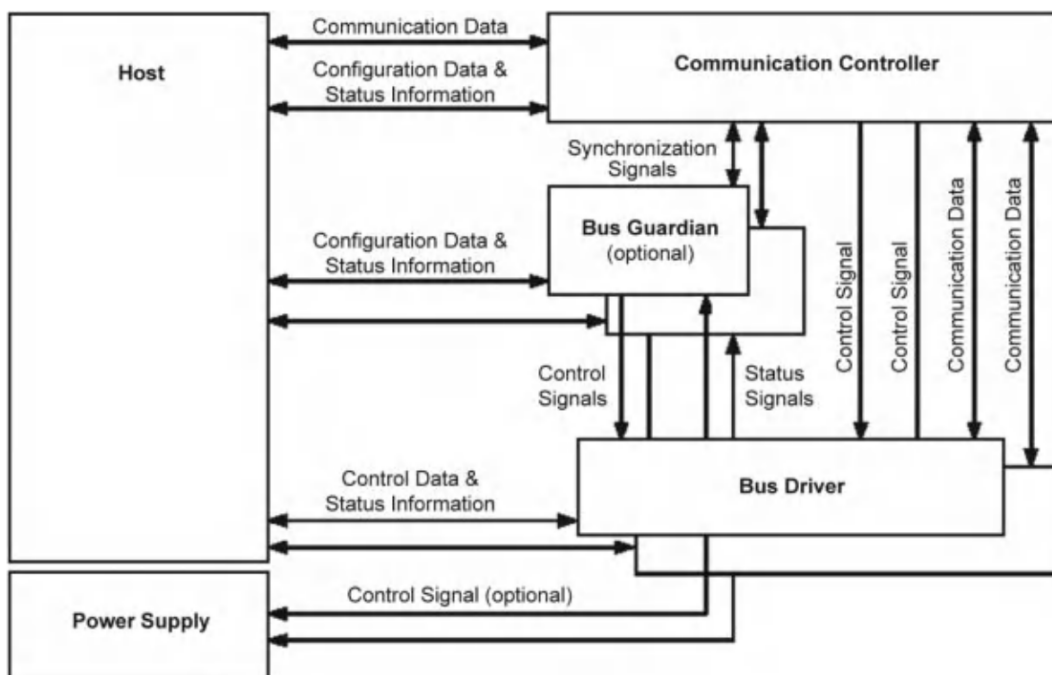
Ο στόχος του συγκεκριμένου πεδίου είναι η προστασία του πλαισίου προς μετάδοση, με τον υπολογισμό της απόστασης Hamming. Συγκεκριμένα, η απόσταση πρέπει να είναι 6 για δεδομένα μήκους 248 bytes και 4 για περισσότερα.

3.6 Η Αρχιτεκτονική ενός Κόμβου

Η τυπική αρχιτεκτονική ενός κόμβου FlexRay αποτελείται από τέσσερα μπλοκ:

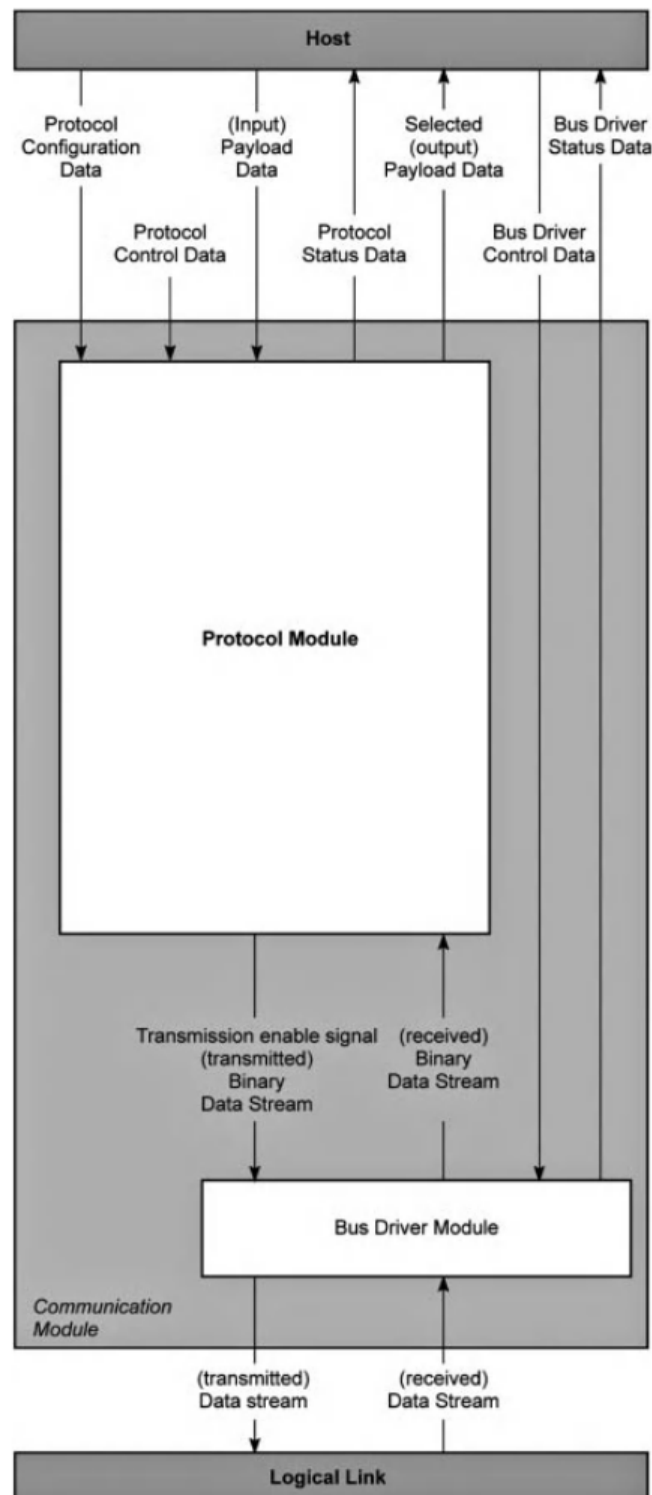
- τον *Host* Μικροελεγκτή, στον οποίο περιέχεται η εφαρμογή και η διαχείριση εφαρμογών του πρωτοκόλλου επικοινωνίας.
- τον *Διαχειριστή Πρωτοκόλλου* (Protocol Manager) ή Communication Controller, ο οποίος κατασκευάζει τους κύκλους, τα τμήματα, κλπ.
- τον *Οδηγό Γραμμής* (Line Driver).
- τον *Bus Guardian*, ο οποίος είναι προαιρετικός.

Στο σχήμα 3.11 φαίνονται οι εσωτερικές και εξωτερικές διασυνδέσεις των προαναφερθέντων συστατικών στοιχείων (ή μπλοκ) σε έναν κόμβο FlexRay. Αξίζει να σημειωθεί ότι, αρκετά συχνά ο διαχειριστής πρωτοκόλλου και ο Host περιλαμβάνονται στο ίδιο ολοκληρωμένο κύκλωμα. Επίσης, ελάχιστες υλοποιήσεις τώρα πια περιέχουν ένα Bus Guardian.



Σχήμα 3.11: Οι διασυνδέσεις εντός ενός κόμβου FlexRay

Το σχήμα 3.12 απεικονίζει με μεγαλύτερη ακρίβεια τις λογικές σχέσεις μεταξύ των μπλοκ, που αναφέρθηκαν προηγουμένως. Επίσης, μελετώντας το σχήμα 3.12 από κάτω προς τα πάνω, αναγνωρίζεται ο διαχωρισμός των επιπέδων 1 και 2 απ' η μια μεριά, και του 7 απ' την άλλη, με βάση το μοντέλο OSI.¹

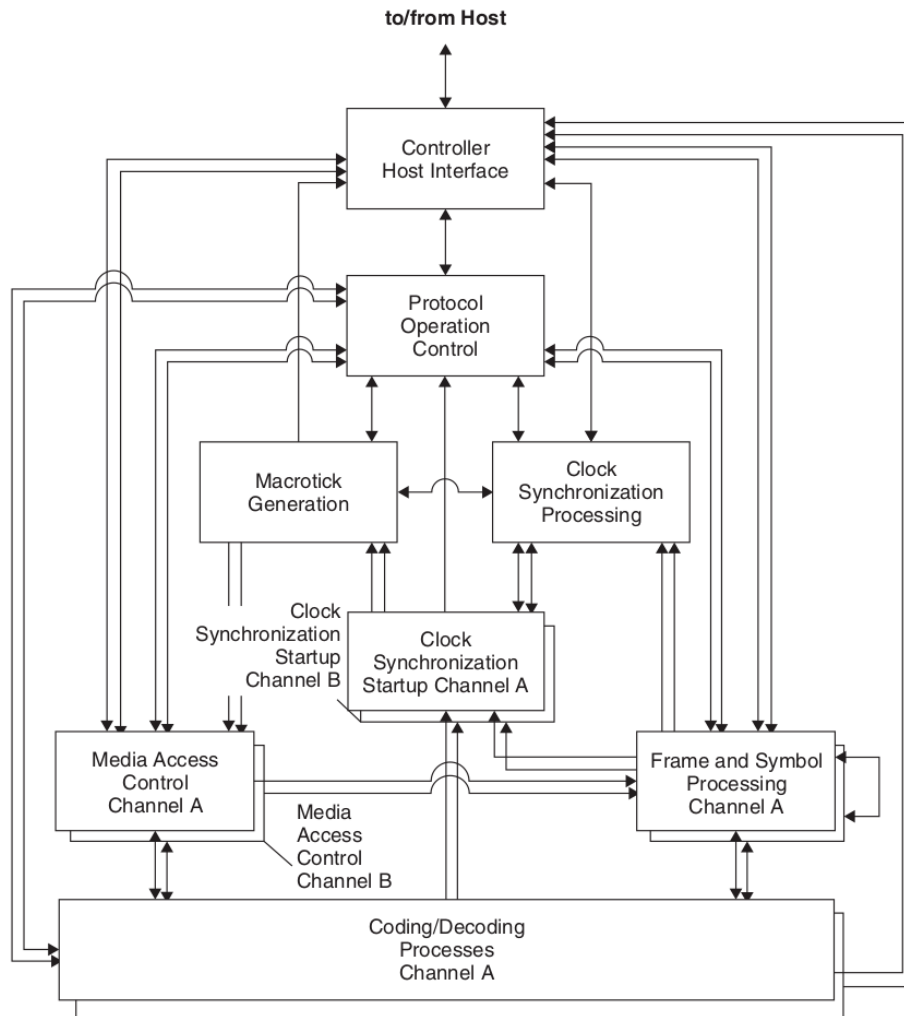


Σχήμα 3.12: Οι λογικές σχέσεις των συστατικών μερών του κόμβου FlexRay

Στο σχήμα 3.13 φαίνεται η εσωτερική αρχιτεκτονική του Ελεγκτή Πρωτοκόλλου (Protocol Controller), όπως προτείνεται απ' τις προδιαγραφές του πρωτοκόλλου. Παρατηρούμε ότι, όλες

¹Βλέπε Παράρτημα Ε.

οι λειτουργίες έχουν υλοποιηθεί δύο φορές, αφού το FlexRay θεωρείται ικανό για την υποστήριξη δύο καναλιών επικοινωνίας.[5]



Σχήμα 3.13: Εσωτερική αρχιτεκτονική του Protocol Controller

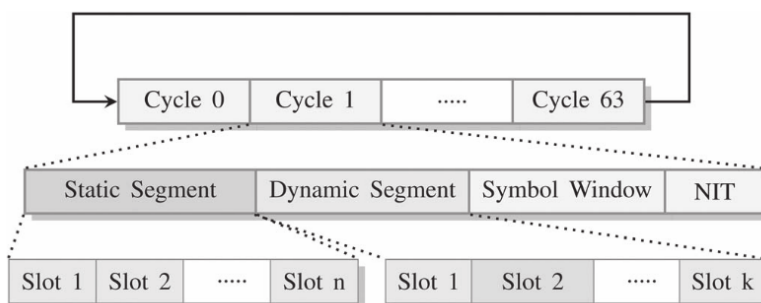
4. Προσέγγιση στη λύση του προβλήματος

Στο παρόν κεφάλαιο παρουσιάζεται η δουλειά των συγγραφέων στο [3], δηλαδή η προσέγγισή τους στη λύση του προβλήματος που περιγράφηκε στην ενότητα 1.1. Συγκεκριμένα, οι συγγραφείς συνιστούν ορισμένες βελτιώσεις στο πρωτόκολλο FlexRay, και επακολούθως παρουσιάζουν ένα Communication Controller με επαυξημένες δυνατότητες, βασισμένο σε FPGA. Επίσης, αποδεικνύουν τη θετική συμβολή της τροποποίησης του Communication Controller, παρουσιάζοντας πειραματικά αποτελέσματα από ρεαλιστικές περιπτώσεις χρήσης με εφαρμογές, οι οποίες εστιάζουν στην ασφάλεια.

4.1 Βελτιώσεις στο πρωτόκολλο FlexRay

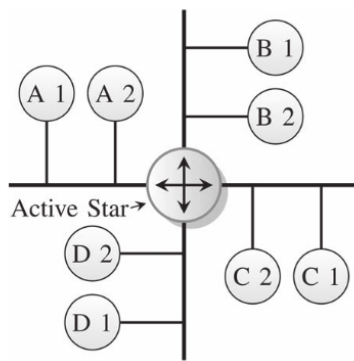
4.1.1 Τοπολογία

Τυπικά κάθε κόμβος/ECU περιέχει ένα ενσωματωμένο ή διακριτό Communication Controller, μέσω του οποίου έχει τη δυνατότητα να αποστείλει ή/και να λάβει πλαίσια δεδομένων. Η μετάδοση των πλαισίων δεδομένων πραγματοποιείται στο/στα slot(s), που του έχουν ανατεθεί, όπως φαίνεται στην παρακάτω εικόνα.

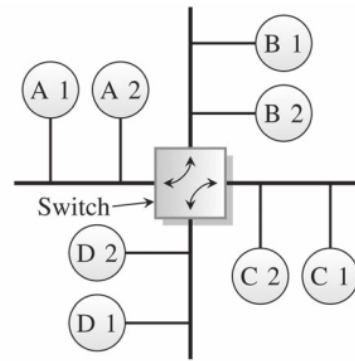


Σχήμα 4.1: Κύκλος επικοινωνίας του FlexRay

Το πρότυπο FlexRay ορίζει την τοπολογία ενεργού αστέρα, όπως φαίνεται στην παρακάτω εικόνα.



Σχήμα 4.2: Τοπολογία ενεργού αστέρα



Σχήμα 4.3: Τοπολογία μεταγωγέα

Το θέμα με τη συγκεκριμένη τοπολογία είναι ότι, ο ενεργός αστέρας που μεταδίδει την πληροφορία από μια διακλάδωση του δικτύου σε όλες τις άλλες διακλαδώσεις. Έτσι, οι κόμβοι μπορούν να μοιραστούν το ίδιο slot μόνο σε διαφορετικούς κύκλους. Αυτός ο σχεδιασμός οδηγεί σε εκτεταμένη σπατάλη του συνολικού εύρους ζώνης του δικτύου.

Αντίθετα, με τη χρήση ενός μεταγωγέα, όπως φαίνεται στο σχήμα 4.3, τα ίδια slots μπορούν να χρησιμοποιηθούν ταυτόχρονα από πολλαπλούς κόμβους. Αυτό συμβαίνει γιατί, η συγκεκριμένη αρχιτεκτονική επιτρέπει την παράλληλη χρησιμοποίηση των διακλαδώσεων, καθώς ο μεταγωγέας αποστέλλει τα πλαίσια μόνο σε διακλαδώσεις, όπου περιέχεται ο παραλήπτης τους.

4.1.2 Χρονοπρογραμματισμός

Έχουν γίνει πολλές μελέτες όσον αφορά το χρονοπρογραμματισμό στο διαμοιραζόμενο μέσο μετάδοσης δεδομένων, αλλά κύριος στόχος όλων είναι η βελτιστοποίηση του αριθμού των χρησιμοποιούμενων slots, με στόχο την εκπλήρωση των δεδομένων απαιτήσεων.

Συγκεκριμένα, έχουν παρουσιαστεί δύο γενικές προσεγγίσεις. Η μία αποσκοπεί στην εύρεση ελάχιστου αριθμού απαιτούμενων κύκλων επικοινωνίας που επιλύουν το πρόβλημα, αλλά με τη χρήση μικρού εύρους ζώνης. Η εναλλακτική προσέγγιση στοχεύει στη μεγιστοποίηση των χρησιμοποιούμενων κύκλων για μέγιστη ευελιξία στο μέλλον.

4.1.3 Βελτιστοποιήσεις σε επίπεδο δικτύου

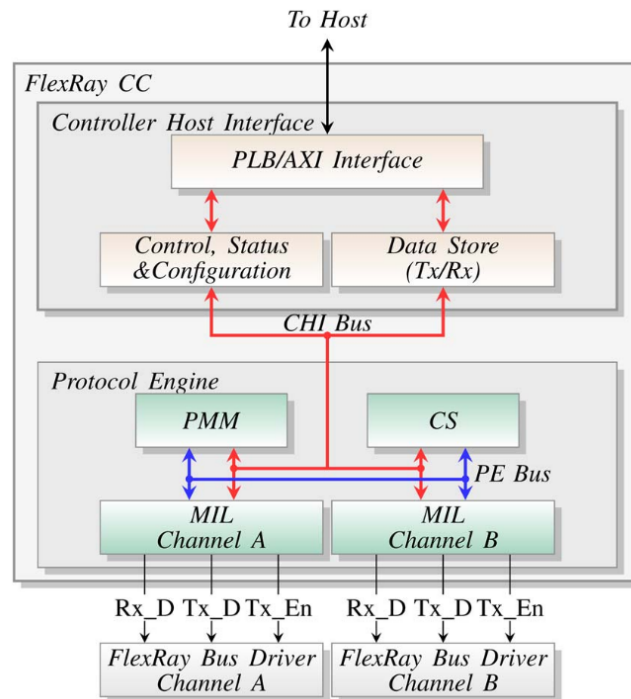
Η επέκταση των δυνατοτήτων του Communication Controller του επιτρέπει να εξυπηρετήσει διάφορα αιτήματα εκ μέρους του ECU. Έτσι, το ECU έχει την ικανότητα να παραμείνει σε κατάσταση «sleep», βελτιώνοντας την αποτελεσματικότητα του εκάστοτε κόμβου.

4.2 Το Communication Controller

Όπως είναι γνωστό και απ' την ενότητα 3.6, σε ένα δίκτυο που υπακούει στο FlexRay, ένας κόμβος αποτελείται από ένα Communication Controller κι ένα host ECU, το οποίο τρέχει μια εξειδικευμένη εφαρμογή. Το Communication Controller φροντίζει για τη συμμόρφωση του κόμβου, με βάση τις προδιαγραφές του πρωτοκόλλου FlexRay. Επίσης, καθιστά εφικτή την επικοινωνία του ECU με τα υπόλοιπα ECUs στο δίκτυο, αποστέλλοντας και λαμβάνοντας δεδομένα εκ μέρους του. Η κατάσταση λειτουργίας του Communication Controller ελέγχεται απ' το host ECU.

4.2.1 Αρχιτεκτονική

Στο παρακάτω σχήμα αναπαρίσταται η αρχιτεκτονική του Communication Controller, το οποίο υλοποίησαν οι ερευνητές.



Σχήμα 4.4: Αρχιτεκτονική τροποποιημένου Communication Controller

Το *Controller Host Interface* (CHI) επικοινωνεί με τον host και διαχειρίζεται εντολές και παραμέτρους για τη ρύθμιση του κόμβου FlexRay. Κατά τη διάρκεια λειτουργίας, το CHI παρέχει συνεχώς στον host την τρέχουσα κατάσταση λειτουργίας και σήματα ελέγχου, ώστε ο host να λάβει διορθωτικές ενέργειες αν αυτό κριθεί απαραίτητο.

Τα *Clock Synchronization* και *Medium Interface Layer* είναι υπομονάδες του *Protocol Engine* (PE), και ελέγχονται απ' τη μονάδα *Protocol Management Module* (PMM). Το PMM βεβαιώνεται ότι οι αλλαγές στην κατάσταση λειτουργίας γίνονται σύμφωνα με το πρωτόκολλο FlexRay.

Το *Medium Interface Layer* (MIL) είναι υπεύθυνο για την αποστολή/λήψη δεδομένων προς/από το Data Bus. Επίσης, πραγματοποιεί την κωδικοποίηση και αποκωδικοποίηση για την αποστολή και λήψη των δεδομένων αντίστοιχα.

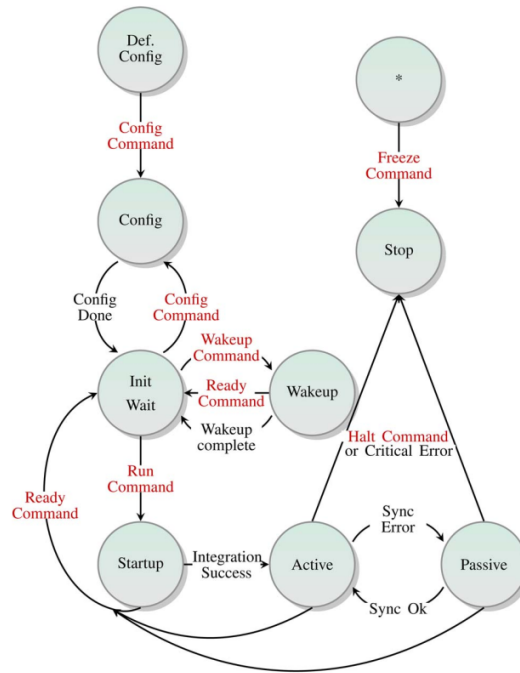
Το *Clock Synchronization* (CS) χρησιμοποιείται για την παραγωγή και τη διάδοση του τοπικού ρολογιού στις υπόλοιπες μονάδες εντός του Communication Controller. Ακόμη, είναι υπεύθυνο για το συγχρονισμό του παρόντος κόμβου με το υπόλοιπο δίκτυο.

Ο συγχρονισμός πραγματοποιείται με τον έλεγχο του αριθμού των Microticks (μέτρο ευθυγράμμισης τοπικού ρολογιού), που αντιστοιχεί σε ένα Macro-tick (μέτρο ευθυγράμμισης καθολικού/global ρολογιού ή ενός cluster του δικτύου FlexRay). Το πρωτόκολλο FlexRay χρησιμοποιεί έναν κατανεμημένο διορθωτικό μηχανισμό για το συγχρονισμό, δηλαδή κάθε κόμβος πραγματοποιεί ατομικά τις κατάλληλες αλλαγές, παρατηρώντας πληροφορίες λαμβανόμενες από τους υπόλοιπους κόμβους.

4.2.2 Υλοποίηση και Βελτιστοποιήσεις

Η κατάσταση του PMM αντικατοπτρίζει την τρέχουσα κατάσταση λειτουργίας του κόμβου. Στο παρακάτω διάγραμμα φαίνονται όλες οι πιθανές καταστάσεις, όπου μπορεί να βρεθεί ο

κόμβος.



Σχῆμα 4.5: Καταστάσεις λειτουργίας του Communication Controller

Η αλλαγή στην κατάσταση λειτουργίας συμβαίνει από εντολές του host ή από καταστάσεις του δικτύου. Ο παρακάτω πίνακας αναγράφει όλες τις εντολές του host, και πώς η κάθε μια απ' αυτές επηρεάζει την κατάσταση λειτουργίας.

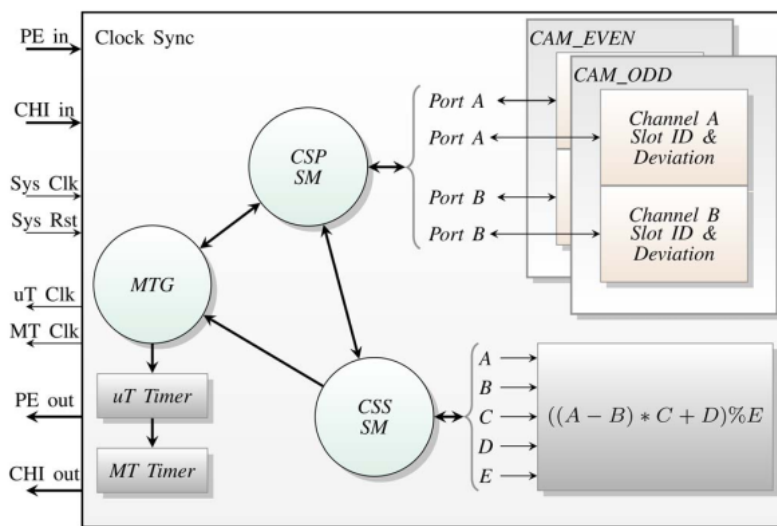
Host Com-mand	Affected States	Final State	Processed at
ALL SLOTS	Active, Passive	no state change	End of cycle
ALLOW COLD-START	All states except Def. Config, Config, Stop	no state change	Immediate
CONFIG	Def. Config, Init Wait	Config	Immediate
CONFIG COM-plete	Config	Init Wait	Immediate
DEFAULT CONFIG	Stop	Def. Config	Immediate
FREEZE	All States	Stop	Immediate
HALT	Active, Passive	Stop	End of cycle
READY	All states except Def. Config, Config, Init Wait, Stop	Init Wait	Immediate
RUN	Init Wait	Start-Up	Immediate
WAKEUP	Init Wait	Wake-Up	Immediate

Σχῆμα 4.6: Εντολές του host

Wakeup State: Κάθε κόμβος σε ένα δίκτυο FlexRay έχει τη δυνατότητα να μεταβεί σε κατάσταση sleep, για να εξοικονομήσει ενέργεια. Όταν ένας κόμβος επιθυμεί να επικοινωνήσει, αρχικά μεταβαίνει σε κατάσταση wakeup. Σε εκείνη την κατάσταση, ο κόμβος αποστέλλει μια ακολουθία συμβόλων, με στόχο να ξυπνήσει τους υπόλοιπους κόμβους. Όταν οι κόμβοι, που είναι σε κατάσταση sleep, αντιληφθούν τη συγκεκριμένη ακολουθία, θα εγκαταλείψουν την κατάσταση sleep, για να ανταποκριθούν σε μελλοντικά αιτήματα.

Startup State: Όταν ο κόμβος μεταβαίνει σε κατάσταση startup, εκκινείται η διεργασία συγχρονισμού. Συγκεκριμένα, στη μονάδα Clock Synchronization ελέγχεται συνεχώς η απόκλιση του ρολογιού σε σχέση με τη χρονική στιγμή λήψης του πλαισίου. Αν η απόκλιση είναι εντός ορίων, ο συγχρονισμός ήταν επιτυχής, κι έτσι ο κόμβος μεταβαίνει σε κατάσταση *active*. Αν όχι, ο κόμβος μεταβαίνει σε κατάσταση *passive*. Καθ' όλη τη διάρκεια λειτουργίας ο κόμβος εναλλάσσεται συνεχώς μεταξύ των καταστάσεων *active* και *sleep*, ανάλογα με την κατάσταση συγχρονισμού.

Παρακάτω απεικονίζεται η απλοποιημένη αρχιτεκτονική της μονάδας Clock Synchronization.



Σχήμα 4.7: Αρχιτεκτονική του Clock Synchronization

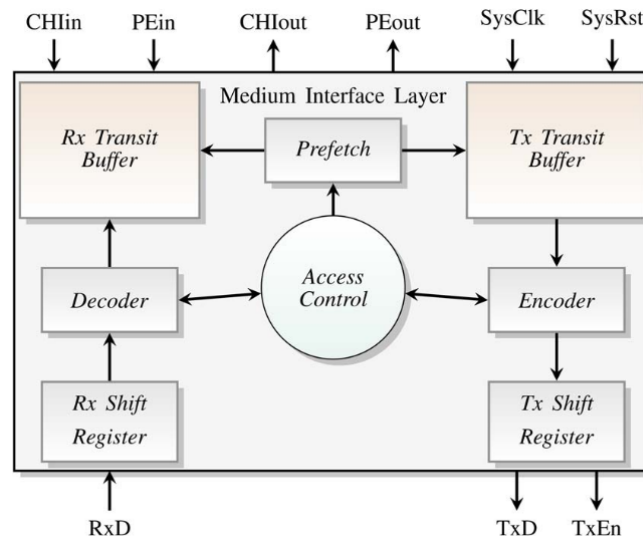
Το *Macro Tick Generator* ή *MTG* είναι η υπομονάδα, που ελέγχει τους μετρητές των κύκλων ρολογιού και των Macro ticks. Επίσης, είναι υπεύθυνη για τη διόρθωση των τιμών της συχνότητας και της φάσης.

Το *Clock Synchronization Process* ή *CSP* φροντίζει για την αρχικοποίηση κατά την έναρξη, αλλά και για τη μέτρηση και την αποθήκευση των τιμών απόκλισης. Εκτός αυτού, υπολογίζει τις τιμές για τη φάση και τη συχνότητα.

Το *Clock Synchronization Startup* ή *CSS* είναι υπεύθυνο για την εκκίνηση ενός σύγχρονου ρολογιού κατά το συγχρονισμό του κόμβου ή την εγκαθίδρυση επικοινωνίας με ένα αδρανές δίκτυο.

Στην ουσία, η μηχανή πεπερασμένων καταστάσεων CSP ελέγχει τη λειτουργία της μονάδας CS, αλληλεπιδρώντας με τις άλλες δύο υπομονάδες, τις MTG και CSS.

Στο παρακάτω σχήμα αναπαρίσται η αρχιτεκτονική της μονάδας Medium Interface Layer (MIL).



Σχήμα 4.8: Αρχιτεκτονική του Medium Interface Layer

Όπως φαίνεται στην παραπάνω εικόνα, το MIL περιέχει ανεξάρτητα buffers για την προσωρινή αποθήκευση των πλαισίων, που αποστέλλονται και λαμβάνονται.

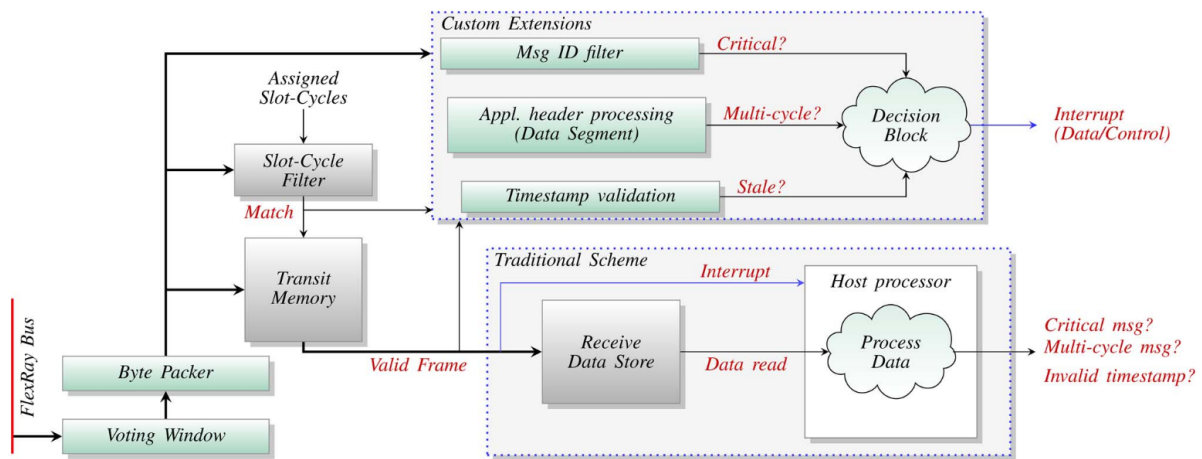
Το Access Control είναι η μηχανή καταστάσεων, που διαχειρίζεται την πρόσβαση στο Data Bus, σύμφωνα με την τρέχουσα τιμή μετρητή του slot και τμήμα του slot. Έτσι, η πρόσβαση στο Data Bus πραγματοποιείται μόνο εντός των slots, που έχουν ανατεθεί στον εκάστοτε κόμβο.

Τα πλαίσια προς αποστολή διαπερνούν τις υπομονάδες προσωρινής αποθήκευσης (buffers), και στη συνέχεια κωδικοποιούνται για τη σειριακή τους αποστολή στο τρέχον slot. Ο αποκωδικοποιητής, που αναλαμβάνει και την αποκωδικοποίηση των ληφθέντων δεδομένων, ενσωματώνει περαιτέρω λειτουργικότητα, όπως bit-strobing, majority-voting, byte-packing, και επικύρωση των δεδομένων.

4.2.3 Επεκτάσεις στο Data Path

Ο παραδοσιακός τρόπος λειτουργίας ενός Communication Controller ορίζει ότι, όταν αυτό λάβει τα δεδομένα, θα πρέπει να απευθυνθεί στον host, για να γνωστοποιηθεί η χρησιμότητά τους. Αυτό γίνεται μέσω διακοπών (interrupts) προς τον host. Με αυτόν τον τρόπο όμως, υπάρχει η πιθανότητα να σπαταληθεί επεξεργαστική ισχύ χωρίς λόγο, καθώς τα ληφθέντα δεδομένα μπορεί να είναι απαρχαιωμένα, χωρίς πια καμία αξία. Αυτό μπορεί να έχει σοβαρές επιπτώσεις σε εφαρμογές με κύριο γνώμονα την ασφάλεια, όπου ορισμένα πλαίσια δεδομένων περιέχουν κρίσιμες πληροφορίες και πρέπει να επεξεργαστούν άμεσα.

Οι συγγραφείς λοιπόν προτείνουν την προσθήκη ειδικών επεκτάσεων στο Communication Controller, που θα του επιτρέψουν να επεξεργάζεται πρώτα τις πληροφορίες, κι έπειτα να πληροφορεί τον host. Ο host φυσικά διατηρεί τον απόλυτο έλεγχο του Communication Controller, αλλά δε χρειάζεται να ασχολείται με την επεξεργασία των δεδομένων σε χαμηλό επίπεδο. Στην παρακάτω εικόνα απεικονίζεται η λειτουργικότητα τέτοιων επεκτάσεων στη μεριά λήψης πλαισίων του Communication Controller.



Σχήμα 4.9: Επεκτάσεις στο Data Path του Communication Controller

Όπως φαίνεται στο σχήμα 4.9, κατά τη λήψη του πλαισίου είναι δυνατός ο έλεγχος αρκετών πληροφοριών, με βάση τις οποίες θα αποφασιστεί η μοίρα του πλαισίου.

Msg ID filter: Το μοναδικό αναγνωριστικό (ID) του πλαισίου θα μπορούσε να ενσωματώνει ενδείξεις σφάλματος, ώστε να εκκινείται άμεσα μια διεργασία αποκατάστασης σφαλμάτων σε εφαρμογές με κρίσιμο παράγοντα την ασφάλεια.

Appl. headers: Το τμήμα δεδομένων στο πλαίσιο θα μπορούσε να περιλαμβάνει κεφαλίδες, οι οποίες περιέχουν πληροφορίες σχετικά με το μήκος των δεδομένων και τον ακολουθιακό αριθμό του πλαισίου. Τέτοιου είδους πληροφορίες θα μπορούσαν να φανούν αρκετά χρήσιμες, σε περιπτώσεις όπου η ολοκλήρωση μεταφοράς των δεδομένων απαιτεί πολλούς κύκλους ρολογιού. Έτσι, το Communication Controller θα είναι σε θέση να αναγνωρίζει κατακερματισμένα δεδομένα, και να χρονοπρογραμματίζει κατάλληλα την αποστολή τους στο host, για τη βέλτιστη χρησιμοποίηση των πόρων.

Timestamp validation: Η εισαγωγή χρονοσφραγίδων στα πλαίσια δεδομένων θα συμβάλλει στην αναγνώριση απαρχαιωμένων δεδομένων, ώστε να απορριφθούν άμεσα, και να μην επιβαρυνθεί ο host με την επεξεργασία τους.

4.3 Πειραματικά Αποτελέσματα

Για την αξιολόγηση του σχεδιασμού και της απόδοσης της προαναφερθείσας υλοποίησης μοντελοποιήθηκε ένα ολοκληρωμένο Electronic Control Unit στο FPGA Xilinx Spartan 6 XC6SLX45. Ο Host περιγράφηκε ως μια μηχανή καταστάσεων. Επίσης, για τις δοκιμές επικοινωνίας με το δίκτυο πραγματοποιήθηκε η εξομοίωση ενός FlexRay DataBus εντός του FPGA, το οποίο λαμβάνει αιτήματα από ένα πραγματικό δίκτυο FlexRay. Το Communication Controller είναι άμεσα συνδεδεμένο με το FlexRay DataBus, και έχει ρυθμιστεί με τις αντίστοιχες παραμέτρους για τη λειτουργία του στο δίκτυο.

Παρακάτω παρουσιάζονται τα αποτελέσματα της μιας από τις τρεις περιπτώσεις χρήσης, που παρουσιάζονται στο paper [3]. Τα αποτελέσματα αποδεικνύουν ότι, οι προστιθέμενες επεκτάσεις στο Communication Controller οδηγούν σε πιο έξυπνα και αποτελεσματικά συστήματα σε σχέση με τις παραδοσιακές υλοποιήσεις.

4.3.1 Περίπτωση Χρήσης: Εντοπισμός Σφαλμάτων & Εφεδρική Λειτουργία

Οι εφαρμογές με κύριο γνώμονα την ασφάλεια χρησιμοποιούν εφεδρικές καταστάσεις λειτουργίας, οι οποίες εγγυούνται ελάχιστη λειτουργικότητα, σε περίπτωση που προκύψουν σφάλματα στο υλικό ή στο λογισμικό. Μια κρίσιμη παράμετρος σε αυτή την περίπτωση είναι το απαιτούμενο χρονικό διάστημα για την ενεργοποίηση της εφεδρικής λειτουργίας, από τη στιγμή που αναγνωριστεί το σφάλμα.

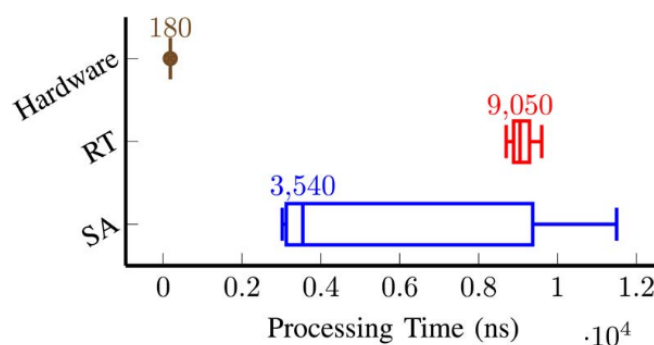
Η συγκεκριμένη περίπτωση χρήσης αφορά ένα σύστημα αυτοματοποιημένου φρεναρίσματος. Για το συγκεκριμένο πείραμα χρησιμοποιήθηκαν τα ακόλουθα ECUs:

1. Ο *Αισθητήρας Φρένων*, ο οποίος ελέγχει τις μονάδες αισθητήρων.
2. Ο *Ενεργοποιητής Φρένων*, ο οποίος στέλνει τις κατάλληλες εντολές στο σύστημα φρένων.

Κάθε ένα απ' τα παραπάνω ECUs ενσωματώνει μια λογική εφεδρικής λειτουργίας, η οποία ενεργοποιείται όταν ληφθεί ένα μήνυμα ένδειξης σφάλματος. Τα μηνύματα ένδειξης σφάλματος αποστέλλονται από μια μονάδα εντοπισμού σφαλμάτων, η οποία παρακολουθεί συνεχώς το DataBus για τυχόν εσφαλμένα δεδομένα ή εντολές.

Για τη διεξαγωγή του πειράματος εισήχθησαν εσφαλμένα δεδομένα στη μνήμη του αισθητήρα φρένων, προοριζόμενα για τον ενεργοποιητή φρένων μέσω του FlexRay DataBus. Κατά τη μετάδοση των εσφαλμένων δεδομένων, αυτά έγιναν αντιληπτά από τη μονάδα εντοπισμού σφαλμάτων, η οποία στη συνέχεια έστειλε ένα μήνυμα ένδειξης σφάλματος.

Χρησιμοποιώντας ένα συμβατικό Communication Controller, το λαμβανόμενο μήνυμα ένδειξης σφάλματος αποκωδικοποιείται και αποστέλλεται απευθείας στον επεξεργαστή, όπου τα δεδομένα υπόκεινται σε επεξεργασία για την ενεργοποίηση της εφεδρικής λειτουργίας. Η μέση καθυστέρηση σε αυτή την περίπτωση ανέρχεται στα 9.05 ms , όπως φαίνεται με την ένδειξη "RT" στο σχήμα 4.10.



Σχήμα 4.10: Καθυστέρηση για την επεξεργασία κρίσιμων δεδομένων

Αντιθέτως, μεταφέροντας την επεξεργασία των δεδομένων στο Communication Controller, μπορεί να μειωθεί η καθυστέρηση σε μεγάλο βαθμό. Σ' αυτή την περίπτωση, κατά το πέρας της επεξεργασίας των δεδομένων ενεργοποιείται μια διακοπή προς τον επεξεργαστή, για να γίνει η μετάβαση στην κατάσταση εφεδρικής λειτουργίας. Εδώ επιτυγχάνεται μια πολύ μικρότερη καθυστέρηση, με αποτέλεσμα το αίτημα να εξυπηρετείται έως και 50 φορές ταχύτερα (Σχήμα 4.10, ένδειξη "Hardware").

Επίσης, το πείραμα πραγματοποιήθηκε και με τη χρήση λογισμικού για την επεξεργασία των κρίσιμων δεδομένων. Όμως, όπως φαίνεται και στο σχήμα 4.10 (ένδειξη "SA"), η καθυστέρηση είναι αρκετά μεγαλύτερη.

4.4 Συμπεράσματα

Συνοψίζοντας, η επαύξηση των δυνατοτήτων του Communication Controller σε ένα ECU επιτυγχάνει σημαντική μείωση της κατανάλωσης ενέργειας, αλλά και παράλληλα προσφέρει στον κόμβο FlexRay επιπλέον λειτουργίες, οι οποίες δεν υπήρχαν στο πρωτόκολλο. Γίνεται λοιπόν κατανοητό ότι, η συγκεκριμένη προσέγγιση αποτελεί μια ιδανική λύση στο πρόβλημα αποδοτικότητας του πρωτοκόλλου FlexRay. Τέλος, αξίζει να σημειωθεί ότι, η συγκεκριμένη προσέγγιση είναι εφαρμόσιμη και σε άλλα χρονο-πυροδοτούμενα πρωτόκολλα, όπως το Ethernet.

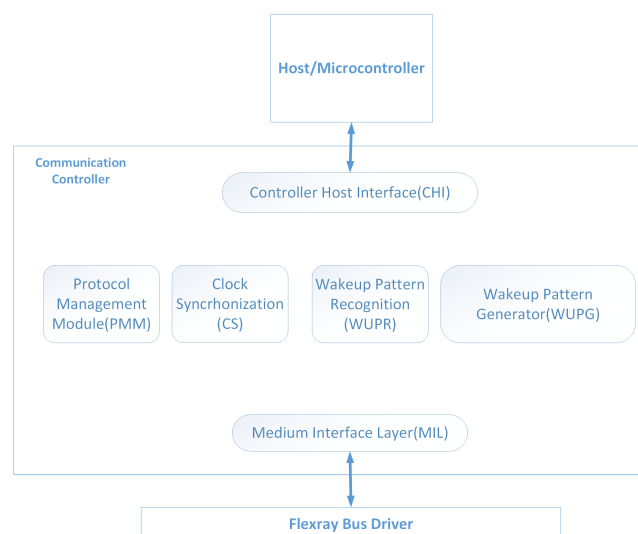
5. Υλοποίηση Communication Controller

Σε αυτό το κεφάλαιο παρουσιάζεται το κύριο μέρος της διπλωματικής εργασίας, δηλαδή ο σχεδιασμός και η ανάπτυξη ενός Communication Controller, με βάση το πρωτόκολλο FlexRay. Η αρχιτεκτονική και η λειτουργικότητα έχουν βασιστεί στο Communication Controller του paper [3], που παρουσιάστηκε περιληπτικά στο κεφάλαιο 4. Παρ' όλα αυτά όσον αφορά τις αρχιτεκτονικές των υπομονάδων εντός του Communication Controller, αυτές είναι ελλιπείς στο paper. Γι' αυτό το λόγο, έχω ακολουθήσει μια δικιά μου προσέγγιση, σχεδιάζοντας τις αρχιτεκτονικές τους.

Αρχικά, παρουσιάζεται η γενική αρχιτεκτονική του Communication Controller προς υλοποίηση, και αναφέρεται η γενική λειτουργικότητα κάθε μιας υπομονάδας στο εσωτερικό του. Επίσης, παρουσιάζεται η δομή του πλαισίου δεδομένων, που χρησιμοποιείται. Έπειτα, αναλύεται ξεχωριστά κάθε υπομονάδα, όσον αφορά την αρχιτεκτονική και τον τρόπο λειτουργίας της. Εκτός αυτού, ακολουθεί και μια εξομοίωση για κάθε υπομονάδα, για να επιβεβαιωθεί η ορθή λειτουργία της. Τέλος, πραγματοποιείται μια εξομοίωση ολόκληρου του συστήματος για την επιβεβαίωση της ορθής λειτουργίας του, και στη συνέχεια παρουσιάζονται τα αποτελέσματα της σύνθεσής του.

5.1 Γενική Αρχιτεκτονική

Στο σχήμα 5.1 απεικονίζεται η γενική αρχιτεκτονική του Communication Controller. Τα περισσότερα components εντός του Communication Controller είναι εμπνευσμένα απ' το paper. Όμως, τα Wakeup Pattern Recognition (WUPR) και Wakeup Pattern Generator (WUPG) έχουν προστεθεί από εμένα, για να εμπλουτίσω το Controller περισσότερη λειτουργικότητα, η οποία κανονικά θα πραγματοποιούνταν από τον Host/Microcontroller. Περισσότερες πληροφορίες για τις συγκεκριμένες υπομονάδες δίνονται παρακάτω.



Σχήμα 5.1: Γενική Αρχιτεκτονική Communication Controller

Controller Host Interface (CHI): Λειτουργεί ως διεπαφή για τον Host. Μέσω του CHI, ο Host έχει τη δυνατότητα να ελέγχει συνεχώς την κατάσταση λειτουργίας του Communication Controller, να στέλνει κατάλληλες εντολές, αλλά και να στέλνει πλαίσια δεδομένων, που αφορούν την εφαρμογή του Host.

Medium Interface Layer (MIL): Παίζει το ρόλο του διαμεσολαβητή μεταξύ του Communication Controller και του FlexRay Bus Driver/DataBus. Είναι υπεύθυνο για τη μεταβίβαση πλαισίων δεδομένων από και προς το δίκτυο, επιτελεί λειτουργίες κωδικοποίησης/αποκωδικοποίησης, χρονοπρογραμματισμού των πλαισίων, αλλά και εντοπισμού σφαλμάτων.

Protocol Management Module (PMM): Αποτελεί τον εγκέφαλο του Communication Controller. Φροντίζει για τη συμμόρφωση του συστήματος με βάση το πρωτόκολλο FlexRay, μέσω μιας μηχανής πεπερασμένων καταστάσεων. Ερμηνεύει τις εντολές που λαμβάνονται απ' τον Host, και ελέγχει τα υπόλοιπα components.

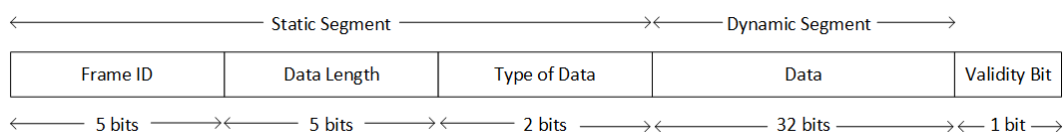
Clock Synchronization (CS): Η συγκεκριμένη υπομονάδα επιτελεί ακριβώς αυτό που υποδηλώνει το όνομά της. Δηλαδή, είναι υπεύθυνη για το συγχρονισμό του κόμβου, στον οποίο βρίσκεται το Communication Controller, με το υπόλοιπο δίκτυο.

Wakeup Pattern Generator (WUPG): Παράγει μια συγκεκριμένη ακολουθία συμβόλων, η οποία προορίζεται για άλλους κόμβους εντός του δικτύου FlexRay. Η συγκεκριμένη ακολουθία χρησιμοποιείται για την αφύπνιση των άλλων κόμβων, οι οποίοι βρίσκονται σε κατάσταση «sleep».

Wakeup Pattern Recognition (WUPR): Χρησιμοποιείται για την αναγνώριση της ακολουθίας συμβόλων, η οποία περιγράφηκε παραπάνω. Όταν η ακολουθία αναγνωριστεί, αποστέλλεται κατάλληλο σήμα στο Controller Host Interface, και ακολουθώς στον Host.

5.2 Η δομή του πλαισίου δεδομένων

Στο σχήμα 5.2 φαίνεται η δομή του πλαισίου δεδομένων, που χρησιμοποιείται.



Σχήμα 5.2: Χρησιμοποιούμενο πλαίσιο δεδομένων

Όπως φαίνεται και στην εικόνα, το πλαίσιο αποτελείται από τρία τμήματα: το στατικό, το δυναμικό και ένα bit εγκυρότητας.

Το *στατικό τμήμα* είναι στην ουσία η κεφαλίδα του πλαισίου, και αποτελείται απ' τα ακόλουθα πεδία:

Frame ID: Είναι ένας αριθμός μήκους 5 bits, και αποτελεί μοναδικό αναγνωριστικό του πλαισίου. Στην παρούσα υλοποίηση το συγκεκριμένο πεδίο δεν έχει κάποια ιδιαίτερη χρησιμότητα. Παρ' όλα αυτά προστέθηκε για μελλοντικές εφαρμογές.

Data Length: Αντιπροσωπεύει το μήκος των δεδομένων, που περιέχει το πλαίσιο. Δηλαδή, το πραγματικό μέγεθος του δυναμικού τμήματος.

Type of Data: Αυτός ο αριθμός αντιπροσωπεύει το είδος των δεδομένων, που περιέχονται στο πλαίσιο. Το συγκεκριμένο πεδίο χρησιμοποιείται κυρίως για τον προσδιορισμό του προορισμού του πλαισίου δεδομένων, εντός του Communication Controller. Το είδος των δεδομένων θα μπορούσε να είναι ένα απ' τα παρακάτω:

- Συνήθη δεδομένα, τα οποία προορίζονται για το Controller Host Interface.
- Μοτίβο αφύπνισης, που χρησιμοποιείται απ' την υπομονάδα Wakeup Pattern Recognition για την αναγνώριση του μοτίβου (ή ακολουθία συμβόλων).
- Δεδομένα συγχρονισμού, τα οποία αποτελούν κρίσιμα δεδομένα για το συγχρονισμό του κόμβου, και υπόκεινται σε επεξεργασία από το Clock Synchronization.

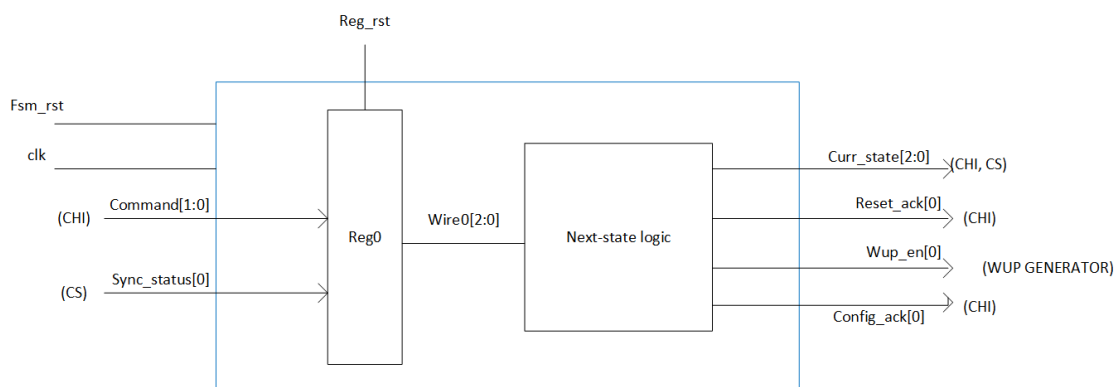
Απ' τη στιγμή που χρησιμοποιούνται τρία είδη δεδομένων, είναι φανερό ότι απαιτούνται τουλάχιστον 2 bits για την αναπαράστασή τους ($\log_2 3 = 1.58$). Γι' αυτό το λόγο και το μήκος του πεδίου είναι των 2 bits.

Το *δυναμικό τμήμα* αποτελείται από ένα και μόνο πεδίο: τα δεδομένα που περιέχονται στο πλαίσιο. Είναι φανερό ότι, το μέγεθος των δεδομένων που μπορούν να μεταφερθούν σε ένα πλαίσιο, δεν μπορεί να ξεπερνά τα 32 bits.

Τέλος, το *ψηφίο εγκυρότητας* δηλώνει αν το πλαίσιο είναι έγκυρο, δηλαδή ότι περιέχει πραγματικά δεδομένα και όχι «σκουπίδια». Χρησιμοποιείται για να αποφασιστεί το αν επιτραπεί στο πλαίσιο να αποσταλεί στο Data Bus του δικτύου.

5.3 Protocol Management Module

Όπως αναφέρθηκε και στην ενότητα 5.1, το Protocol Management Module (PMM) περιέχει μια μηχανή πεπερασμένων καταστάσεων, που υπακούει στο πρωτόκολλο FlexRay. Η μηχανή αυτή αποθηκεύει την τρέχουσα κατάσταση λειτουργίας του Communication Controller, ερμηνεύει εντολές απ' τον Host, και ελέγχει τα υπόλοιπα components με κατάλληλα σήματα. Στο παρακάτω σχήμα δίνεται η αρχιτεκτονική του PMM.



Σχήμα 5.3: Αρχιτεκτονική του Protocol Management Module

Στο σχήμα 5.3 το Reg0 αντιπροσωπεύει έναν καταχωρητή, ο οποίος έχει υλοποιηθεί με ένα D Flip-Flop. Το Next-state Logic είναι η μηχανή πεπερασμένων καταστάσεων. Επίσης, στο σχήμα φαίνονται οι ονομασίες των σημάτων, το μήκος τους, αλλά και ο προορισμός τους. Για παράδειγμα, για το σήμα εξόδου curr_state[2:0] παρατηρούμε ότι, έχει μήκος 3 bits (από 2 έως 0) και στέλνεται σε δύο υπομονάδες, τις Controller Host Interface (CHI) και Clock Synchronization (CS).

5.3.1 Οι εντολές από τον Host/Microcontroller

Παρακάτω αναγράφονται όλες οι πιθανές εντολές, που είναι δυνατό να ληφθούν απ' τον Host.

WAKEUP: Σκοπεύει στην αποστολή μιας ακολουθίας συμβόλων για την αφύπνιση των άλλων κόμβων.

STARTUP: Ουσιαστικά, εκκινεί την κανονική λειτουργία του Communication Controller. Με την έναρξη της κανονικής λειτουργίας, ξεκινά και ο συγχρονισμός του κόμβου με το υπόλοιπο δίκτυο.

CONFIG: Αρχικοποιούνται κάποια σήματα πριν την έναρξη της κανονικής λειτουργίας. Όταν ολοκληρωθεί η αρχικοποίησή τους, αποστέλλεται ένα σήμα επιβεβαίωσης (ACK) στον Host.

HALT: Σταματά άμεσα τη λειτουργία του Communication Controller.

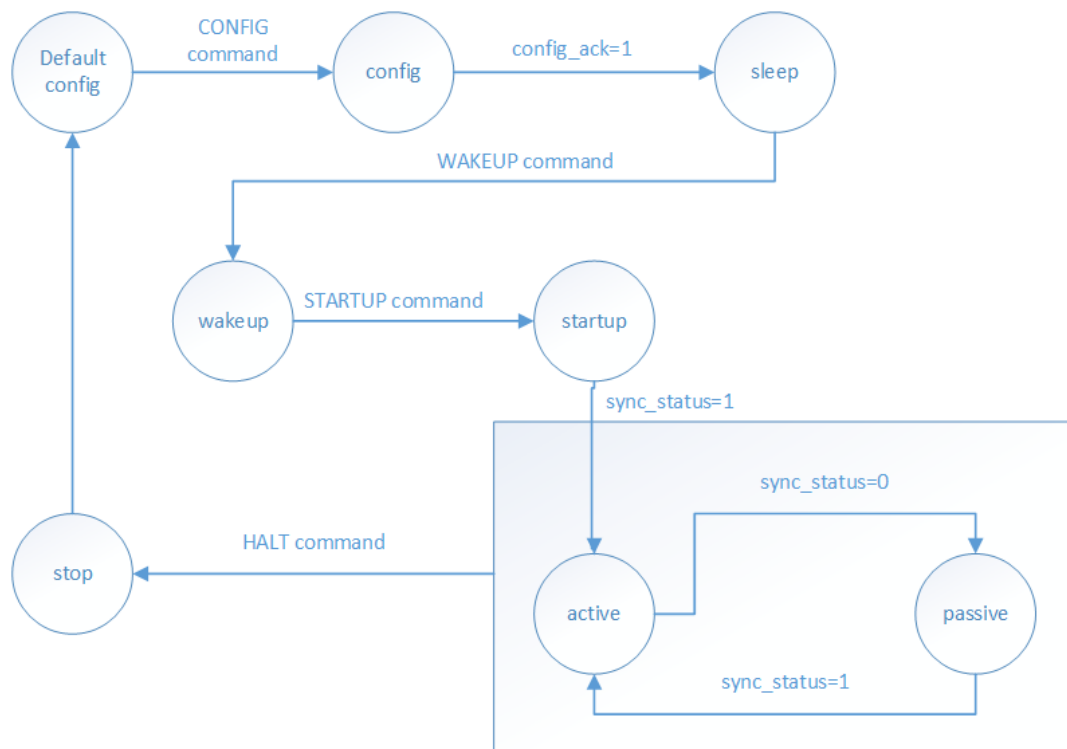
5.3.2 Η μηχανή πεπερασμένων καταστάσεων

Η μηχανή πεπερασμένων καταστάσεων έχει υλοποιηθεί εντός της υπομονάδας Next-state Logic του σχήματος 5.3. Οι προδιαγραφές της αναγράφονται στον παρακάτω πίνακα καταστάσεων.

Current State	command	Input config_ack	sync_status	Next State	config_ack	Output WUP_EN	reset_ack
default_config	CONFIG			config			
config		1		sleep	1		
sleep	WAKEUP			wakeup		1	
wakeup	STARTUP			startup			
startup			1	active			
active	HALT		0	passive stop			
passive	HALT		1	active stop			
stop				default_config			1

Πίνακας 5.1: Πίνακας καταστάσεων της FSM στο PMM

Για την καλύτερη κατανόηση των προδιαγραφών, ακολουθεί ένα διάγραμμα καταστάσεων, το οποίο αναπαριστά τη λειτουργία της μηχανής σε μορφή γραφήματος.



Σχήμα 5.4: Διάγραμμα καταστάσεων της FSM

Στο σχήμα 5.4 είναι φανερό ότι, τα commands που λαμβάνονται απ' τον Host εκτελούν τις λειτουργίες που περιγράφηκαν στην υποενότητα 5.3.1. Παρ' όλα αυτά, αξίζει να σημειωθούν τα παρακάτω:

- Όταν τερματιστεί η λειτουργία του Communication Controller με την εκτέλεση της εντολής HALT, η μηχανή μεταβαίνει σε μια κατάσταση stop. Μετά από ένα χρονικό διάστημα, πραγματοποιείται μετάβαση στην αρχική κατάσταση του συστήματος (Default Config), για την επανεκκίνησή του.
- Όπως αναφέρθηκε και προηγουμένως, όταν ξεκινήσει η κανονική λειτουργία του Controller (κατάσταση Startup), ακολουθεί η διαδικασία συγχρονισμού. Μετά τον επιτυχή συγχρονισμό του κόμβου με το δίκτυο, γίνεται η μετάβαση σε μια superstate (συμπαγές τετράγωνο στο σχήμα 5.4), όπου και ελέγχεται συνεχώς αν ο κόμβος είναι συγχρονισμένος με το υπόλοιπο δίκτυο. Εντός αυτής της superstate, το Controller μεταβαίνει συνεχώς μεταξύ των καταστάσεων active και passive, αναλόγως με το αν είναι επιτυχής ο συγχρονισμός ή όχι.

Τέλος, αξίζει να σημειωθεί ότι η συνεχής αλληλεπίδραση των Communication Controller και Host/Microcontroller μέσω των Commands, αποδεικνύει πως ο Host εξακολουθεί να έχει τον έλεγχο του Controller. Δηλαδή, παρά την επαύξηση των δυνατοτήτων του Communication Controller για την εκτέλεση κάποιων λειτουργιών ανεξάρτητα, ο Host διατηρεί τον έλεγχό του.

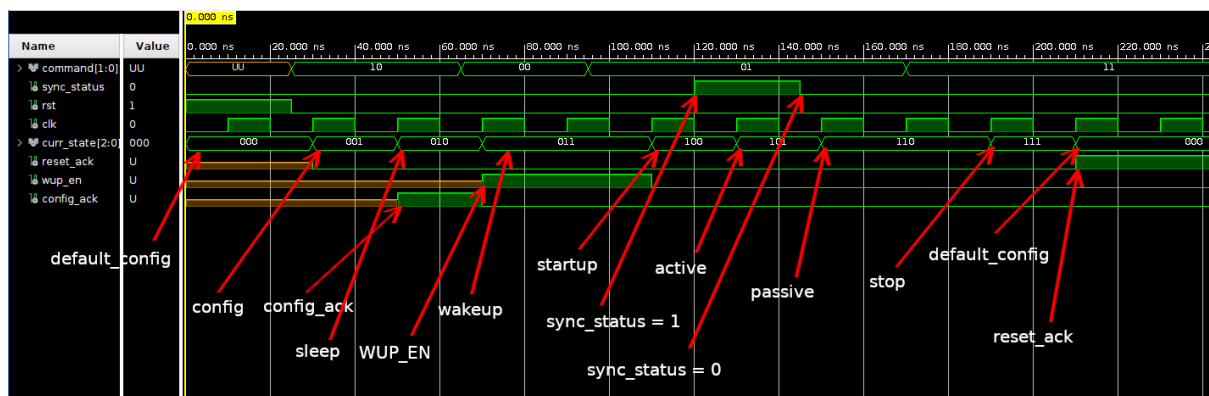
5.3.3 Testbench

State	Codeword
default_config	000
config	001
sleep	010
wakeup	011
startup	100
active	101
passive	110
stop	111

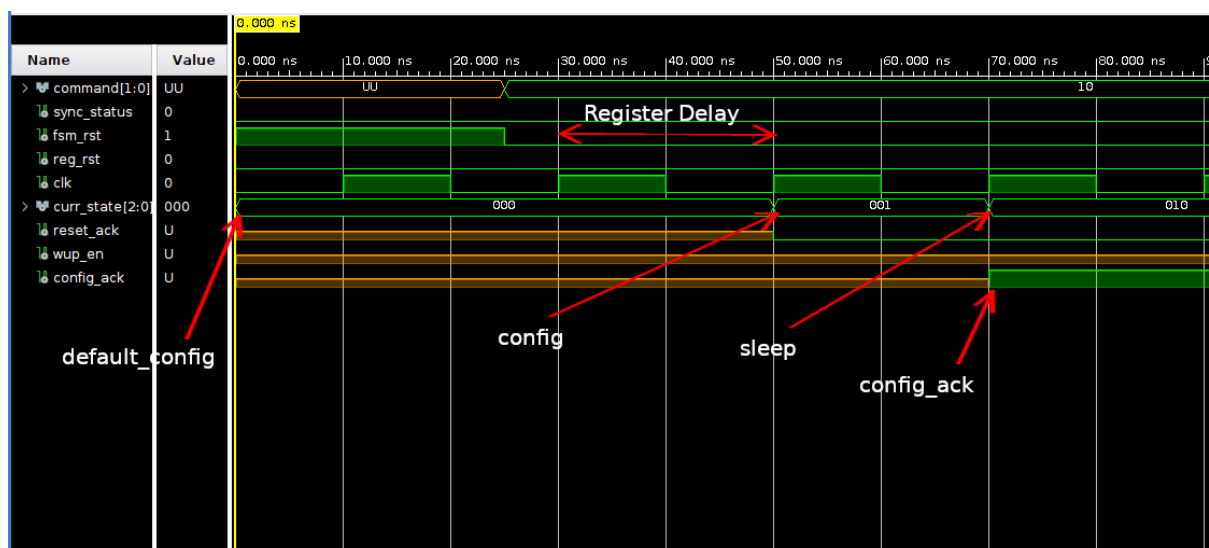
Command	Codeword
WAKEUP	00
STARTUP	01
CONFIG	10
HALT	11

Πίνακας 5.3: Κωδικοποίηση Commands από τον Host

Πίνακας 5.2: Κωδικοποίηση Καταστάσεων



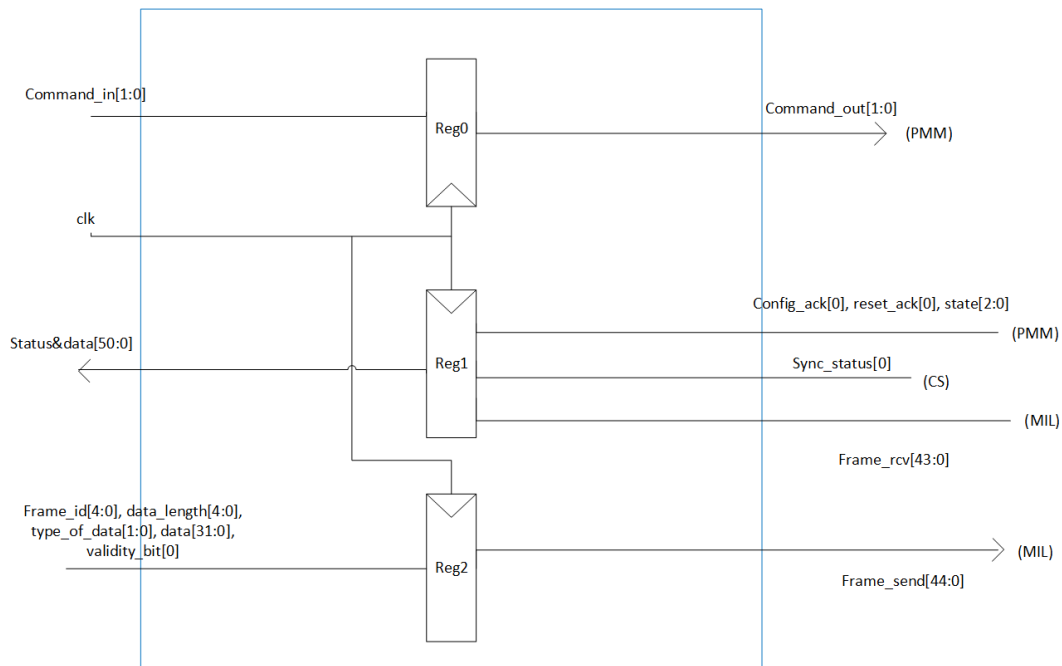
Σχήμα 5.5: Testbench της υπομονάδας Next-state logic (FSM), Αρχείο: pmm/fsm_tb.vhd



Σχήμα 5.6: Testbench του Protocol Management Module, Αρχείο: pmm/pmm_tb.vhd

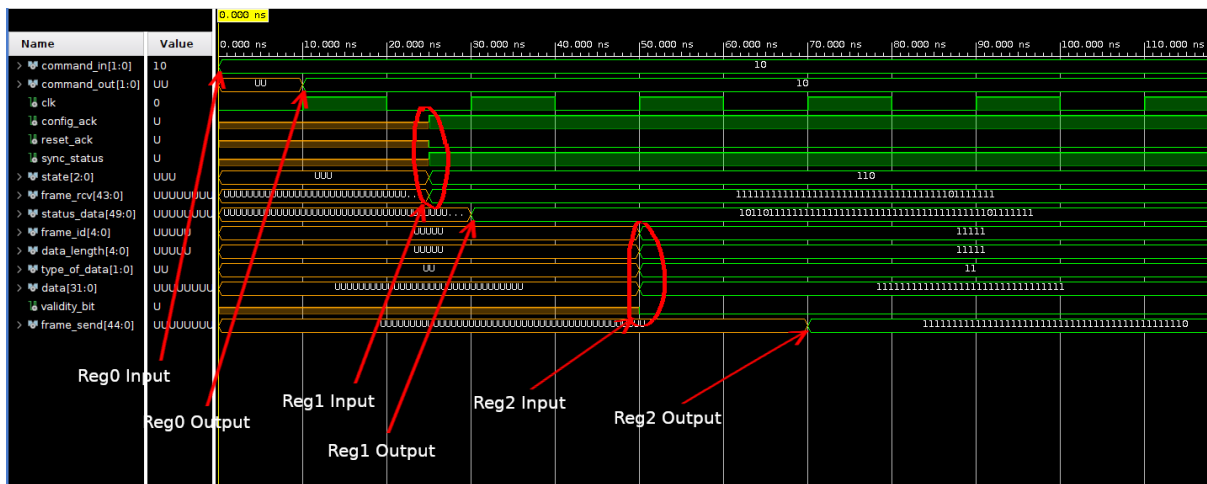
5.4 Controller Host Interface

Το Controller Host Interface (CHI) αποτελεί το μέσο μεταβίβασης δεδομένων μεταξύ των Host και Communication Controller. Αυτά τα δεδομένα αφορούν την τρέχουσα κατάσταση λειτουργίας, εντολές και πλαίσια δεδομένων. Στο σχήμα 5.7 φαίνεται η αρχιτεκτονική του CHI. Όπως φαίνεται και στο σχήμα, οι κύριες λειτουργίες του είναι προσωρινή αποθήκευση δεδομένων (buffering), αλλά και ομαδοποίηση ορισμένων σημάτων για την καλύτερη κατανόησή τους και τη διευκόλυνση αναγνώρισής τους στα testbenches.



Σχήμα 5.7: Αρχιτεκτονική του Controller Host Interface

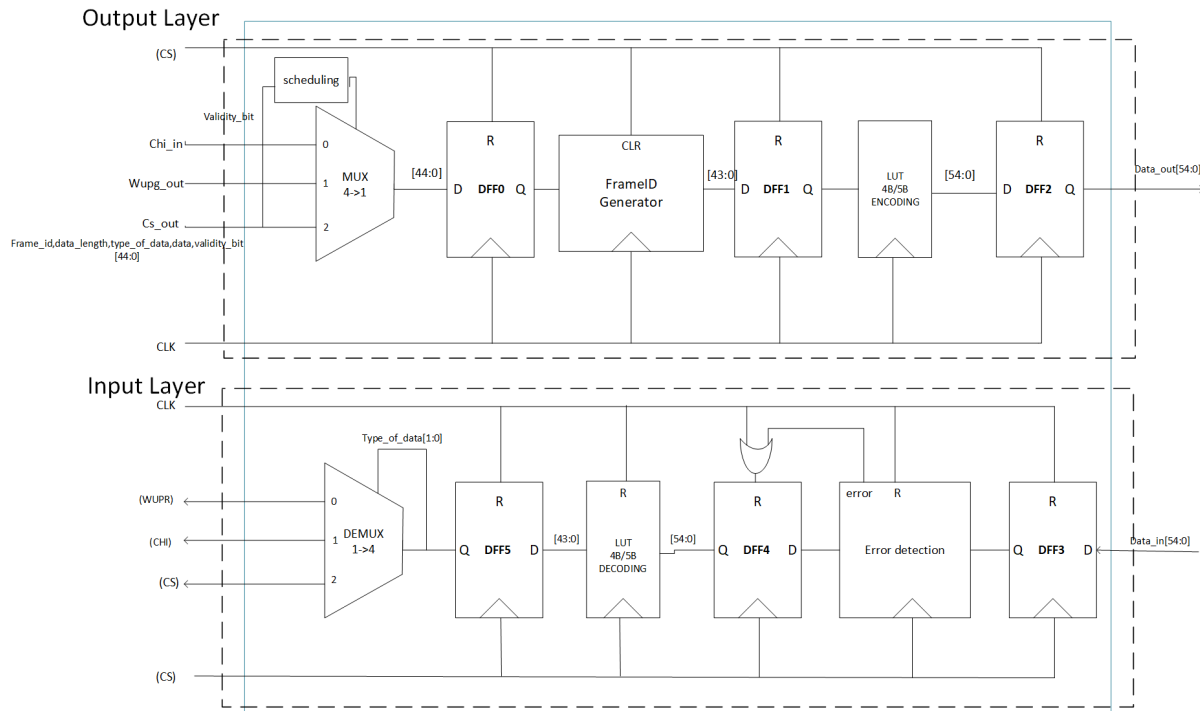
5.4.1 Testbench



Σχήμα 5.8: Testbench του Controller Host Interface, Αρχείο: chi/main_tb.vhd

5.5 Medium Interface Layer

Το Medium Interface Layer (MIL) είναι υπεύθυνο για την αποστολή και τη λήψη πλαισίων δεδομένων από και προς το Flexray Data Bus. Επίσης, εκτελεί ποικίλες λειτουργίες επί των πλαισίων. Μερικές από αυτές είναι η κωδικοποίηση εξερχόμενων πλαισίων, η αποκωδικοποίηση εισερχόμενων πλαισίων, ο χρονοπρογραμματισμός των εξερχόμενων πλαισίων, και ο εντοπισμός σφαλμάτων. Στο παρακάτω σχήμα απεικονίζεται η αρχιτεκτονική του MIL.



Σχήμα 5.9: Αρχιτεκτονική του Medium Interface Layer

Το *Output Layer* είναι το τμήμα επεξεργασίας πλαισίων δεδομένων, που προορίζονται για αποστολή στο δίκτυο, ενώ το *Input Layer* είναι το τμήμα επεξεργασίας των λαμβανόμενων πλαισίων από το δίκτυο.

Ακόμη, να σημειωθεί ότι το DFFX (όπου X ακέραιος αριθμός) αποτελεί ένα D Flip Flop, ενώ τα Mux και Demux σημαίνουν πολυπλέκτης και αποπλέκτης αντίστοιχα.

5.5.1 Output Layer

Όπως φαίνεται και στο σχήμα 5.9, όταν τα πλαίσια εισάγονται στο Output Layer, αρχικά περνάνε από έναν πολυπλέκτη (MUX). Η παρουσία του πολυπλέκτη είναι απαραίτητη, καθώς η απουσία του θα οδηγούσε σε συγκρούσεις, αφού πολλαπλά εσωτερικά components στο Communication Controller, θα μπορούσαν να στείλουν πλαίσια στον ίδιο κύκλο ρολογιού.

Το ποίο πλαίσιο δεδομένων θα περάσει πρώτο, αποφασίζεται απ' το κύκλωμα *Scheduling*. Το συγκεκριμένο κύκλωμα παίρνει ως είσοδο τα ψηφία εγκυρότητας (*validity_bit* στο σχήμα 5.9) των πλαισίων, ώστε να γνωρίζει τα έγκυρα πλαίσια. Έπειτα, προωθεί τα πλαίσια με βάση την προέλευσή τους. Η σειρά προτεραιότητας είναι η ακόλουθη:

1. Clock Synchronization (CS)
2. Controller Host Interface (CHI)
3. Wakeup Pattern Generator (WUPG)

Άρα, προκύπτει ο παρακάτω αλγόριθμος:

```

if (validity_bit_cs == 1) {
    output = frame_from_cs;
} else if (validity_bit_chi == 1) {
    output = frame_from_chi;
} else if (validity_bit_wup == 1) {
    output = frame_from_wup;
} else {
    output = Z;
}

```

Το 'Z' στη VHDL σημαίνει υψηλή εμπέδηση.

Με βάση τον αλγόριθμο, συμπληρώνεται ο πίνακας αληθείας του κυκλώματος Scheduling:

Input (validity bits: cs, chi, wup (= a, b, c))	Output (select s1, s0)
000	ZZ
001	01
010	00
011	00
100	11
101	11
110	11
111	11

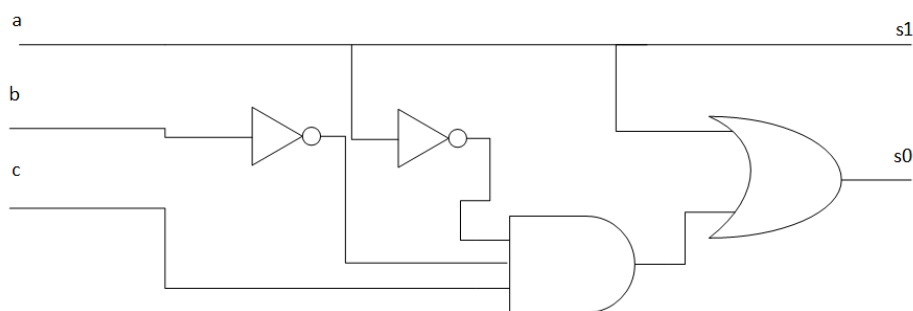
Πίνακας 5.4: Πίνακας αληθείας του κυκλώματος χρονοπρογραμματισμού πλαισίων

Από τον παραπάνω πίνακα προκύπτουν οι ακόλουθες λογικές εξισώσεις:

$$S1 = ab'c' + ab'c + abc' + abc = a$$

$$S0 = a + a'b'c$$

Τώρα, γνωρίζοντας απ' τις λογικές εξισώσεις πώς προκύπτει η έξοδος του κυκλώματος με βάση τις εισόδους του, είμαστε σε θέση πια να σχεδιάσουμε το κύκλωμα σε επίπεδο ψηφιακών πυλών. Το κύκλωμα απεικονίζεται στο σχήμα 5.10.



Σχήμα 5.10: Κύκλωμα χρονοπρογραμματισμού πλαισίων

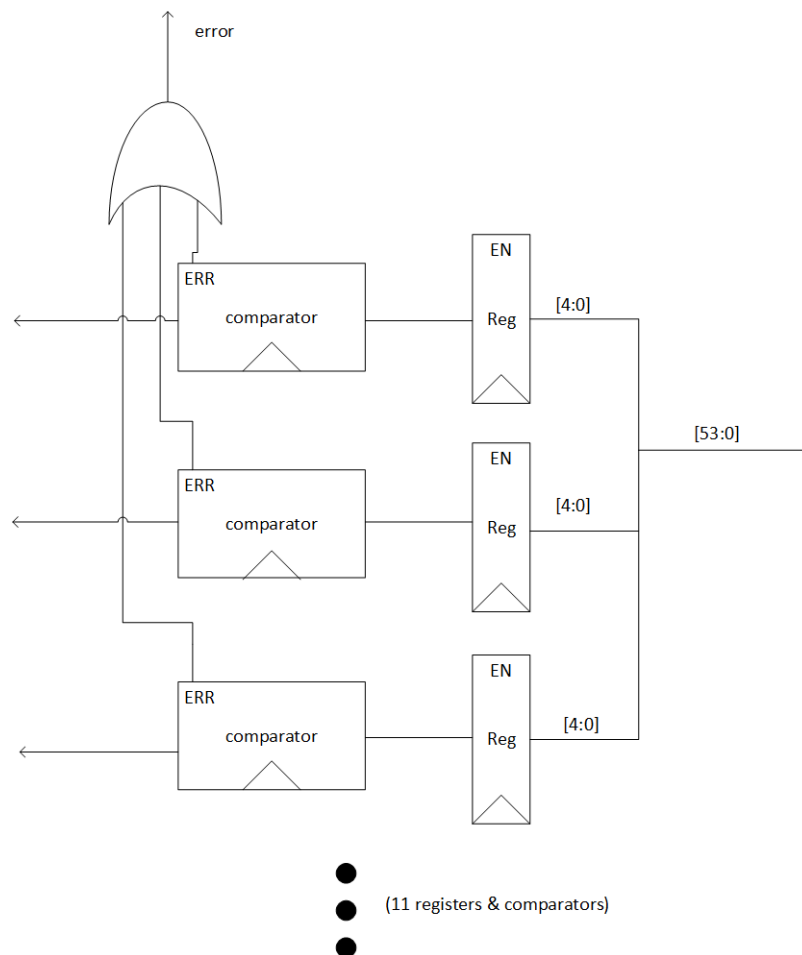
Το *Frame ID Generator* είναι ουσιαστικά ένας μετρητής, ο οποίος αναθέτει σε κάθε εξαγόμενο πλαίσιο έναν αριθμό, ως μοναδικό αναγνωριστικό του πλαισίου. Όπως αναφέρθηκε στην ενότητα 5.2, το Frame ID δεν έχει κάποια χρησιμότητα στην υλοποίησή μας, αλλά θα μπορούσε να χρησιμοποιηθεί για μελλοντικές εφαρμογές, π.χ. αποστολή επιβεβαίωσης (ACK) για την επιτυχή λήψη ενός πλαισίου.

Επίσης, παρατηρούμε ότι στην έξοδο του Frame ID Generator το σήμα έχει ένα λιγότερο bit. Αυτό συμβαίνει γιατί, αφαιρείται το ψηφίο εγκυρότητας (validity bit), καθώς έχει μονάχα τοπική σημασία για το Communication Controller.

Η υπομονάδα *LUT 4B/5B Encoding* πραγματοποιεί την κωδικοποίηση του πλαισίου δεδομένων προς προώθηση. Η κωδικοποίηση 4B/5B έχει εξηγηθεί αναλυτικά στην ενότητα [Α.1](#). Όπως φαίνεται και στο σχήμα [5.9](#), το τελικό πλαίσιο είναι των 55 bits, αφού με αυτόν τον τρόπο κωδικοποίησης κάθε 4 bits αντικαθίστανται από 5 bits.

5.5.2 Input Layer

Κατά την εισαγωγή ενός πλαισίου δεδομένων στον κόμβο, καλό είναι αυτό να ελέγχεται για πιθανά σφάλματα, που μπορεί να έχουν συμβεί κατά τη μετάδοσή του στο δίκτυο. Αυτό πραγματοποιείται από το component *Error-detection*. Ο εντοπισμός σφαλμάτων βασίζεται στον τρόπο κωδικοποίησης 4B/5B. Όπως αναφέρθηκε και στην ενότητα [Α.1](#), κάθε πιθανή ακολουθία των 4 bits αντιστοιχεί σε μια μοναδική των 5 bits. Άρα, είναι αρκετά απλός ο έλεγχος εγκυρότητας ενός κωδικοποιημένου μπλοκ, αφού αρκεί η εύρεσή του εντός του λεξικού (πίνακας [Α.1](#)). Ακόμη, να σημειωθεί ότι σε αυτήν την υλοποίηση, δε χρησιμοποιούνται οι ακολουθίες ελέγχου (πίνακας [Α.2](#)), άρα η παρουσία τους αντιμετωπίζεται ως σφάλμα. Ακολουθεί η αρχιτεκτονική της υπομονάδας Error-Detection.



Σχήμα 5.11: Αρχιτεκτονική υπομονάδας εντοπισμού σφαλμάτων

Όπως φαίνεται στο σχήμα [5.11](#), κάθε μπλοκ των 5 ψηφίων ελέγχεται ξεχωριστά από έναν συγκριτή. Αν βρεθεί έστω και ένα εσφαλμένο μπλοκ, ο συγκριτής θα θέσει την έξοδο ERR στην τιμή 1. Τότε, η έξοδος της πύλης OR θα πάρει την τιμή 1. Επιστρέφοντας στο σχήμα [5.9](#) παρατηρούμε ότι, με την έξοδο error του κυκλώματος Error-detection στην τιμή 1 θα γίνει reset το επόμενο Flip-Flop στον επόμενο κύκλο ρολογιού. Με αυτόν τον τρόπο, το πλαίσιο με το εσφαλμένο μπλοκ δεδομένων θα γίνει «drop», δηλαδή θα απορριφθεί.

Πριν το πλαίσιο κωδικοποιηθεί προσάπτεται το frame id, το οποίο είναι 00001, αφού είναι το πρώτο πλαίσιο προς αποστολή. Άρα, το περιεχόμενο του πλαισίου τώρα είναι:

00001|00000000...0001

Παρατηρούμε ότι το ψηφίο εγκυρότητας έχει αφαιρεθεί. Με βάση την κωδικοποίηση 4B/5B (πίνακας [ΑΠ.1](#)), τα περιεχόμενα του πλαισίου μετασχηματίζονται ως εξής:

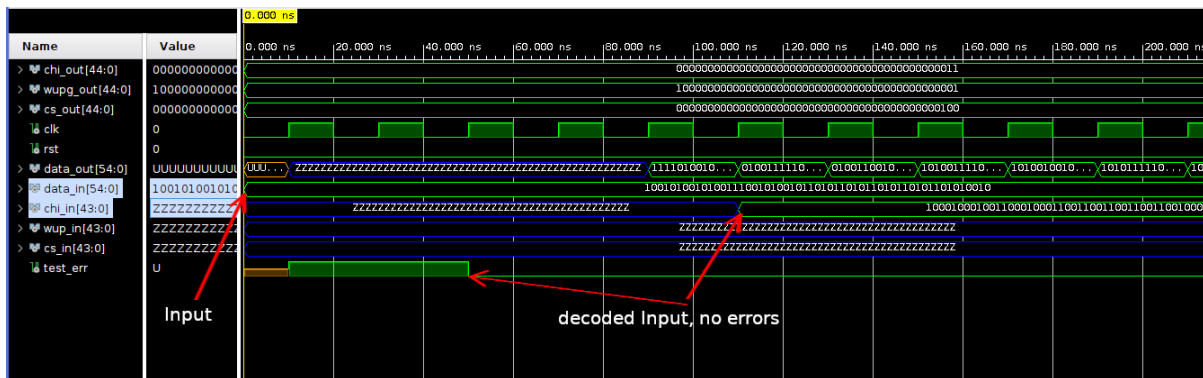
Οργάνωση σε τετράδες:

0000 1000 0000 ... 0001

Μετασχηματισμός τετράδων σε πεντάδες:

11110 10010 11110 11110 ... 01001

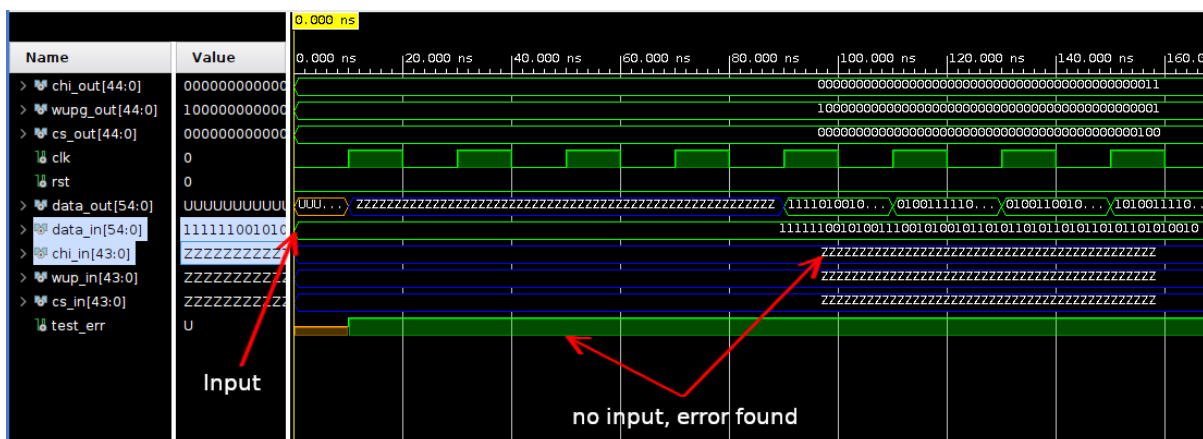
Η παραπάνω ψηφιολέξη ταυτίζεται με αυτή στο σχήμα [5.14](#), άρα επιβεβαιώνεται η ορθή λειτουργία του κυκλώματος.



Σχήμα 5.15: Testbench #1 για το MIL (Input Layer), Αρχείο: mil/mil_main_tb.vhd

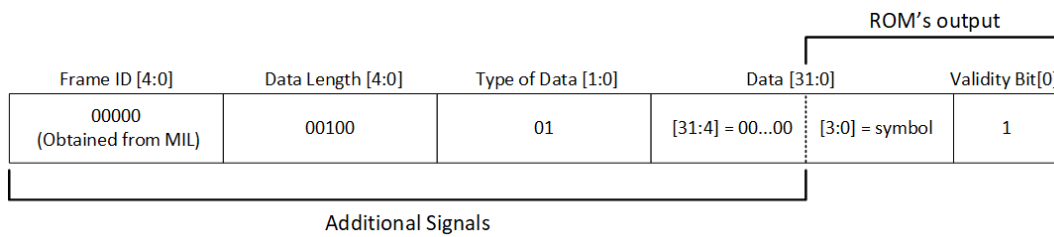
Το σήμα test_err στο σχήμα [5.15](#) αντιπροσωπεύει την έξοδο του κυκλώματος Error Detection, δηλαδή άμα υπάρχει σφάλμα. Η έξοδος του προκύπτει στη θετική ακμή του τρίτου κύκλου ρολογιού. Στο παραπάνω σχήμα είναι 0, άρα δεν έχει προκύψει σφάλμα, και το πλαίσιο εμφανίζεται στην έξοδο chi_in.

Αντιθέτως, στο παρακάτω σχήμα η τιμή του test_err είναι 1, με αποτέλεσμα το πλαίσιο δεδομένων να γίνεται drop, κι έτσι δεν εμφανίζεται στην έξοδο. Αυτό συμβαίνει γιατί, το πρώτο μπλοκ δεδομένων (11111), δεν αντιστοιχεί σε κάποιο έγκυρο στοιχείο στο λεξικό. (Υπενθυμίζεται ότι για την υλοποίηση δεν έχουν χρησιμοποιηθεί τα ειδικά σύμβολα στον πίνακα [ΑΠ.2](#).)



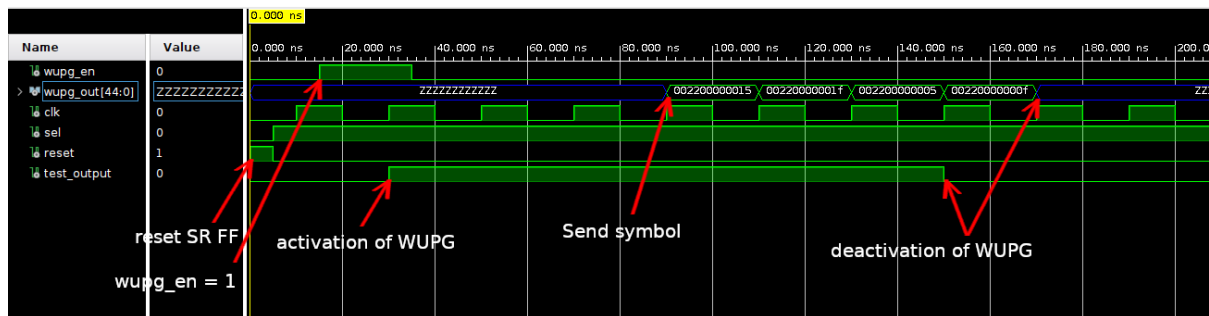
Σχήμα 5.16: Testbench #2 για το MIL (Input Layer), Αρχείο: mil/mil_main_tb.vhd

Ακόμη, η πλαίσίωση των δεδομένων (του συμβόλου) πραγματοποιείται εντός του Wakeup Pattern Generator. Αυτό επιτυγχάνεται με τη βοήθεια των πρόσθετων σημάτων «Additional signals for framing the data», που συνενώνονται στην είσοδο του DFF1 με τα δεδομένα. Στην παρακάτω εικόνα φαίνεται η δομή του τελικού πλαισίου και η προέλευση των πεδίων του.



Σχήμα 5.18: Πλαίσιο δεδομένων του Wakeup Pattern Generator

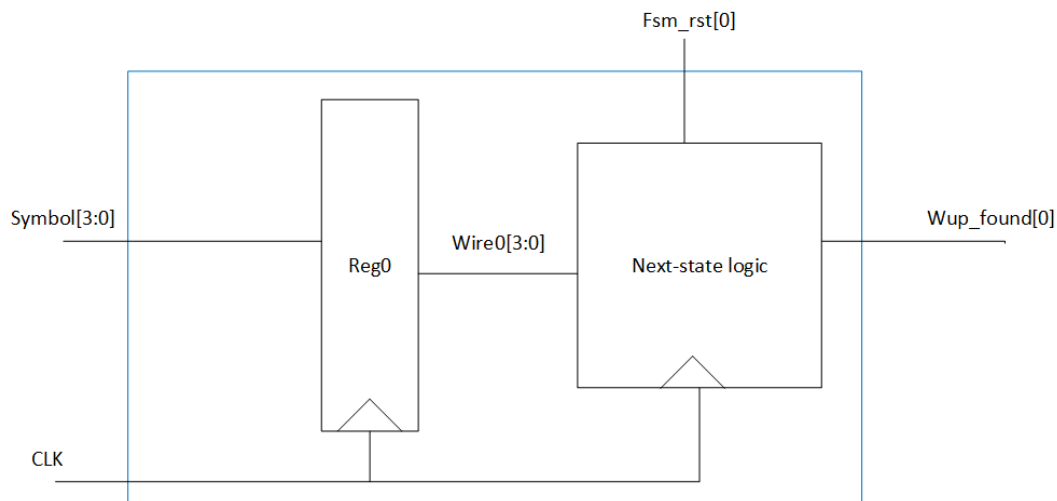
5.6.1 Testbench



Σχήμα 5.19: Testbench του Wakeup Pattern Generator, Αρχείο: wupg/wupg_main_tb.vhd

5.7 Wakeup Pattern Recognition

Το Wakeup Pattern Recognition (WUPR) αναγνωρίζει μια προκαθορισμένη ακολουθία συμβόλων. Αν η ακολουθία συμβόλων είναι η σωστή, ο κόμβος μεταβαίνει από την κατάσταση sleep στην κατάσταση wakeup. Η αρχιτεκτονική του WUPR απεικονίζεται στο παρακάτω σχήμα.



Σχήμα 5.20: Αρχιτεκτονική του Wakeup Pattern Recognition

Στο σχήμα 5.20 το Reg0 είναι ένας καταχωρητής, ενώ το Next-state logic περιέχει μια μηχανή πεπερασμένων καταστάσεων για την αναγνώριση του μοτίβου (ή ακολουθία συμβόλων).

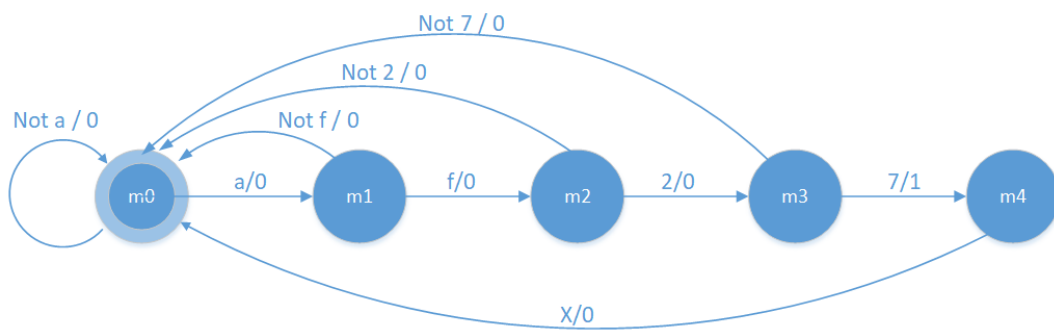
Θεωρούμε ότι η ακολουθία συμβόλων προς αναγνώριση είναι η 0xa, 0xf, 0x2, 0x7 (δεκαδική μορφή των αριθμών 10, 15, 2, και 7 αντίστοιχα). Ακολουθεί ο πίνακας καταστάσεων της

FSM, που καθορίζει την επόμενη κατάσταση, τις απαιτούμενες συνθήκες, και την έξοδο σε κάθε μετάβαση.

Current State	Input: Symbol	Next State	Output: wup_found
m0	a / not a	m1/m0	0/0
m1	f / not f	m2/m0	0/0
m2	2 / not 2	m3/m0	0/0
m3	7 / not 7	m4/m0	1/0
m4	X	m0	0

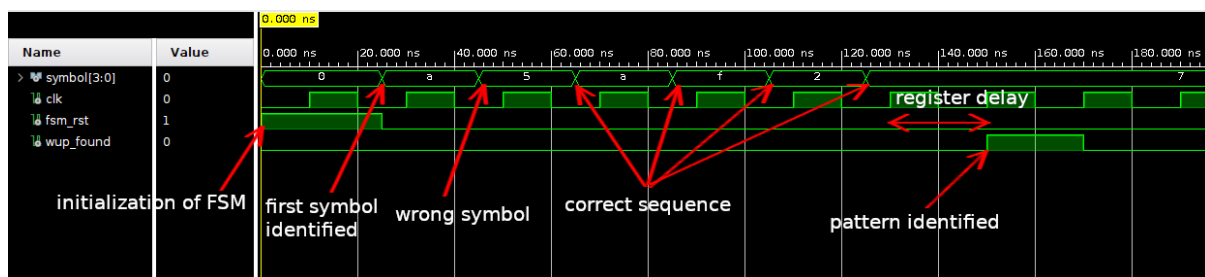
Πίνακας 5.5: Πίνακας καταστάσεων της FSM στο WUPR

Όπως φαίνεται και στον πίνακα 5.5, η κάθετος αντιπροσωπεύει εναλλακτική είσοδο ή επόμενη κατάσταση ή έξοδο. Για παράδειγμα, αν στην κατάσταση M0 το λαμβανόμενο σύμβολο είναι a, η μηχανή μεταβαίνει στην κατάσταση m1, και η έξοδος είναι 0. Διαφορετικά, για οποιαδήποτε άλλη είσοδο παραμένει στην κατάσταση m0, και η έξοδος είναι πάλι 0. Όταν αναγνωριστεί όλη η ακολουθία, η έξοδος είναι 1. Το 'X' αντιπροσωπεύει το αδιάφορο (don't care), δηλαδή η αλλαγή κατάστασης συμβαίνει ούτως ή άλλως στον επόμενο κύκλο ρολογιού. Τα παραπάνω φαίνονται καλύτερα στο παρακάτω διάγραμμα καταστάσεων.



Σχήμα 5.21: Διάγραμμα καταστάσεων της FSM στο WUPR

5.7.1 Testbench



Σχήμα 5.22: Testbench του WUPR, Αρχείο: wup_recognition/wupr_main_tb.vhd

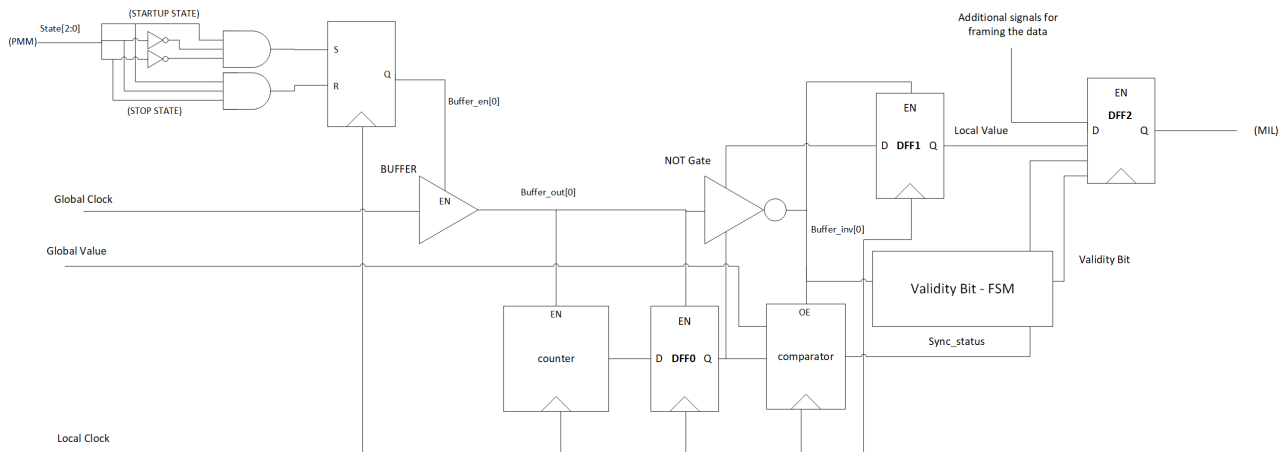
5.8 Clock Synchronization

Το Clock Synchronization είναι υπεύθυνο για το συγχρονισμό του κόμβου με το υπόλοιπο δίκτυο. Έχω δημιουργήσει την υπομονάδα για τον έλεγχο συγχρονισμού, αλλά δεν έχω

υλοποιήσει κάποιο διορθωτικό μηχανισμό του ρολογιού. Ο έλεγχος του αν ο κόμβος είναι συγχρονισμένος με το υπόλοιπο δίκτυο, πραγματοποιείται απ' την υπομονάδα *Microtick Counter*, οπότε η παρούσα ενότητα είναι αφιερωμένη σ' αυτή την υπομονάδα.

Σύμφωνα με το paper [3], τα Microticks και τα Macroticks έχουν να κάνουν με την ευθυγράμμιση του τοπικού και καθολικού (global) ρολογιού αντίστοιχα. Για την παρούσα υλοποίηση έχω θεωρήσει ότι τα Microticks είναι ο αριθμός των κύκλων του τοπικού ρολογιού, που θα πρέπει να αντιστοιχούν σε ένα Macrotick, δηλαδή κύκλο του καθολικού ρολογιού. Αν η αντιστοιχία είναι σωστή, τότε η έξοδος του κυκλώματος είναι 1, διαφορετικά 0.

Στο παρακάτω σχήμα απεικονίζεται η αρχιτεκτονική της υπομονάδας Microtick Counter.



Σχήμα 5.23: Αρχιτεκτονική του Microtick Counter

5.8.1 Τρόπος Λειτουργίας

Στο σχήμα 5.23 είναι φανερό ότι, το Microtick Counter ενεργοποιείται στην κατάσταση λειτουργίας STARTUP, όπου το S-R Flip-Flop γίνεται set και η έξοδος του τίθεται στο 1. Όταν ανιχνευθεί η κατάσταση λειτουργίας STOP, το Microtick Counter τερματίζεται, καθώς το S-R Flip-Flop γίνεται reset, και άρα η έξοδος του τίθεται στο 0.

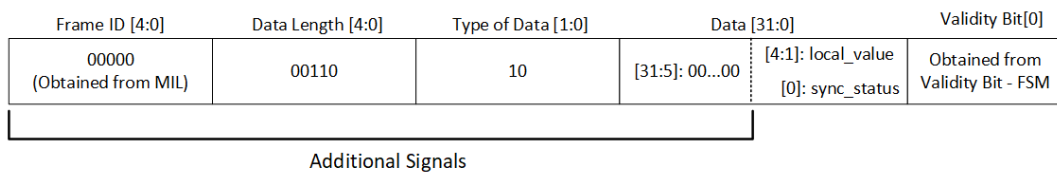
Η έξοδος του S-R FF ελέγχει το σήμα ενεργοποίησης ενός buffer, που επιτρέπει τη διέλευση του σήματος του καθολικού ρολογιού. Όταν το σήμα του καθολικού ρολογιού μεταβαίνει στην τιμή 1, ενεργοποιείται ένας μετρητής. Αυτός ο μετρητής αυξάνει σε κάθε θετική ακμή του τοπικού ρολογιού (Microtick), όσο παραμένει το σήμα του καθολικού στην τιμή 1 (Macrotick).

Κατά την κατερχόμενη ακμή του καθολικού ρολογιού, η έξοδος του buffer παίρνει την τιμή 0, και ο μετρητής απενεργοποιείται. Επίσης, χάρη στην πύλη NOT ενεργοποιείται ένας συγκριτής, που ελέγχει αν ο μετρούμενος αριθμός των Microticks αντιστοιχεί στο σωστό (Global Value στο σχήμα 5.23). Ο σωστός αριθμός των Microticks θα μπορούσε να λαμβάνεται από κάποιο διαιτητή του δικτύου. Αν οι τιμές λοιπόν ταυτίζονται, η έξοδος του συγκριτή είναι 1, διαφορετικά 0.

Στον επόμενο κύκλο ρολογιού το DFF2 επιτρέπει την έξοδο των δεδομένων, τα οποία αποτελούνται από την κατάσταση συγχρονισμού του κόμβου (έξοδος του συγκριτή), αλλά και από την τοπική τιμή, δηλαδή το μετρούμενο αριθμό των Microticks (έξοδος του DFF1).

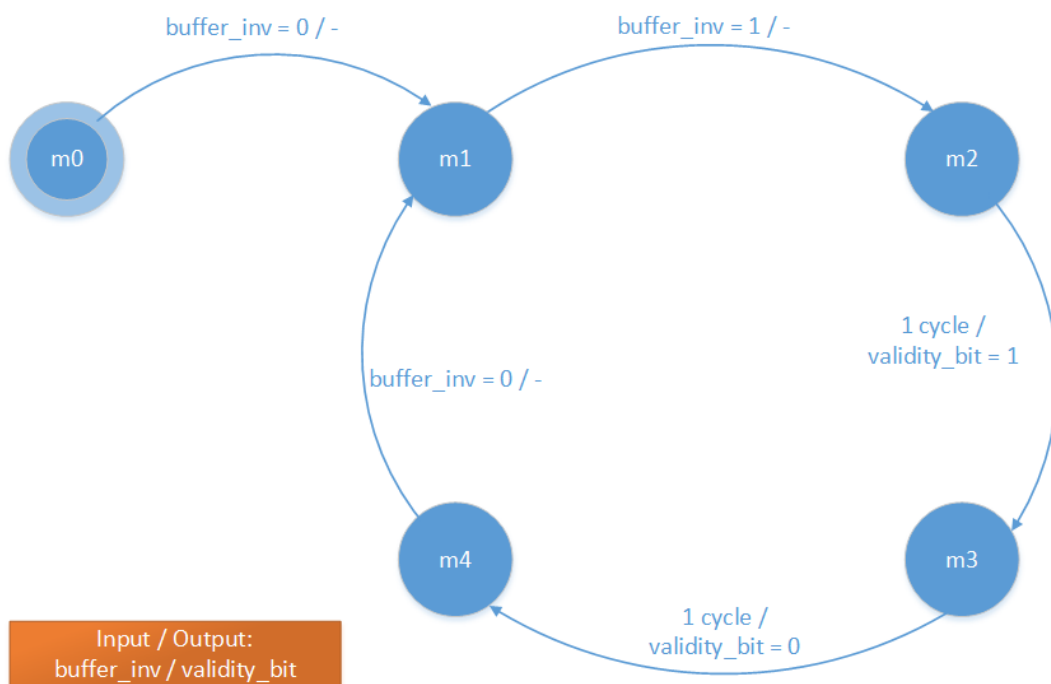
5.8.2 Πλαισίωση Δεδομένων

Για την αποστολή των δεδομένων εντός ενός πλαισίου, χρησιμοποιούνται κάποια επιπλέον σήματα. Αυτά φαίνονται πάνω δεξιά στο σχήμα 5.23 ως «Additional signals for framing the data», και προστίθενται στις εισόδους του καταχωρητή DFF2. Στο παρακάτω σχήμα απεικονίζεται η δομή του πλαισίου εξόδου.



Σχήμα 5.24: Πλαίσιο δεδομένων του Clock Synchronization

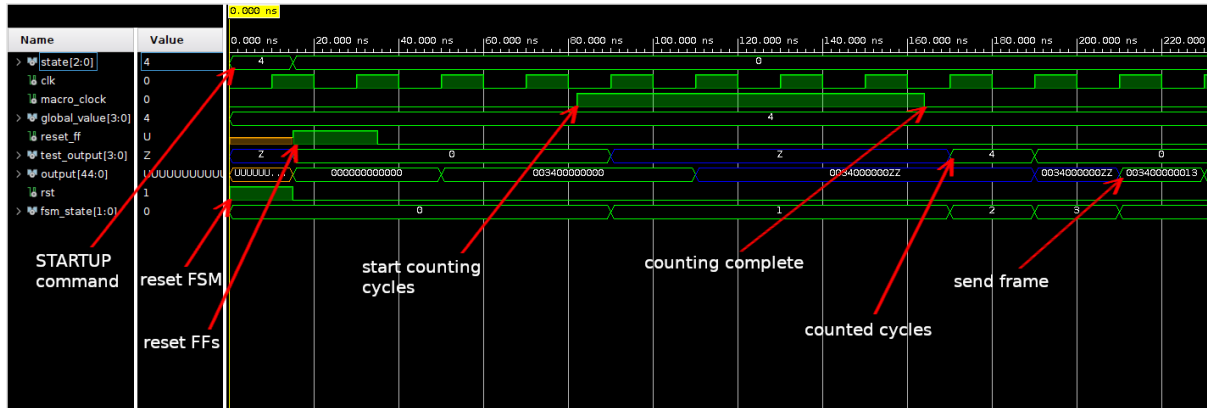
Όπως φαίνεται στα σχήματα 5.23 και 5.24, το ψηφίο εγκυρότητας λαμβάνεται από μια μηχανή πεπερασμένων καταστάσεων, «Validity Bit FSM». Η παρουσία αυτής της μηχανής είναι απαραίτητη, ώστε να αποστέλλονται τα δεδομένα μονάχα στον πρώτο κύκλο ρολογιού, μετά τον τερματισμό της μέτρησης των Microticks. Ο τερματισμός της μέτρησης σηματοδοτείται με την κάθοδο του σήματος του καθολικού ρολογιού. Στο σχήμα 5.25 απεικονίζεται το διάγραμμα καταστάσεων αυτής της FSM.



Σχήμα 5.25: Μηχανή πεπερασμένων καταστάσεων για το ψηφίο εγκυρότητας

- m0:** Αρχική κατάσταση, αναμονή για ενεργοποίηση του Microtick Counter.
- m1:** Έχει ενεργοποιηθεί το Microtick Counter, αφού `buffer_inv = 0` (έξοδος πύλης NOT στο σχήμα 5.23). Πραγματοποιείται η μέτρηση των Microticks.
- m2:** Ολοκλήρωση της μέτρησης των Microticks, απ' τη στιγμή που `buffer_inv = 1`. Χρειάζεται άλλος 1 κύκλος ρολογιού, για να εμφανιστούν τα δεδομένα στην έξοδο του DFF1.
- m3:** Το ψηφίο εγκυρότητας έχει τεθεί στην τιμή 1, αφού τα δεδομένα είναι διαθέσιμα για αποστολή.
- m4:** Το ψηφίο εγκυρότητας παίρνει την τιμή 0, αφού τα δεδομένα στάλθηκαν στον προηγούμενο κύκλο ρολογιού.
- m1:** Η διαδικασία μέτρησης έχει επανεκκινηθεί, αφού `buffer_inv = 0`.

5.8.3 Testbench



Σχήμα 5.26: Testbench #1 του Microtick Counter, Αρχείο: microtick_counter_tb.vhd

Απ' το παραπάνω σχήμα παρατηρούμε ότι, το περιεχόμενο του πλαισίου δεδομένων είναι: 0x03400000013

Το αρχικό ψηφίο (0) έχει προστεθεί απ' το εργαλείο εξομοίωσης. Παρατηρούμε ότι το τελευταίο ψηφίο είναι το 3, δηλαδή 0011 στο δυαδικό. Άρα, το πλαίσιο είναι έγκυρο, αφού το τελευταίο ψηφίο (validity bit) είναι 1.

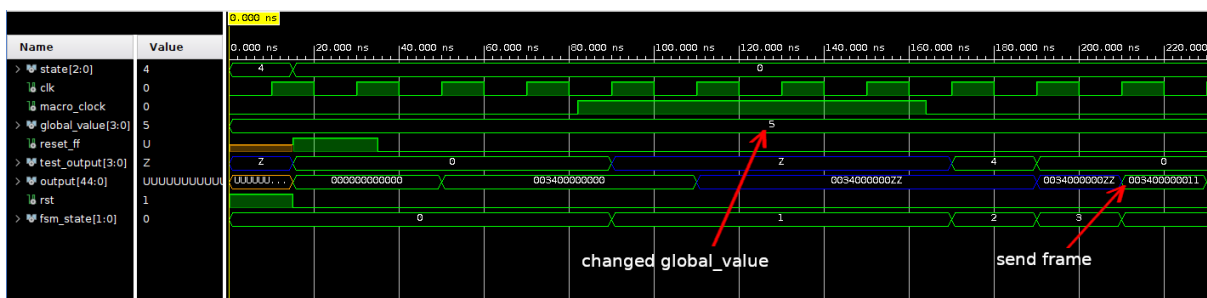
Μετατρέποντας τα περιεχόμενα του πλαισίου σε δυαδική μορφή παίρνουμε:

0000 0011 0100 0000...0000 0001 0011

Στο σχήμα 5.26 το global_value αντιπροσωπεύει το σωστό αριθμό κύκλων (Microticks) τοπικού ρολογιού.

Σύμφωνα με το σχήμα, έχουμε ότι οι συνολικοί κύκλοι που μετρήθηκαν είναι 100 στο δυαδικό, δηλαδή τέσσερις (= global_value). Επίσης, παρατηρούμε ότι το προτελευταίο ψηφίο (sync_status) είναι 1, δηλαδή ο κόμβος είναι συγχρονισμένος.

Δοκιμάζοντας να αλλάξουμε την τιμή global_value σε 5, παίρνουμε τα παρακάτω αποτελέσματα.



Σχήμα 5.27: Testbench #2 του Microtick Counter (changed global_value), Αρχείο: microtick_counter_tb.vhd

Σε αυτή την περίπτωση, το περιεχόμενο του πλαισίου δεδομένων είναι:

0x03400000011 = 0000 0011 0100 0000...0000 0001 0001

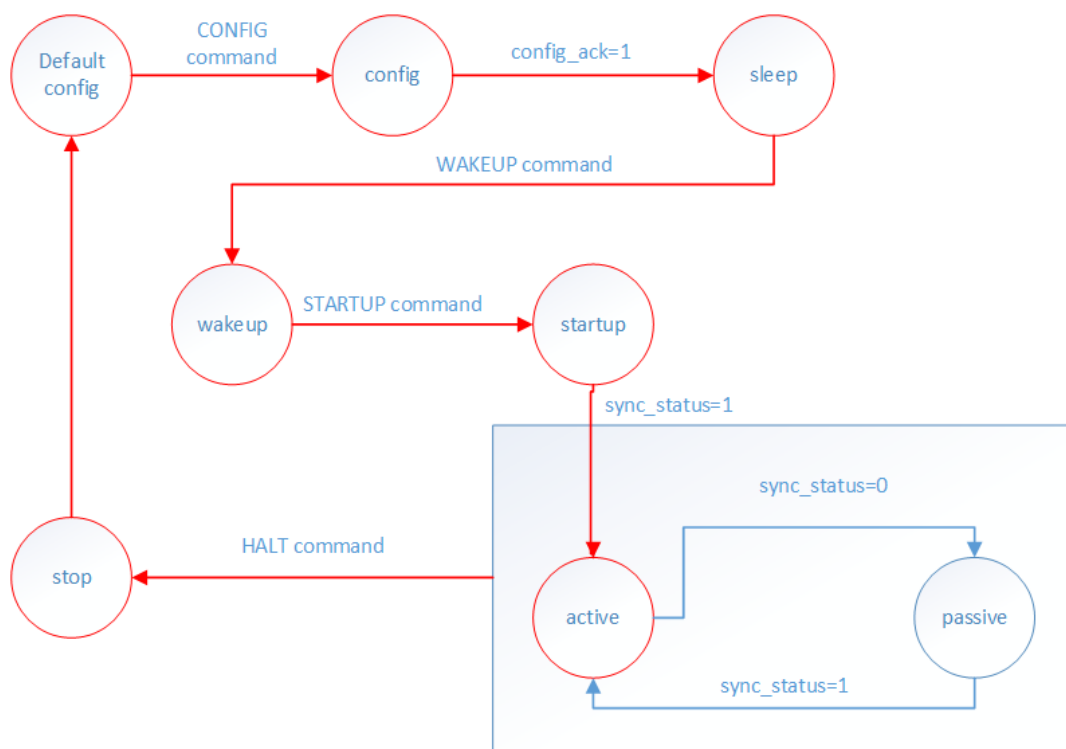
Παρατηρούμε ότι οι κύκλοι που μετρήθηκαν είναι πάλι 4 ($\neq$ global_value), αλλά το προτελευταίο ψηφίο είναι 0 (sync_status), αφού υπάρχει ασυμφωνία με το global_value.

5.9 Δοκιμές Συστήματος

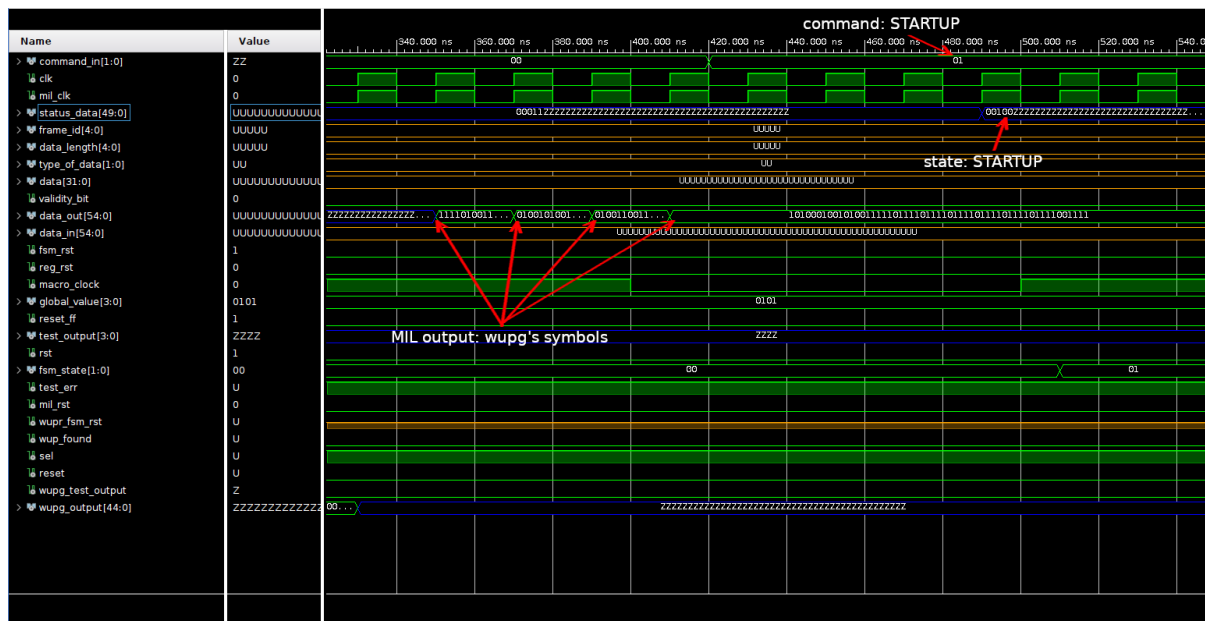
Σε αυτή την ενότητα πραγματοποιείται ένα testbench για την επιβεβαίωση της ορθής λειτουργίας ολόκληρου του συστήματος, το οποίο έχει προκύψει απ' την ενοποίηση των υπομονάδων, που περιγράφηκαν στις προηγούμενες ενότητες.

Για την επιβεβαίωση της ορθής λειτουργίας του συστήματος είναι απαραίτητη η παρακολούθηση των σημάτων, κατά τη διάρκεια μετάβασης μεταξύ διαφορετικών καταστάσεων λειτουργίας. Η τρέχουσα κατάσταση λειτουργίας του συστήματος καθορίζεται από τη μηχανή πεπερασμένων καταστάσεων στο Protocol Management Module, η οποία παρουσιάστηκε στην υποενότητα 5.3.2.

Στο παρακάτω σχήμα απεικονίζεται το διάγραμμα της FSM στο PMM. Στο διάγραμμα έχει επισημανθεί με κόκκινο χρώμα η ροή εκτέλεσης, στην οποία στοχεύει το testbench.



Σχήμα 5.28: Ροή εκτέλεσης κατά τη δοκιμή του συστήματος



Σχήμα 5.31: Testbench του Συστήματος (3/5)

Στο σχήμα 5.31 παρατηρούμε ότι, μετά από μερικούς κύκλους ρολογιού τα σύμβολα που παρήγαγε το Wakeup Pattern Generator, έχουν εμφανιστεί στις εξόδους του Medium Interface Layer (σήμα data_out) σε πλαίσια, τα οποία έχουν κωδικοποιηθεί.

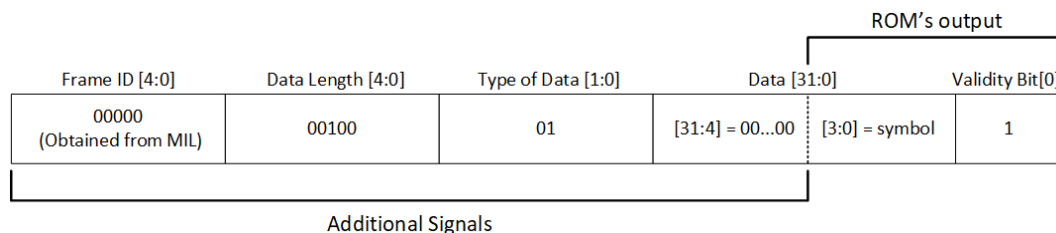
Τα περιεχόμενα του τελευταίου πλαισίου είναι τα παρακάτω:

10100|01001|01001|11110|11110|11110|11110|11110|11110|11110|01111

Αποκωδικοποιώντας τα παίρνουμε το παρακάτω αποτέλεσμα:

0010|0001|0001|0000|0000|0000|0000|0000|0000|0000|0111

Υπενθυμίζεται ότι η δομή του πλαισίου του WUPG είναι η παρακάτω:

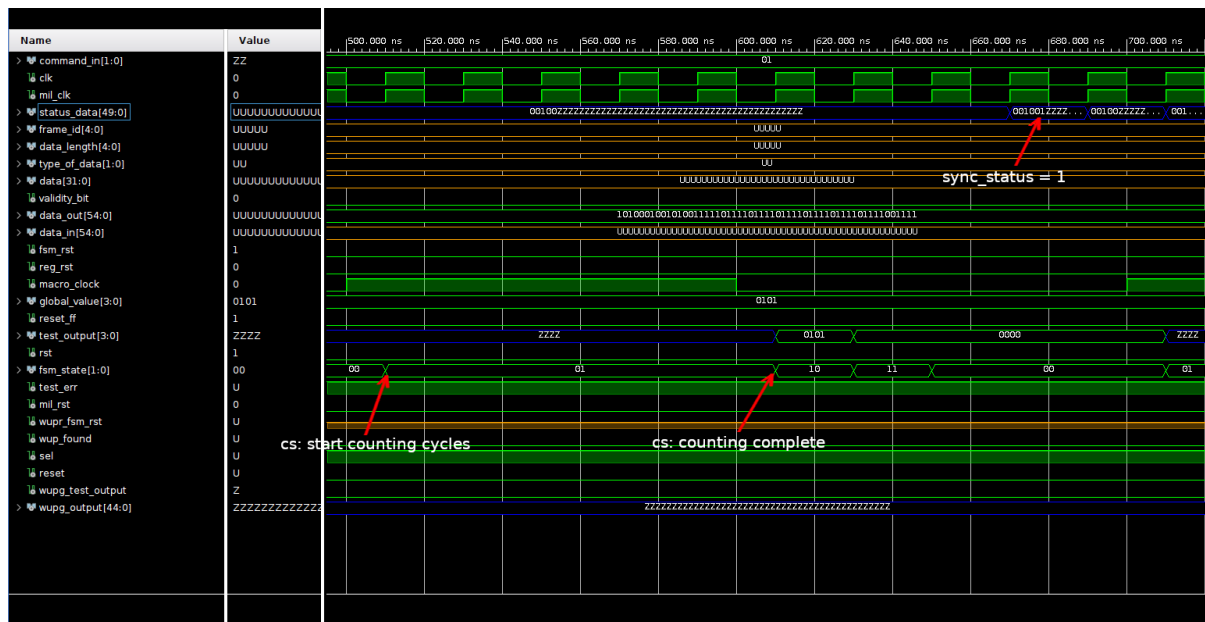


Σχήμα 5.32: Πλαίσιο δεδομένων του Wakeup Pattern Generator

Μένει να επιβεβαιώσουμε ότι τα περιεχόμενα του πλαισίου δεδομένων είναι ορθά. Αντιστοιχώντας τα περιεχόμενα του τελευταίου πλαισίου στα πεδία της παραπάνω εικόνας έχουμε:

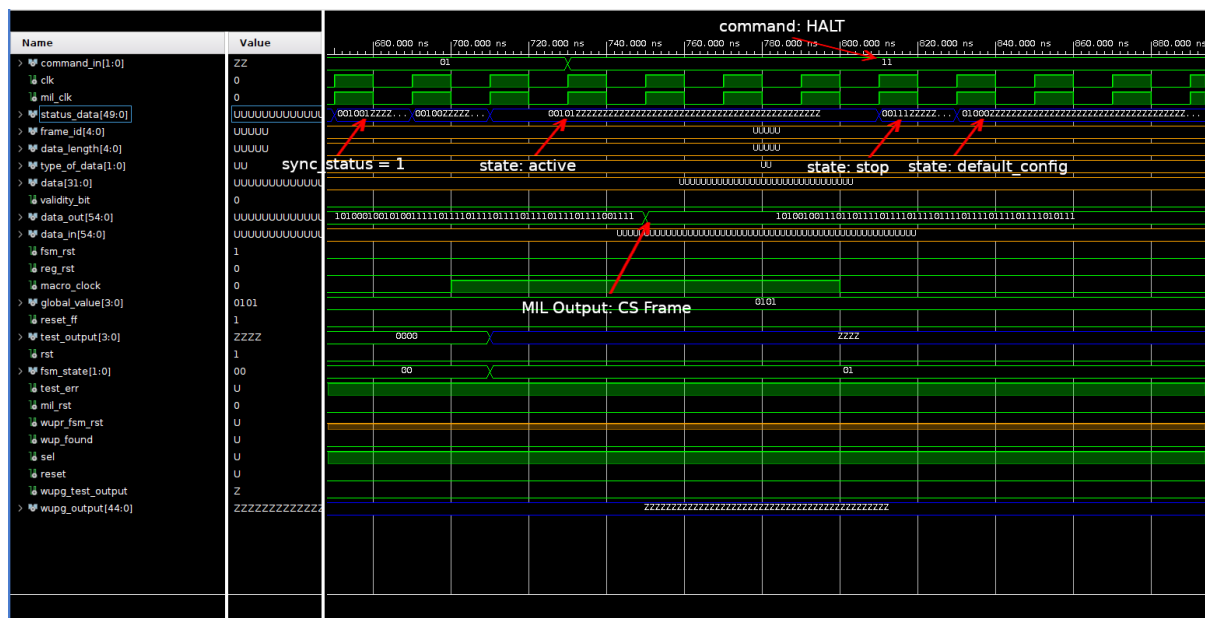
- Frame ID = 00100 = 4, δηλαδή είναι το 4ο πλαίσιο δεδομένων, που αποστέλλεται απ' το MIL.
- Data Length = 00100 = 4, δηλαδή το μήκος των δεδομένων είναι 4 bits.
- Type of Data = 01, δηλαδή το είδος δεδομένων είναι όντως ένα από τα σύμβολα αφύπνισης.
- Data[31:4] = 00...00
Data[3:0] = 0111 = 7, που είναι όντως το τελευταίο σύμβολο αφύπνισης, όπως αναμενόταν.
- Το ψηφίο εγκυρότητας έχει αφαιρεθεί στο MIL, γιατί έχει τοπική σημασία.

Απ' τα παραπάνω προκύπτει ότι, τα σύμβολα εμφανίζονται κωδικοποιημένα στις εξόδους του Medium Interface Layer επιτυχώς, για την αποστολή τους στο υπόλοιπο δίκτυο.



Σχήμα 5.33: Testbench του Συστήματος (4/5)

Στο σχήμα 5.31 το σύστημα εισήχθη σε κατάσταση STARTUP. Αυτό σημαίνει ότι, τώρα θα πρέπει να ακολουθήσει έλεγχος συγχρονισμού με το δίκτυο. Στο σχήμα 5.33 εκκινείται η διαδικασία μέτρησης κύκλων τοπικού ρολογιού, και μετρούνται συνολικά 5. Παρατηρούμε ότι, το σήμα global_value έχει την τιμή 0101 = 5. Αφού οι δύο τιμές ταυτίζονται ο συγχρονισμός είναι επιτυχής, και το σήμα sync_status (6ο ψηφίο του σήματος status_data) τίθεται στο 1.



Σχήμα 5.34: Testbench του Συστήματος (5/5)

Αφού ο συγχρονισμός με το δίκτυο ήταν επιτυχής, το σύστημα μεταβαίνει σε κατάσταση active, όπως φαίνεται και στο σχήμα 5.34.

Επίσης, στο ίδιο σχήμα παρατηρούμε ότι στις εξόδους του Medium Interface Layer εμφανίζεται και το πλαίσιο δεδομένων απ' την υπομονάδα Clock Synchronization.

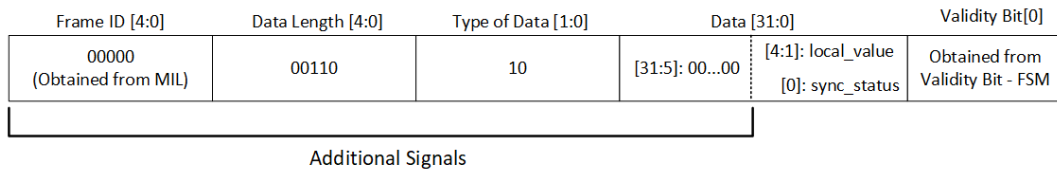
Τα περιεχόμενα του πλαισίου δεδομένων είναι τα παρακάτω:

10100|10011|10110|11110|11110|11110|11110|11110|11110|11110|10111

Αποκωδικοποιώντας τα παίρνουμε το παρακάτω αποτέλεσμα:

0010|1001|1010|0000|0000|0000|0000|0000|0000|0000|1011

Υπενθυμίζεται ότι η δομή του πλαισίου δεδομένων του CS είναι η παρακάτω:



Σχήμα 5.35: Πλαίσιο δεδομένων του Clock Synchronization

Μένει να επιβεβαιώσουμε ότι τα περιεχόμενα του πλαισίου δεδομένων είναι ορθά. Αντιστοιχώντας τα περιεχόμενα με τα πεδία στην παραπάνω εικόνα έχουμε:

- Frame ID = 00101 = 5, δηλαδή είναι το 5ο πλαίσιο δεδομένων ,που αποστέλλεται απ' το MIL.
- Data Length = 00110 = 6, δηλαδή το μήκος των δεδομένων είναι 6 bits.
- Type of Data = 10, δηλαδή τα δεδομένα αφορούν όντως την υπομονάδα Clock Synchronization.
- Data[31:5] = 00...00
Data[4:1] = 0101 = 5, δηλαδή ο αριθμός των κύκλων ρολογιού που μετρήθηκαν.
Data[0] = 1, δηλαδή ο συγχρονισμός ήταν επιτυχής.
- Το ψηφίο εγκυρότητας έχει αφαιρεθεί στο MIL, γιατί έχει τοπική σημασία.

Τα παραπάνω επιβεβαιώνουν την ορθή αποστολή των δεδομένων απ' την υπομονάδα Clock Synchronization.

Ακόμη, όπως φαίνεται στο σχήμα 5.34, η εισαγωγή της εντολής HALT έχει ως αποτέλεσμα τη μετάβαση του Communication Controller στην κατάσταση stop, κι έπειτα στην κατάσταση default_config για την επαναφορά του.

5.10 Σύνθεση Συστήματος

5.10.1 Γενικές Πληροφορίες

Η σύνθεση αποτελεί ένα απ' τα στάδια της ροής σχεδιασμού πάνω σε ένα FPGA. Η ροή σχεδιασμού (για περιγραφές RTL ή Register-Transfer Logic) αποτελείται απ' τα ακόλουθα βήματα:

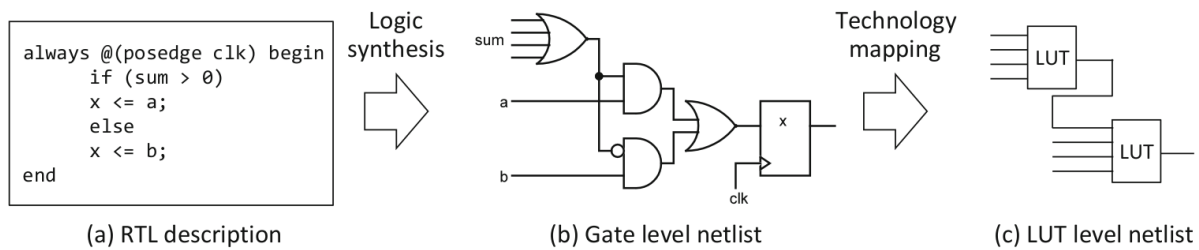
1. Λογική Σύνθεση (Logic Synthesis)
2. Χαρτογράφηση Τεχνολογίας (Technology Mapping)
3. Τοποθέτηση και Δρομολόγηση (Place and Route)
4. Δημιουργία δεδομένων ρύθμισης παραμέτρων (Generating Configuration Data)
5. Προγραμματισμός FPGA

6. Εκτέλεση σε FPGA

Η σύνθεση στο εργαλείο που χρησιμοποιήθηκε (Xilinx Vivado), πραγματοποιεί τα δύο πρώτα βήματα της ροής σχεδιασμού.

Η Λογική Σύνθεση είναι η διαδικασία, κατά την οποία δημιουργείται ένα λογικό και ακολουθιακό κύκλωμα από μια περιγραφή RTL (π.χ. περιγραφή σε VHDL). Το αποτέλεσμα της σύνθεσης είναι γνωστό ως «netlist», το οποίο αποτελείται από ένα σύνολο λογικών στοιχείων, όπως λογικές πύλες, Flip-Flops, και διασυνδέσεις μεταξύ των στοιχείων.

Η Χαρτογράφηση Τεχνολογίας (Technology Mapping) είναι η διαδικασία, κατά την οποία τα λογικά στοιχεία του netlist εκχωρούνται σε πραγματικά λογικά στοιχεία του FPGA.[6]



Σχήμα 5.36: Λογική Σύνθεση & Χαρτογράφηση Τεχνολογίας από περιγραφή RTL

5.10.2 Αποτελέσματα Σύνθεσης

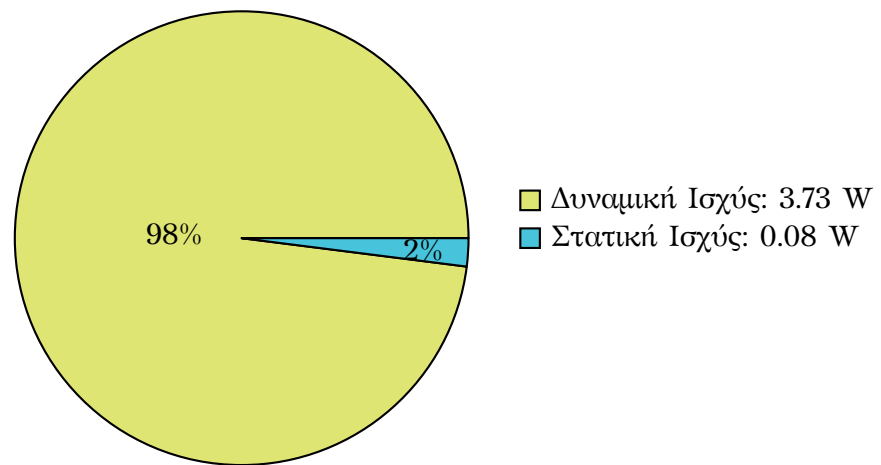
Για τη σύνθεση χρησιμοποιήθηκε το FPGA xc7a15tfgg484-3 της οικογένειας Artix-7, με βάση κριτήρια βελτιστοποίησης της αξιοποίησης των πόρων. Ακολουθούν τα αποτελέσματα της σύνθεσης.

Component	Slice LUTs	Slice Registers	Bonded IOB	BUFGCTRL
Clock Synchronization	52	22	0	0
Medium Interface Layer	50	364	0	0
Protocol Management Module	7	9	0	0
Wakeup Pattern Generator	5	11	0	0
System (Total)	114	406	228	2

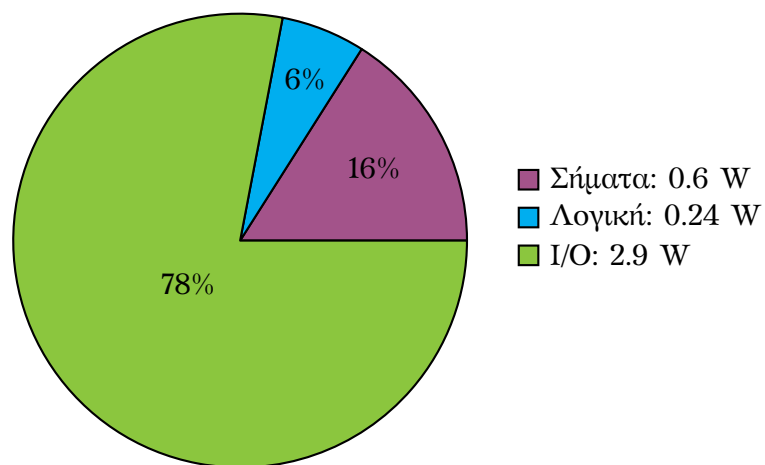
Πίνακας 5.6: Χρησιμοποίηση πόρων στην ιεραρχία

Resource	Utilization	Available	Utilization (%)
LUT	114	10400	1.10
FF	406	20800	1.95
IO	228	250	91.20

Πίνακας 5.7: Σύνοψη χρησιμοποίησης πόρων



Σχήμα 5.37: Κατανάλωση Ισχύος



Σχήμα 5.38: Κατανάλωση Δυναμικής Ισχύος

6. Συμπεράσματα & Μελλοντική Εργασία

6.1 Συμπεράσματα

Η ραγδαία εξέλιξη της τεχνολογίας έχει οδηγήσει στην εφαρμογή ενσωματωμένων συστημάτων σε πολλούς τομείς, άρα και τη συνεχής εξάρτηση του ανθρώπου απ' αυτά. Η ανάγκη για την προσθήκη νέων εφαρμογών, και ιδιαίτερα αυτών πραγματικού χρόνου και ασφαλείας, προϋποθέτουν την υιοθέτηση διαφορετικών πρωτοκόλλων λειτουργίας αυτών των ενσωματωμένων συστημάτων.

Αυτό το φαινόμενο παρατηρείται στα αυτοκίνητα, τα οποία χρησιμοποιούν πολυάριθμα ενσωματωμένα συστήματα (Electronic Control Units ή ECUs) για ποικίλες εφαρμογές. Έχει διαπιστωθεί ότι χρειάζεται η υιοθέτηση ενός πιο προηγμένου πρωτοκόλλου επικοινωνίας, με υψηλότερες ταχύτητες μετάδοσης των δεδομένων, όπως το FlexRay. Όμως, η εφαρμογή του αποτελεί μια πρόκληση, αφού οδηγεί σε μεγαλύτερη κατανάλωση ενέργειας και εύρους ζώνης.

Λύση σ' αυτό το πρόβλημα θα μπορούσε να δώσει η επαύξηση των δυνατοτήτων του Communication Controller εντός ενός κόμβου (ECU) του δικτύου. Αυτό θα είχε ως αποτέλεσμα τη βελτίωση της ποιότητας και αποτελεσματικότητας του συστήματος, επιτρέποντας ταυτόχρονα και την ενσωμάτωση περισσότερων λειτουργιών. Στο κεφάλαιο 4 παρουσιάστηκε αναλυτικά μια τέτοια προσέγγιση, αλλά και πειραματικά αποτελέσματα που αποδεικνύουν την ορθότητα της συγκεκριμένης πρότασης.

Σε αυτή τη διπλωματική εργασία, σχεδιάστηκε και αναπτύχθηκε ένα Communication Controller, σύμφωνα με τις προαναφερθείσες περιγραφές. Οι επιπρόσθετες υπομονάδες εντός του Communication Controller του προσδίδουν επιπλέον λειτουργικότητα, η οποία κατά κανόνα θα πραγματοποιούνταν στον Host/Microcontroller. Συγκεκριμένα, περιγράφηκε η λειτουργικότητά τους, παρουσιάστηκαν πιθανές αρχιτεκτονικές τους, και τέλος ακολούθησε μια εξομοίωση για την επιβεβαίωση της ορθής λειτουργίας τους. Τέλος, ολοκληρώθηκε το σύστημα με την ενοποίησή τους, και επιβεβαιώθηκε η ορθή λειτουργία του μέσω κατάλληλου testbench.

6.2 Μελλοντική Εργασία

Αρχικά, ως προτεραιότητα για τη συνέχεια του έργου της διπλωματικής εργασίας αποτελεί η εξομοίωση της λειτουργικότητας του συγκεκριμένου Communication Controller σε ένα εικονικό δίκτυο, για τη μέτρηση της αποδοτικότητάς του. Στη συνέχεια, θα μπορούσε να αναπτυχθεί ένα παρόμοιο Communication Controller, αλλά με λιγότερες λειτουργίες. Στην προκειμένη περίπτωση, οι απύουσες λειτουργίες θα εκτελούνταν απ' τον Host. Έτσι, θα μπορούσε να συγκριθεί η αποδοτικότητα και η κατανάλωση ενέργειας και στις δύο περιπτώσεις, αποδεικνύοντας την αποτελεσματικότητα της λύσης που προτάθηκε.

Επίσης, θα μπορούσαν να πραγματοποιηθούν περαιτέρω βελτιώσεις στις αρχιτεκτονικές και τη λειτουργικότητα των υπομονάδων, που παρουσιάστηκαν στην παρούσα διπλωματική εργασία.

Για παράδειγμα, ο έλεγχος του συγχρονισμού του κόμβου, όταν αυτό βρίσκεται είτε σε κατάσταση active είτε σε κατάσταση passive, θα μπορούσε να πραγματοποιούνταν σε διαφορετικό

άριθμο κύκλων ρολογιού. Αυτό θα εξαρτώταν από την εκτελούμενη εφαρμογή στον εκάστοτε κόμβο. Έτσι, θα μπορούσε να καταναλωθεί λιγότερη ενέργεια, αφού κάποιες εφαρμογές δεν είναι ιδιαίτερα κρίσιμες, και επομένως δε χρειάζεται να συγχρονιστούν άμεσα οι κόμβοι τους.

Ακόμη, στο Medium Interface Layer (ενότητα 5.5), μετά την ανίχνευση σφάλματος σε ένα πλαίσιο δεδομένων, θα μπορούσε να στελνόταν μια ειδοποίηση στον αποστολέα (μήνυμα ACK), γνωστοποιώντας το σφάλμα, για να ακολουθήσει η επαναποστολή του πλαισίου.

Τέλος, θα μπορούσε να πραγματοποιηθεί μια αναθεώρηση των αρχιτεκτονικών για λιγότερη κατανάλωση ενέργειας, υψηλότερη ταχύτητα επεξεργασίας, και ελαχιστοποίηση χρησιμοποιούμενης επιφάνειας, με βάση το επιλεγμένο FPGA.

Παραρτήματα

A . Κώδικας

Σε αυτό το κεφάλαιο παρουσιάζεται ο κώδικας, που χρησιμοποιήθηκε για την υλοποίηση της παρούσας διπλωματικής εργασίας. Εκτός από τις περιγραφές των κυκλωμάτων, υπάρχουν και αντίστοιχα testbenches για τη δοκιμή τους (όνομα με κατάληξη _tb). Στην ενότητα [Α.7](#) υπάρχει ο κώδικας του συστήματος, που ενοποιεί όλες τις υπομονάδες εντός του Communication Controller.

Να σημειωθεί ότι, κάποια components (π.χ. dff.vhd) παρά που κατατάσσονται μόνο σε μια ενότητα, μπορεί να χρησιμοποιούνται και από άλλες μονάδες. Επίσης, οι κώδικες «main» υλοποιούν το σύστημα στην εκάστοτε υπομονάδα, καθώς ενοποιούν τα υπόλοιπα components.

A .1 Protocol Management Module

fsm.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY fsm IS
    PORT (command      : IN std_logic_vector(1 DOWNTO 0);
          sync_status  : IN std_logic;
          clk, rst      : IN std_logic;

          curr_state    : OUT std_logic_vector(2 DOWNTO 0);
          reset_ack     : OUT std_logic;
          wup_en        : OUT std_logic;
          config_ack    : OUT std_logic);
END fsm;

-- η αρχιτεκτονική βασίζεται στο στυλ σχεδιασμού #2 από το βιβλίο του Pedroni
ARCHITECTURE fsm_default OF fsm IS
    TYPE states IS (default_config, config, sleep, wakeup, startup, active, passive, stop);
    SIGNAL prv_state, nxt_state : states;
    SIGNAL temp_reset_ack, temp_wup_en, temp_config_ack : std_logic;
BEGIN

    PROCESS (rst, clk)
    BEGIN
        IF (rst = '1') THEN
            curr_state <= std_logic_vector(to_unsigned(states'pos(default_config), 3));
            prv_state <= default_config;
        ELSIF (clk'event AND clk = '1') THEN
            curr_state <= std_logic_vector(to_unsigned(states'pos(nxt_state), 3));
            reset_ack <= temp_reset_ack;
            wup_en <= temp_wup_en;
            config_ack <= temp_config_ack;
            prv_state <= nxt_state;
        END IF;
    END PROCESS;

    PROCESS (prv_state, command, sync_status)
    BEGIN
        CASE prv_state IS
```

```

WHEN default_config =>
  IF (command = "10") THEN
    temp_reset_ack <= '0';
    nxt_state      <= config;
  END IF;

WHEN config =>
  temp_config_ack <= '1';
  nxt_state      <= sleep;

WHEN sleep =>
  IF (command = "00") THEN
    temp_config_ack <= '0';
    temp_wup_en     <= '1';
    nxt_state       <= wakeup;
  END IF;

WHEN wakeup =>
  temp_wup_en <= '0'; -- ώστε να παραμένει στο 1 μόνο για 1 κύκλο ρολογιού
  IF (command = "01") THEN
    nxt_state <= startup;
  END IF;

WHEN startup =>
  IF (sync_status = '1') THEN
    nxt_state <= active;
  END IF;

WHEN active =>
  IF (command = "11") THEN
    nxt_state <= stop;
  ELSIF (sync_status = '0') THEN
    nxt_state <= passive;
  END IF;

WHEN passive =>
  IF (command = "11") THEN
    nxt_state <= stop;
  ELSIF (sync_status = '1') THEN
    nxt_state <= active;
  END IF;

WHEN stop =>
  temp_reset_ack <= '1';
  nxt_state      <= default_config;

END CASE;
END PROCESS;

END fsm_default;

```

fsm_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS
  SIGNAL command      : std_logic_vector(1 DOWNTO 0);
  SIGNAL sync_status, rst : std_logic;
  SIGNAL clk           : std_logic := '0';

  SIGNAL curr_state    : std_logic_vector(2 DOWNTO 0);
  SIGNAL reset_ack, wup_en, config_ack : std_logic;
BEGIN
  fsm : ENTITY work.fsm PORT MAP (command => command, sync_status => sync_status, clk => clk, rst => rst,
    curr_state => curr_state, reset_ack => reset_ack, wup_en => wup_en, config_ack => config_ack);

  clk <= NOT clk AFTER 10 ns;

PROCESS

```

```

BEGIN
    sync_status <= '0';
    rst <= '1';
    WAIT FOR 25 ns;

    rst <= '0';
    command <= "10";
    WAIT FOR 40 ns;

    command <= "00";
    WAIT FOR 30 ns;

    command <= "01";
    WAIT FOR 25 ns;

    sync_status <= '1';
    WAIT FOR 25 ns;

    sync_status <= '0';
    WAIT FOR 25 ns;

    command <= "11";
    WAIT FOR 80 ns;

END PROCESS;

```

```
END tb;
```

dff.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY dff IS
    GENERIC (d_len : integer := 1;
             q_len : integer := 1);
    PORT (d : IN std_logic_vector(d_len-1 DOWNTO 0);
          q : OUT std_logic_vector(q_len-1 DOWNTO 0);
          clk, rst, en : IN std_logic);
END dff;

ARCHITECTURE default OF dff IS -- architecture with asynchronous reset
BEGIN

    PROCESS(rst, clk)
    BEGIN
        CASE en IS
            WHEN '1' =>
                CASE rst IS
                    WHEN '1' => q <= (OTHERS => '0');
                    WHEN '0' =>
                        IF (clk'event AND clk = '1') THEN
                            q <= d;
                        END IF;
                    WHEN OTHERS => NULL;
                END CASE;
            WHEN OTHERS =>
                q <= (OTHERS => 'Z');
            END CASE;
        END PROCESS;
    END default;

```

pmm.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

```

```

ENTITY pmm IS
  PORT (command      : IN std_logic_vector(1 DOWNTO 0);
        sync_status  : IN std_logic;
        clk, fsm_rst, reg_rst : IN std_logic;

        curr_state : OUT std_logic_vector(2 DOWNTO 0);
        reset_ack  : OUT std_logic;
        wup_en     : OUT std_logic;
        config_ack : OUT std_logic);
END pmm;

ARCHITECTURE pmm_arch OF pmm IS
  COMPONENT dff IS
    GENERIC (d_len : integer := 1;
             q_len : integer := 1);
    PORT (d      : IN  std_logic_vector(d_len-1 DOWNTO 0);
          q      : OUT std_logic_vector(q_len-1 DOWNTO 0);
          clk, rst, en : IN  std_logic);
  END COMPONENT;
  COMPONENT fsm IS
    PORT (command      : IN std_logic_vector(1 DOWNTO 0);
          sync_status  : IN std_logic;
          clk, rst     : IN std_logic;

          curr_state : OUT std_logic_vector(2 DOWNTO 0);
          reset_ack  : OUT std_logic;
          wup_en     : OUT std_logic;
          config_ack : OUT std_logic);
  END COMPONENT;

  SIGNAL wire0 : std_logic_vector(2 DOWNTO 0);
BEGIN
  reg0 : dff GENERIC MAP (d_len => 3, q_len => 3) PORT MAP (d(2 DOWNTO 1) => command, d(0) => sync_status, q
    => wire0, clk => clk, rst => reg_rst, en => '1');

  fsm0 : fsm PORT MAP (command => wire0(2 DOWNTO 1), sync_status => wire0(0), clk => clk, rst => fsm_rst,
    curr_state => curr_state, reset_ack => reset_ack, wup_en => wup_en, config_ack => config_ack);

END pmm_arch;

```

pmm_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY pmm_testbench IS
END pmm_testbench;

ARCHITECTURE pmm_tb OF pmm_testbench IS
  SIGNAL command      : std_logic_vector(1 DOWNTO 0);
  SIGNAL sync_status  : std_logic;
  SIGNAL fsm_rst, reg_rst : std_logic;
  SIGNAL clk          : std_logic := '0';

  SIGNAL curr_state : std_logic_vector(2 DOWNTO 0);
  SIGNAL reset_ack  : std_logic;
  SIGNAL wup_en     : std_logic;
  SIGNAL config_ack : std_logic;
BEGIN
  pmm : ENTITY work.pmm PORT MAP (command => command, sync_status => sync_status, clk => clk, fsm_rst =>
    fsm_rst, reg_rst => reg_rst, curr_state => curr_state, reset_ack => reset_ack, wup_en => wup_en,
    config_ack => config_ack);

  clk      <= NOT clk AFTER 10 ns;
  reg_rst <= '0';

  PROCESS
  BEGIN
    sync_status <= '0';

```

```

    fsm_rst      <= '1';
    WAIT FOR 25 ns;

    fsm_rst <= '0';
    command <= "10";
    WAIT FOR 80 ns;

END PROCESS;

END pmm_tb;

```

A .2 Controller Host Interface

chi.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY chi IS
    PORT (
        -- input, direction: micro => communication
        command_in  : IN  std_logic_vector(1 DOWNTO 0);
        --corresponding output
        command_out  : OUT std_logic_vector(1 DOWNTO 0);
        -- input, origin: micro
        clk          : IN  std_logic;
        -- inputs, direction: communication => micro
        config_ack   : IN  std_logic;
        reset_ack    : IN  std_logic;
        state        : IN  std_logic_vector(2 DOWNTO 0);
        sync_status  : IN  std_logic;
        frame_rcv    : IN  std_logic_vector(43 DOWNTO 0);
        -- corresponding output
        status_data   : OUT std_logic_vector(49 DOWNTO 0);
        -- inputs, direction: micro => communication
        frame_id      : IN  std_logic_vector(4 DOWNTO 0);
        data_length   : IN  std_logic_vector(4 DOWNTO 0);
        type_of_data  : IN  std_logic_vector(1 DOWNTO 0);
        data          : IN  std_logic_vector(31 DOWNTO 0);
        validity_bit  : IN  std_logic;
        -- corresponding output
        frame_send    : OUT std_logic_vector(44 DOWNTO 0));
END chi;

ARCHITECTURE chi_interface OF chi IS
    COMPONENT dff IS
        GENERIC (d_len : integer := 1;
                 q_len : integer := 1);
        PORT (d      : IN  std_logic_vector(d_len-1 DOWNTO 0);
              q      : OUT std_logic_vector(q_len-1 DOWNTO 0);
              clk, rst, en : IN  std_logic);
    END COMPONENT;

    signal temp : std_logic_vector(4 downto 0);
BEGIN
    temp <= config_ack & reset_ack & state;

    reg0 : dff GENERIC MAP (d_len => 2, q_len => 2) PORT MAP (d => command_in, q => command_out, clk => clk,
        rst => '0', en => '1');

    reg1 : dff GENERIC MAP (d_len => 50, q_len => 50) PORT MAP (d(49 DOWNTO 45) => temp, d(44) => sync_status,
        d(43 DOWNTO 0) => frame_rcv, q => status_data, clk => clk, rst => '0', en => '1');

    reg2 : dff GENERIC MAP (d_len => 45, q_len => 45) PORT MAP (d(44 DOWNTO 40) => frame_id, d(39 DOWNTO 35) =>
        data_length, d(34 DOWNTO 33) => type_of_data, d(32 DOWNTO 1) => data, d(0) => validity_bit, q =>
        frame_send, clk => clk, rst => '0', en => '1');

```

```
END chi_interface;
```

main_tb.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS
    SIGNAL command_in : std_logic_vector(1 DOWNTO 0);
    SIGNAL command_out : std_logic_vector(1 DOWNTO 0);

    SIGNAL clk : std_logic := '0';

    SIGNAL config_ack, reset_ack, sync_status : std_logic;
    SIGNAL state : std_logic_vector(2 DOWNTO 0);
    SIGNAL frame_rcv : std_logic_vector(43 DOWNTO 0);

    SIGNAL status_data : std_logic_vector(49 DOWNTO 0);

    SIGNAL frame_id, data_length : std_logic_vector(4 DOWNTO 0);
    SIGNAL type_of_data : std_logic_vector(1 DOWNTO 0);
    SIGNAL data : std_logic_vector(31 DOWNTO 0);
    SIGNAL validity_bit : std_logic;

    SIGNAL frame_send : std_logic_vector(44 DOWNTO 0);
BEGIN
    interface : ENTITY work.chi PORT MAP (command_in => command_in, command_out => command_out, clk => clk,
        config_ack => config_ack, reset_ack => reset_ack, sync_status => sync_status, state => state, frame_rcv
        => frame_rcv, status_data => status_data, frame_id => frame_id, data_length => data_length, type_of_data
        => type_of_data, data => data, validity_bit => validity_bit, frame_send => frame_send);

    clk <= NOT clk AFTER 10 ns;

    PROCESS
    BEGIN
        -- reg0
        command_in <= "10";
        WAIT FOR 25 ns;
        -- reg1
        config_ack <= '1';
        reset_ack <= '0';
        state <= "110";
        sync_status <= '1';
        frame_rcv <= (7 => '0', OTHERS => '1');
        WAIT FOR 25 ns;
        -- reg2
        frame_id <= (OTHERS => '1');
        data_length <= (OTHERS => '1');
        type_of_data <= (OTHERS => '1');
        data <= (OTHERS => '1');
        validity_bit <= '0';
        WAIT FOR 25 ns;
    END PROCESS;

END tb;
```

A .3 Medium Interface Layer

comparator.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE work.encoding.ALL;

ENTITY comparator IS
  PORT (input      : IN   std_logic_vector(4 DOWNIO 0);
        output     : OUT  std_logic_vector(4 DOWNIO 0);
        en, clk     : IN   std_logic;
        found_error : OUT  std_logic);
END comparator;

ARCHITECTURE default OF comparator IS
BEGIN
  PROCESS (clk)
  BEGIN
    CASE en IS
      WHEN '1' =>
        IF (clk'event AND clk = '1') THEN
          check(input, output, found_error);
        END IF;
      WHEN OTHERS =>
        output      <= (OTHERS => 'Z');
        found_error <= 'Z';
    END CASE;
  END PROCESS;
END default;

```

encoding.vhd (package)

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

PACKAGE encoding IS

  -- πίνακες για την αποθήκευση αντιστοιχιών 4b5b κωδικοποίησης/αποκωδικοποίησης/
  TYPE array4b IS ARRAY (0 TO 15) OF std_logic_vector(3 DOWNIO 0);
  CONSTANT array0 : array4b := (
    ("0000"), ("0001"), ("0010"), ("0011"), ("0100"), ("0101"),
    ("0110"), ("0111"), ("1000"), ("1001"), ("1010"), ("1011"),
    ("1100"), ("1101"), ("1110"), ("1111")
  );
  TYPE array5b IS ARRAY (0 TO 15) OF std_logic_vector(4 DOWNIO 0);
  CONSTANT array1 : array5b := (
    ("11110"), ("01001"), ("10100"), ("10101"), ("01010"),
    ("01011"), ("01110"), ("01111"), ("10010"), ("10011"),
    ("10110"), ("10111"), ("11010"), ("11011"), ("11100"),
    ("11101")
  );

  -- κωδικοποίηση
  PROCEDURE encode(SIGNAL input  : IN   std_logic_vector(3 DOWNIO 0);
                  SIGNAL output : OUT  std_logic_vector(4 DOWNIO 0));

  -- αποκωδικοποίηση
  PROCEDURE decode(SIGNAL input  : IN   std_logic_vector(4 DOWNIO 0);
                  SIGNAL output : OUT  std_logic_vector(3 DOWNIO 0));

  -- έλεγχος για σφάλματα
  PROCEDURE check(SIGNAL input      : IN   std_logic_vector(4 DOWNIO 0);
                  SIGNAL output     : OUT  std_logic_vector(4 DOWNIO 0);
                  SIGNAL found_error : OUT  std_logic);

END encoding;

PACKAGE BODY encoding IS

  PROCEDURE check(SIGNAL input : IN   std_logic_vector(4 DOWNIO 0); SIGNAL output : OUT  std_logic_vector(4
    DOWNIO 0); SIGNAL found_error : OUT  std_logic) IS
    VARIABLE found_match : boolean := false; --η μεταβλητή δηλώνει αν βρέθηκε
        --αντιστοιχία, αρχικά υποθέτουμε ότι
        -- δε βρέθηκε
    VARIABLE counter    : integer := 0;

```

```

BEGIN
  c0 : FOR i IN 0 TO 15 LOOP
    IF (input = array1(counter)) THEN      -- αν υπάρχει αντιστοιχία
      output <= input;
      found_match := true;
      found_error <= '0';                  -- δηλώνουμε ότι δεν υπάρχει σφάλμα
      EXIT;
    END IF;
    counter := counter + 1;
  END LOOP;
  IF (NOT found_match) THEN                --αν δε βρέθηκε αντιστοιχία
    found_error <= '1';                    -- δηλώνουμε ότι υπάρχει σφάλμα
    output <= (OTHERS => 'Z');
  END IF;
END check;

PROCEDURE encode(SIGNAL input : std_logic_vector(3 DOWNTO 0); SIGNAL output : OUT std_logic_vector(4 DOWNTO 0)) IS
  VARIABLE counter : integer := 0;
BEGIN
  c0 : FOR i IN 0 TO 15 LOOP
    IF (input = array0(counter)) THEN
      output <= array1(counter);
      EXIT;
    END IF;
    counter := counter + 1;
  END LOOP;
END encode;

PROCEDURE decode(SIGNAL input : std_logic_vector(4 DOWNTO 0); SIGNAL output : OUT std_logic_vector(3 DOWNTO 0)) IS
  VARIABLE counter : integer := 0;
BEGIN
  c0 : FOR i IN 0 TO 15 LOOP
    IF (input = array1(counter)) THEN
      output <= array0(counter);
      EXIT;
    END IF;
    counter := counter + 1;
  END LOOP;
END decode;

END encoding;

```

encoder.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE work.encoding.ALL;

ENTITY encoder IS
  PORT (input : IN std_logic_vector(43 DOWNTO 0);
        en, clk : IN std_logic;
        output : OUT std_logic_vector(54 DOWNTO 0) := (OTHERS => 'Z'));
END encoder;

ARCHITECTURE default OF encoder IS
BEGIN
  PROCESS (clk)
  BEGIN
    CASE en IS
      WHEN '1' =>
        IF (clk'event AND clk = '1') THEN
          encode(input(43 DOWNTO 40), output(54 DOWNTO 50));
          encode(input(39 DOWNTO 36), output(49 DOWNTO 45));
          encode(input(35 DOWNTO 32), output(44 DOWNTO 40));
          encode(input(31 DOWNTO 28), output(39 DOWNTO 35));
          encode(input(27 DOWNTO 24), output(34 DOWNTO 30));
          encode(input(23 DOWNTO 20), output(29 DOWNTO 25));
          encode(input(19 DOWNTO 16), output(24 DOWNTO 20));
          encode(input(15 DOWNTO 12), output(19 DOWNTO 15));

```

```

        encode(input(11 DOWNIO 8), output(14 DOWNIO 10));
        encode(input(7  DOWNIO 4), output(9  DOWNIO 5));
        encode(input(3  DOWNIO 0), output(4  DOWNIO 0));
    END IF;
    WHEN OTHERS =>
        output <= (OTHERS => 'Z');
    END CASE;
END PROCESS;

END default;

```

decoder.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE work.encoding.ALL;

ENTITY decoder IS
    PORT (input  : IN  std_logic_vector(54 DOWNIO 0);
          en, clk : IN  std_logic;
          output  : OUT std_logic_vector(43 DOWNIO 0) := (OTHERS => 'Z'));
END decoder;

ARCHITECTURE default OF decoder IS
BEGIN
    PROCESS (clk)
    BEGIN
        CASE en IS
            WHEN '1' =>
                IF (clk'event AND clk = '1') THEN
                    decode(input(54 DOWNIO 50), output(43 DOWNIO 40));
                    decode(input(49 DOWNIO 45), output(39 DOWNIO 36));
                    decode(input(44 DOWNIO 40), output(35 DOWNIO 32));
                    decode(input(39 DOWNIO 35), output(31 DOWNIO 28));
                    decode(input(34 DOWNIO 30), output(27 DOWNIO 24));
                    decode(input(29 DOWNIO 25), output(23 DOWNIO 20));
                    decode(input(24 DOWNIO 20), output(19 DOWNIO 16));
                    decode(input(19 DOWNIO 15), output(15 DOWNIO 12));
                    decode(input(14 DOWNIO 10), output(11 DOWNIO 8));
                    decode(input(9  DOWNIO 5), output(7  DOWNIO 4));
                    decode(input(4  DOWNIO 0), output(3  DOWNIO 0));
                END IF;
            WHEN OTHERS =>
                output <= (OTHERS => 'Z');
            END CASE;
        END PROCESS;
    END default;

```

demux.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY demux IS
    PORT (sel  : IN  std_logic_vector(1 DOWNIO 0);
          input : IN  std_logic_vector(43 DOWNIO 0);

          a, b, c : OUT std_logic_vector(43 DOWNIO 0));
END demux;

ARCHITECTURE default OF demux IS
BEGIN
    PROCESS(sel)
    BEGIN
        CASE sel IS
            WHEN "00" =>
                a <= input;

```

```

        b <= (OTHERS => 'Z');
        c <= (OTHERS => 'Z');
    WHEN "01" =>
        a <= (OTHERS => 'Z');
        b <= input;
        c <= (OTHERS => 'Z');
    WHEN "10" =>
        a <= (OTHERS => 'Z');
        b <= (OTHERS => 'Z');
        c <= input;
    WHEN OTHERS =>
        a <= (OTHERS => 'Z');
        b <= (OTHERS => 'Z');
        c <= (OTHERS => 'Z');

    END CASE;
    END PROCESS;

END default;

```

mux.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY mux IS
    PORT (sel      : IN  std_logic_vector(1 DOWNTO 0);
          a, b, c  : IN  std_logic_vector(44 DOWNTO 0);

          output   : OUT std_logic_vector(44 DOWNTO 0));
END mux;

ARCHITECTURE default OF mux IS
BEGIN

    WITH sel SELECT
        output <= a      WHEN "00",
        b WHEN "01",
        c WHEN "11", -- τροποποιήθηκε σε "11", για να είναι συμβατό με τον πολυπλέκτη
        (OTHERS => 'Z') WHEN OTHERS;

END default;

```

error_detection.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY error_detect IS
    PORT (input      : IN  std_logic_vector(54 DOWNTO 0);
          output     : OUT std_logic_vector(54 DOWNTO 0);
          en, rst, clk : IN  std_logic;
          found_error : OUT std_logic);
END error_detect;

ARCHITECTURE default OF error_detect IS
    COMPONENT comparator IS
        PORT (input      : IN  std_logic_vector(4 DOWNTO 0);
              output     : OUT std_logic_vector(4 DOWNTO 0);
              en, clk    : IN  std_logic;
              found_error : OUT std_logic);
    END COMPONENT;

    COMPONENT dff IS
        GENERIC (d_len : integer := 5;
                 q_len : integer := 5);
        PORT (d          : IN  std_logic_vector(d_len-1 DOWNTO 0);

```

```

        q          : OUT std_logic_vector(q_len-1 DOWNIO 0);
        clk, rst, en : IN  std_logic);
END COMPONENT;

-- wires for flip flops' output
SIGNAL dff0_out : std_logic_vector(4 DOWNIO 0);
SIGNAL dff1_out : std_logic_vector(4 DOWNIO 0);
SIGNAL dff2_out : std_logic_vector(4 DOWNIO 0);
SIGNAL dff3_out : std_logic_vector(4 DOWNIO 0);
SIGNAL dff4_out : std_logic_vector(4 DOWNIO 0);
SIGNAL dff5_out : std_logic_vector(4 DOWNIO 0);
SIGNAL dff6_out : std_logic_vector(4 DOWNIO 0);
SIGNAL dff7_out : std_logic_vector(4 DOWNIO 0);
SIGNAL dff8_out : std_logic_vector(4 DOWNIO 0);
SIGNAL dff9_out : std_logic_vector(4 DOWNIO 0);
SIGNAL dff10_out : std_logic_vector(4 DOWNIO 0);

-- wires for "foun_error" output of comparators
SIGNAL error0 : std_logic;
SIGNAL error1 : std_logic;
SIGNAL error2 : std_logic;
SIGNAL error3 : std_logic;
SIGNAL error4 : std_logic;
SIGNAL error5 : std_logic;
SIGNAL error6 : std_logic;
SIGNAL error7 : std_logic;
SIGNAL error8 : std_logic;
SIGNAL error9 : std_logic;
SIGNAL error10 : std_logic;

BEGIN

-- flip flops before comparators
dff0 : dff PORT MAP (d => input(54 DOWNIO 50), q => dff0_out, clk => clk, rst => rst, en => en);
dff1 : dff PORT MAP (d => input(49 DOWNIO 45), q => dff1_out, clk => clk, rst => rst, en => en);
dff2 : dff PORT MAP (d => input(44 DOWNIO 40), q => dff2_out, clk => clk, rst => rst, en => en);
dff3 : dff PORT MAP (d => input(39 DOWNIO 35), q => dff3_out, clk => clk, rst => rst, en => en);
dff4 : dff PORT MAP (d => input(34 DOWNIO 30), q => dff4_out, clk => clk, rst => rst, en => en);
dff5 : dff PORT MAP (d => input(29 DOWNIO 25), q => dff5_out, clk => clk, rst => rst, en => en);
dff6 : dff PORT MAP (d => input(24 DOWNIO 20), q => dff6_out, clk => clk, rst => rst, en => en);
dff7 : dff PORT MAP (d => input(19 DOWNIO 15), q => dff7_out, clk => clk, rst => rst, en => en);
dff8 : dff PORT MAP (d => input(14 DOWNIO 10), q => dff8_out, clk => clk, rst => rst, en => en);
dff9 : dff PORT MAP (d => input(9 DOWNIO 5), q => dff9_out, clk => clk, rst => rst, en => en);
dff10 : dff PORT MAP (d => input(4 DOWNIO 0), q => dff10_out, clk => clk, rst => rst, en => en);

-- comparators
comp0 : comparator PORT MAP (input => dff0_out, output => output(54 DOWNIO 50), en => en, clk => clk,
    found_error => error0);
comp1 : comparator PORT MAP (input => dff1_out, output => output(49 DOWNIO 45), en => en, clk => clk,
    found_error => error1);
comp2 : comparator PORT MAP (input => dff2_out, output => output(44 DOWNIO 40), en => en, clk => clk,
    found_error => error2);
comp3 : comparator PORT MAP (input => dff3_out, output => output(39 DOWNIO 35), en => en, clk => clk,
    found_error => error3);
comp4 : comparator PORT MAP (input => dff4_out, output => output(34 DOWNIO 30), en => en, clk => clk,
    found_error => error4);
comp5 : comparator PORT MAP (input => dff5_out, output => output(29 DOWNIO 25), en => en, clk => clk,
    found_error => error5);
comp6 : comparator PORT MAP (input => dff6_out, output => output(24 DOWNIO 20), en => en, clk => clk,
    found_error => error6);
comp7 : comparator PORT MAP (input => dff7_out, output => output(19 DOWNIO 15), en => en, clk => clk,
    found_error => error7);
comp8 : comparator PORT MAP (input => dff8_out, output => output(14 DOWNIO 10), en => en, clk => clk,
    found_error => error8);
comp9 : comparator PORT MAP (input => dff9_out, output => output(9 DOWNIO 5), en => en, clk => clk,
    found_error => error9);
comp10 : comparator PORT MAP (input => dff10_out, output => output(4 DOWNIO 0), en => en, clk => clk,
    found_error => error10);

-- OR gate for "found_error" signals
found_error <= error0 OR error1 OR error2 OR error3 OR error4 OR error5 OR error6 OR error7 OR error8 OR
    error9 OR error10;

END default;
```

fid_gen.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY fid_gen IS
  PORT (
    clr, clk : IN std_logic;
    input    : IN std_logic_vector(44 DOWNTO 0);
    output   : OUT std_logic_vector(43 DOWNTO 0)
  );
END fid_gen;

ARCHITECTURE default OF fid_gen IS
  SIGNAL en : std_logic; --εσωτερικό σήμα, γιατί το παίρνουμε απ' το validity bit
BEGIN
  en <= input(0);        -- το τελευταίο bit είναι το validity bit

  PROCESS (clk, clr)
    VARIABLE temp : integer RANGE 0 TO 32;
  BEGIN
    CASE en IS
      WHEN '1' =>
        CASE clr IS
          WHEN '0' =>
            IF (clk'event AND clk = '1') THEN
              temp := temp + 1;
              output(38 DOWNTO 0) <= input(39 DOWNTO 1); -- το τελευταίο bit
                                                           -- χρησιμοποιήθηκε πριν
              output(43 DOWNTO 39) <= std_logic_vector(to_unsigned(temp, 5));
            END IF;
            WHEN OTHERS =>
              temp := 0;
            END CASE;
          WHEN OTHERS =>
            --output <= (OTHERS => 'Z');
            NULL;
          END CASE;
        END PROCESS;
    END default;

```

fid_gen_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS
  SIGNAL clr : std_logic;
  SIGNAL clk : std_logic := '0';
  SIGNAL input : std_logic_vector(44 DOWNTO 0);
  SIGNAL output : std_logic_vector(43 DOWNTO 0);
BEGIN
  fid_gen0 : ENTITY work.fid_gen PORT MAP (clr => clr, clk => clk, input => input, output => output);

  clk <= NOT clk AFTER 10 ns;

  PROCESS
  BEGIN
    --clear counter
    clr <= '1';
    WAIT FOR 25 ns;

    clr <= '0';
    input <= "10101001010010110101101011010110101101011010100101";
    WAIT FOR 20 ns;

    input <= "10101001010010110101101011010110101101011010100100";

```

```

    WAIT FOR 20 ns;

    input <= "101010010100101101011010110101101011010100101";
    WAIT FOR 20 ns;

    input <= "101010010100101101011010110101101011010100101";
    WAIT FOR 20 ns;
    END PROCESS;

END tb;

```

scheduler.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY scheduler IS
    PORT (a, b, c : IN std_logic;
          s1, s0 : OUT std_logic);
END scheduler;

ARCHITECTURE default OF scheduler IS
    SIGNAL check_null : std_logic;
    SIGNAL input0, input1, input2 : std_logic;
BEGIN
    input0 <= '0' WHEN a = 'Z' ELSE a;
    input1 <= '0' WHEN b = 'Z' ELSE b;
    input2 <= '0' WHEN c = 'Z' ELSE c;

    check_null <= input0 OR input1 OR input2;
    PROCESS(check_null)
    BEGIN
        CASE check_null IS
            WHEN '0' =>
                s1 <= 'Z';
                s0 <= 'Z';
            WHEN OTHERS =>
                s1 <= input0;
                s0 <= input0 OR ((NOT input0) AND (NOT input1) AND input2);
        END CASE;
    END PROCESS;
END default;

```

scheduler_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS
    SIGNAL a, b, c, s1, s0 : std_logic;
    SIGNAL counter : std_logic_vector(2 DOWNTO 0) := "000"; -- counter for loop
BEGIN

    scheduler0: entity work.scheduler port map (a=>a, b=>b, c=>c, s1=>s1, s0=>s0);

    PROCESS
    BEGIN
        FOR i IN 0 TO 7 LOOP
            a <= counter(2);
            b <= counter(1);
            c <= counter(0);
            counter <= std_logic_vector(unsigned(counter) + 1);
            WAIT FOR 10 ns;
        END LOOP;
    END PROCESS;

```

```
END PROCESS;
```

```
END tb;
```

mil_main.vhd

```
LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY mil_main IS
  PORT (
    clk : IN std_logic;
    rst : IN std_logic := '0'; --θα αλλάξει, γιατί θα ελέγχεται απ' το clock
    --synchronization module

    -- output layer
    chi_out, wupg_out, cs_out : IN std_logic_vector(44 DOWNTO 0);
    data_out : OUT std_logic_vector(54 DOWNTO 0);

    -- input layer
    data_in : IN std_logic_vector(54 DOWNTO 0);
    -- το wup_in οδηγεί στον controller host interface
    -- το chi_in είναι για τα δεδομένα
    chi_in, wup_in, cs_in : OUT std_logic_vector(43 DOWNTO 0);

    -- testing signals
    test_err : OUT std_logic
  );
END mil_main;

ARCHITECTURE default OF mil_main IS
  -- components
  COMPONENT scheduler IS
    PORT (a, b, c : IN std_logic;
          s1, s0 : OUT std_logic);
  END COMPONENT;

  COMPONENT mux IS
    PORT (sel : IN std_logic_vector(1 DOWNTO 0);
          a, b, c : IN std_logic_vector(44 DOWNTO 0);

          output : OUT std_logic_vector(44 DOWNTO 0));
  END COMPONENT;

  COMPONENT dff IS
    GENERIC (d_len : integer := 1;
             q_len : integer := 1);
    PORT (d : IN std_logic_vector(d_len-1 DOWNTO 0);
          q : OUT std_logic_vector(q_len-1 DOWNTO 0);
          clk, rst, en : IN std_logic);
  END COMPONENT;

  COMPONENT encoder IS
    PORT (input : IN std_logic_vector(43 DOWNTO 0);
          en, clk : IN std_logic;
          output : OUT std_logic_vector(54 DOWNTO 0) := (OTHERS => 'Z'));
  END COMPONENT;

  COMPONENT error_detect IS
    PORT (input : IN std_logic_vector(54 DOWNTO 0);
          output : OUT std_logic_vector(54 DOWNTO 0);
          en, rst, clk : IN std_logic;
          found_error : OUT std_logic);
  END COMPONENT;

  COMPONENT decoder IS
    PORT (input : IN std_logic_vector(54 DOWNTO 0);
          en, clk : IN std_logic;
          output : OUT std_logic_vector(43 DOWNTO 0) := (OTHERS => 'Z'));
  END COMPONENT;

  COMPONENT demux IS
```

```

    PORT (sel      : IN  std_logic_vector(1 DOWNTO 0);
          input    : IN  std_logic_vector(43 DOWNTO 0);
          a, b, c  : OUT std_logic_vector(43 DOWNTO 0));
END COMPONENT;

COMPONENT fid_gen IS
  PORT (
    clr, clk : IN  std_logic;
    input    : IN  std_logic_vector(44 DOWNTO 0);
    output   : OUT std_logic_vector(43 DOWNTO 0)
  );
END COMPONENT;

-- WIRES

-- output layer
SIGNAL mux_s1, mux_s0      : std_logic;
SIGNAL mux_out             : std_logic_vector(44 DOWNTO 0);
SIGNAL dff0_out            : std_logic_vector(44 DOWNTO 0);
SIGNAL encoder_out         : std_logic_vector(54 DOWNTO 0);
SIGNAL dff1_out            : std_logic_vector(43 DOWNTO 0);
SIGNAL fid_gen_out         : std_logic_vector(43 DOWNTO 0);
--input layer
SIGNAL dff3_out            : std_logic_vector(54 DOWNTO 0);
SIGNAL error_status        : std_logic;
SIGNAL error_detect_output : std_logic_vector(54 DOWNTO 0);
SIGNAL dff4_out            : std_logic_vector(54 DOWNTO 0);
SIGNAL OR_gate             : std_logic;
SIGNAL decoder_output      : std_logic_vector(43 DOWNTO 0);
SIGNAL dff5_out            : std_logic_vector(43 DOWNTO 0);
SIGNAL typeofservice       : std_logic_vector(1 DOWNTO 0); --καθορίζει τη διαδρομή
                                                              --του εισερχόμενου πλαισίου στον αποπλέκτη

BEGIN

    -- OUTPUT LAYER

    -- scheduler and multiplexer
    sched0 : scheduler PORT MAP (a => cs_out(0), b => chi_out(0), c => wupg_out(0), s1 => mux_s1, s0 => mux_s0)
    ;
    mux0   : mux PORT MAP (sel(1) => mux_s1, sel(0) => mux_s0, a => chi_out, b => wupg_out, c => cs_out,
        output => mux_out);

    -- flip flop
    dff0 : dff GENERIC MAP (d_len => 45, q_len => 45) PORT MAP (d => mux_out(44 DOWNTO 0), q => dff0_out, clk
        => clk, rst => rst, en => '1');

    -- frame id generator
    fid_gen0 : fid_gen PORT MAP (clr => '0', clk => clk, input => dff0_out, output => fid_gen_out);

    -- flip flop
    dff1 : dff GENERIC MAP (d_len => 44, q_len => 44) PORT MAP (d => fid_gen_out, q => dff1_out, clk => clk,
        rst => rst, en => '1');

    -- encoder
    encoder0 : encoder PORT MAP (input => dff1_out, en => '1', clk => clk, output => encoder_out);

    -- flip flop
    dff2 : dff GENERIC MAP (d_len => 55, q_len => 55) PORT MAP (d => encoder_out, q => data_out, clk => clk,
        rst => rst, en => '1');

    -- INPUT LAYER

    -- κατεύθυνση: είσοδος => έξοδος δεξιά( προς τα αριστερά στο σχήμα)
    -- flip flop
    dff3 : dff GENERIC MAP (d_len => 55, q_len => 55) PORT MAP (d => data_in, q => dff3_out, clk => clk, rst =>
        rst, en => '1');

    -- error detection
    error_detect0 : error_detect PORT MAP (input => dff3_out, output => error_detect_output, en => '1', clk =>
        clk, rst => rst, found_error => error_status);

    -- flip flop
    dff4 : dff GENERIC MAP (d_len => 55, q_len => 55) PORT MAP (d => error_detect_output, q => dff4_out, clk =>
        clk, rst => OR_gate, en => '1');

    -- or gate

```


A .4 Wakeup Pattern Generator

counter.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY counter IS
  PORT (
    en, clr, clk : IN std_logic;
    output       : OUT std_logic_vector(1 DOWNTO 0) := (OTHERS => 'Z')
  );
END counter;

ARCHITECTURE default OF counter IS
BEGIN

  PROCESS (clk, clr)
    VARIABLE temp : integer RANGE 0 TO 3;
  BEGIN
    CASE en IS
      WHEN '1' =>
        CASE clr IS
          WHEN '0' =>
            IF (clk'event AND clk = '1') THEN
              temp := temp + 1;
            END IF;
          WHEN OTHERS =>
            temp := 0;
        END CASE;
      WHEN OTHERS =>
        temp := 0;
    END CASE;
    output <= std_logic_vector(to_unsigned(temp, 2));
  END PROCESS;

END default;

```

wupg_mux.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY wupg_mux IS
  PORT (sel : IN std_logic;
        a, b : IN std_logic;

        output : OUT std_logic);
END wupg_mux;

ARCHITECTURE default OF wupg_mux IS
BEGIN

  WITH sel SELECT
    output <= a WHEN '0',
             b WHEN '1',
             'Z' WHEN OTHERS;

END default;

```

rom.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

```

```

ENTITY rom IS
  PORT (
    addr      : IN  std_logic_vector(1 DOWNTO 0);
    output     : OUT std_logic_vector(4 DOWNTO 0);
    oe, clk    : IN  std_logic
  );
END rom;

ARCHITECTURE default OF rom IS
  TYPE memory4 IS ARRAY (0 TO 3) OF std_logic_vector(4 DOWNTO 0);
  -- τα 3 πρώτα bits αναπαριστούν το σύμβολο και το τελευταίο bit είναι το validity bit
  CONSTANT rom4 : memory4 := (
    "10101", "11111", "00101", "01111"
  );
BEGIN
  PROCESS(clk)
  BEGIN
    CASE oe IS
      WHEN '1' =>
        CASE addr IS
          WHEN (OTHERS => 'Z') =>
            output <= (OTHERS => 'Z');
          WHEN OTHERS =>
            IF (clk'event AND clk = '1') THEN
              output <= rom4(to_integer(unsigned(addr)));
            END IF;
          END CASE;
        WHEN OTHERS =>
          output <= (OTHERS => 'Z');
        END CASE;
      END PROCESS;
    END default;
  
```

srff.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- πάντα enabled
ENTITY srff IS
  PORT (
    set, rst, clk : IN  std_logic;
    q              : OUT std_logic
  );
END srff;

ARCHITECTURE default OF srff IS

BEGIN

  PROCESS (rst, clk, set)
  BEGIN
    CASE rst IS
      WHEN '1' =>
        IF (set = '0') THEN
          q <= '0';
        ELSIF (set = '1') THEN
          q <= 'Z';
        END IF;
      WHEN '0' =>
        IF (clk'event AND clk = '1' AND set = '1') THEN
          q <= '1';
        END IF;
      WHEN OTHERS =>
        q <= 'Z';
      END CASE;
    END PROCESS;
  END default;
  
```

mainv2.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY wupg_main IS
  PORT (
    wupg_en   : IN   std_logic;
    wupg_out  : OUT  std_logic_vector(44 DOWNTO 0);
    clk       : IN   std_logic;

    sel       : IN   std_logic;
    reset     : IN   std_logic;
    test_output : OUT std_logic
  );
END wupg_main;

ARCHITECTURE default OF wupg_main IS

-- components
COMPONENT srff IS
  PORT (
    set, rst, clk : IN   std_logic;
    q             : OUT  std_logic
  );
END COMPONENT;

COMPONENT dff IS
  GENERIC (d_len : integer := 1;
           q_len : integer := 1);
  PORT (d       : IN   std_logic_vector(d_len-1 DOWNTO 0);
        q       : OUT  std_logic_vector(q_len-1 DOWNTO 0);
        clk, rst, en : IN   std_logic);
END COMPONENT;

COMPONENT counter IS
  PORT (
    en, clr, clk : IN   std_logic;
    output       : OUT  std_logic_vector(1 DOWNTO 0)
  );
END COMPONENT;

COMPONENT rom IS
  PORT (
    addr   : IN   std_logic_vector(1 DOWNTO 0);
    output : OUT  std_logic_vector(4 DOWNTO 0);
    oe, clk : IN   std_logic
  );
END COMPONENT;

COMPONENT wupg_mux IS
  PORT (sel : IN   std_logic;
        a, b : IN   std_logic;

        output : OUT std_logic);
END COMPONENT;

-- wires
SIGNAL ctrl_wire : std_logic := '0';
SIGNAL cnt_out   : std_logic_vector(1 DOWNTO 0);
SIGNAL dff0_out  : std_logic_vector(1 DOWNTO 0);
SIGNAL rom_out   : std_logic_vector(4 DOWNTO 0);
SIGNAL dff3_out  : std_logic;
SIGNAL rst_wire  : std_logic;
SIGNAL dff2_out  : std_logic_vector(1 DOWNTO 0);
SIGNAL AND_output : std_logic;
SIGNAL dff4_out  : std_logic_vector(1 DOWNTO 0);
SIGNAL dff5_out  : std_logic_vector(1 DOWNTO 0);
SIGNAL dff1_en   : std_logic;

BEGIN

  srff0 : srff PORT MAP (set => wupg_en, rst => rst_wire, clk => clk, q => ctrl_wire);

```

```

counter0 : counter PORT MAP (en => ctrl_wire, clr => '0', clk => clk, output => cnt_out);

dff0 : dff GENERIC MAP (d_len => 2, q_len => 2) PORT MAP (d => cnt_out, q => dff0_out, clk => clk, rst =>
'0', en => ctrl_wire);

rom0 : rom PORT MAP (addr => dff0_out, output => rom_out, oe => ctrl_wire, clk => clk);

-- DFF1
-- εδώ αποστέλεται το σύμβολο μέσα σε ένα πλαίσιο/frame
-- στο frame id βάζουμε 0, γιατί θα προστεθεί στο medium interface layer
dff1 : dff GENERIC MAP (d_len => 45, q_len => 45) PORT MAP (d(44 DOWNIO 40) => (OTHERS => '0'), d(39 DOWNIO
35) => "00100", d(34 DOWNIO 33) => "01", d(32 DOWNIO 5) => (OTHERS => '0'), d(4 DOWNIO 0) => rom_out, q
=> wupg_out, clk => clk, rst => '0', en => dff1_en);

-- enable signal για το DFF1
-- rom_out(0) = validity bit
dff1_en <= dff3_out AND rom_out(0);

-----

dff3 : dff GENERIC MAP (d_len => 1, q_len => 1) PORT MAP (d(0) => ctrl_wire, q(0) => dff3_out, clk => clk,
rst => '0', en => '1');

dff2 : dff GENERIC MAP (d_len => 2, q_len => 2) PORT MAP (d => cnt_out, q => dff2_out, clk => clk, rst =>
reset, en => '1');

dff4 : dff GENERIC MAP (d_len => 2, q_len => 2) PORT MAP (d => dff2_out, q => dff4_out, clk => clk, rst =>
reset, en => '1');

dff5 : dff GENERIC MAP (d_len => 2, q_len => 2) PORT MAP (d => dff4_out, q => dff5_out, clk => clk, rst =>
reset, en => '1');

mux0 : wupg_mux PORT MAP (sel => sel, a => '1', b => AND_output, output => rst_wire);

-- πύλη AND
AND_output <= dff5_out(1) AND dff5_out(0);

test_output <= ctrl_wire;

END default;

```

wupg_main_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS
    SIGNAL wupg_en : std_logic;
    SIGNAL wupg_out : std_logic_vector(44 DOWNIO 0);
    SIGNAL clk : std_logic := '0';

    SIGNAL sel : std_logic;
    SIGNAL reset : std_logic;
    SIGNAL test_output : std_logic;

BEGIN

    wupg_main0 : ENTITY work.wupg_main PORT MAP (wupg_en => wupg_en, wupg_out => wupg_out, clk => clk,
        test_output => test_output, reset => reset, sel => sel);

    clk <= NOT clk AFTER 10 ns;

    PROCESS
    BEGIN
        -- reset SR FF
        sel <= '0'; -- έξοδος πολυπλέκτη: '1'
        wupg_en <= '0';
        reset <= '1'; -- reset 2, 4, 5 DFFs
        WAIT FOR 5 ns;
    END PROCESS

```

```

    reset <= '0'; --είναι ασύγχρονο το reset
    sel <= '1'; -- έξοδος πολυπλέκτη: έξοδος πύλης AND
    WAIT FOR 10 ns;
    -- set SR FF
    wupg_en <= '1';
    WAIT FOR 20 ns;
    wupg_en <= '0';
    WAIT FOR 200 ns;

    END PROCESS;

END tb;

```

A .5 Wakeup Pattern Recognition

wupr_fsm.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY wupr_fsm IS
    PORT (symbol : IN std_logic_vector(3 DOWNTO 0);
          clk, rst : IN std_logic;

          wup_found : OUT std_logic;

          --testing signal
          curr_state : OUT std_logic_vector(2 DOWNTO 0));
END wupr_fsm;

-- η αρχιτεκτονική βασίζεται στο στυλ σχεδιασμού #2 απτο' βιβλίο του Pedroni
ARCHITECTURE default OF wupr_fsm IS
    TYPE states IS (m0, m1, m2, m3, m4);
    SIGNAL prv_state, nxt_state : states;
    SIGNAL temp_wup_found : std_logic;
BEGIN

    PROCESS (clk, rst)
    BEGIN
        IF (rst = '1') THEN
            prv_state <= m0;
            curr_state <= std_logic_vector(to_unsigned(states'pos(m0), 3));
            wup_found <= '0';
        ELSIF (clk'event AND clk = '1') THEN
            wup_found <= temp_wup_found;
            prv_state <= nxt_state;
            curr_state <= std_logic_vector(to_unsigned(states'pos(nxt_state), 3));
        END IF;
    END PROCESS;

    PROCESS (prv_state, symbol)
    BEGIN
        CASE prv_state IS

            WHEN m0 =>
                CASE symbol IS
                    WHEN "1010" =>
                        temp_wup_found <= '0';
                        nxt_state <= m1;
                    WHEN OTHERS =>
                        temp_wup_found <= '0';
                END CASE;

            WHEN m1 =>
                CASE symbol IS
                    WHEN "1111" =>
                        nxt_state <= m2;
                    WHEN OTHERS =>
                        nxt_state <= m0;
                END CASE;
        END CASE;
    END PROCESS;

```

```

        END CASE;

    WHEN m2 =>
        CASE symbol IS
            WHEN "0010" =>
                nxt_state <= m3;
            WHEN OTHERS =>
                nxt_state <= m0;
            END CASE;

    WHEN m3 =>
        CASE symbol IS
            WHEN "0111" =>
                nxt_state <= m4;
                temp_wup_found <= '1';
            WHEN OTHERS =>
                nxt_state <= m0;
            END CASE;

    WHEN m4 =>
        nxt_state <= m0;
        temp_wup_found <= '0';

    WHEN OTHERS =>
        NULL;

    END CASE;
END PROCESS;

END default;

```

wupr_fsm_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS
    SIGNAL symbol      : std_logic_vector(3 DOWNTO 0);
    SIGNAL clk         : std_logic := '0';
    SIGNAL rst         : std_logic;
    SIGNAL wup_found   : std_logic;
    SIGNAL curr_state  : std_logic_vector(2 DOWNTO 0);
BEGIN

    wupr_fsm0 : ENTITY work.wupr_fsm PORT MAP (symbol => symbol, clk => clk, rst => rst, wup_found => wup_found,
        , curr_state => curr_state);

    clk <= NOT clk AFTER 10 ns;

    PROCESS
    BEGIN

        rst <= '1';
        symbol <= "0000";
        WAIT FOR 25 ns;

        rst <= '0';
        symbol <= "1010";
        WAIT FOR 20 ns;

        symbol <= "0101";
        WAIT FOR 20 ns;

        symbol <= "1010";
        WAIT FOR 20 ns;

        symbol <= "1111";
    END PROCESS

```

```

    WAIT FOR 20 ns;

    symbol <= "0010";
    WAIT FOR 20 ns;

    symbol <= "0111";
    WAIT FOR 200 ns;

    END PROCESS;

END tb;

```

wupr_main.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY wupr_main IS
    PORT (symbol      : IN  std_logic_vector(3 DOWNTO 0);
          clk, fsm_rst : IN  std_logic;
          wup_found    : OUT std_logic);
END wupr_main;

ARCHITECTURE default OF wupr_main IS
    --components
    COMPONENT wupr_fsm IS
        PORT (symbol : IN  std_logic_vector(3 DOWNTO 0);
              clk, rst : IN  std_logic;

              wup_found : OUT std_logic;

              --testing signal
              curr_state : OUT std_logic_vector(2 DOWNTO 0));
    END COMPONENT;

    COMPONENT dff IS
        GENERIC (d_len : integer := 1;
                  q_len : integer := 1);
        PORT (d      : IN  std_logic_vector(d_len-1 DOWNTO 0);
              q      : OUT std_logic_vector(q_len-1 DOWNTO 0);
              clk, rst, en : IN  std_logic);
    END COMPONENT;

    --wires
    SIGNAL wire0 : std_logic_vector(3 DOWNTO 0);
    --testing signal
    SIGNAL curr_state : std_logic_vector(2 DOWNTO 0);
BEGIN

    reg0 : dff GENERIC MAP (d_len => 4, q_len => 4) PORT MAP (d => symbol, q => wire0, clk => clk, rst => '0',
        en => '1');

    wupr_fsm0 : wupr_fsm PORT MAP (symbol => wire0, clk => clk, rst => fsm_rst, wup_found => wup_found,
        curr_state => curr_state);

END default;

```

wupr_main_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS
    SIGNAL symbol : std_logic_vector(3 DOWNTO 0);
    SIGNAL clk : std_logic := '0';

```

```

SIGNAL fsm_rst    : std_logic;
SIGNAL wup_found  : std_logic;
BEGIN

wupr_main0 : ENTITY work.wupr_main PORT MAP (symbol => symbol, clk => clk, fsm_rst => fsm_rst, wup_found =>
wup_found);

clk <= NOT clk AFTER 10 ns;

PROCESS
BEGIN

    fsm_rst    <= '1';
    symbol <= "0000";
    WAIT FOR 25 ns;

    fsm_rst    <= '0';
    symbol <= "1010";
    WAIT FOR 20 ns;

    symbol <= "0101";
    WAIT FOR 20 ns;

    symbol <= "1010";
    WAIT FOR 20 ns;

    symbol <= "1111";
    WAIT FOR 20 ns;

    symbol <= "0010";
    WAIT FOR 20 ns;

    symbol <= "0111";
    WAIT FOR 200 ns;

END PROCESS;

END tb;

```

A .6 Clock Synchronization

buffer.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY tri_state_buffer IS
    PORT (en      : IN  std_logic;
          input   : IN  std_logic;
          output  : OUT std_logic
    );
END tri_state_buffer;

ARCHITECTURE default OF tri_state_buffer IS
BEGIN

    output <= input WHEN en = '1' ELSE
        'Z';

END default;

```

micro_comparator.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY micro_comparator IS

```

```

PORT (local_value : IN std_logic_vector(3 DOWNTO 0); --microticks που μετρήθηκαν
      global_value : IN std_logic_vector(3 DOWNTO 0); -- απαιτούμενα microticks από διατητή
      en, clk : IN std_logic;
      sync_status : OUT std_logic);
END micro_comparator;

ARCHITECTURE default OF micro_comparator IS
BEGIN
  PROCESS (clk)
  BEGIN
    CASE en IS
      WHEN '1' =>
        IF (clk'event AND clk = '1') THEN
          IF (global_value = local_value) THEN
            sync_status <= '1';
          ELSE
            sync_status <= '0';
          END IF;
        END IF;
      WHEN OTHERS =>
        sync_status <= 'Z';
      END CASE;
    END PROCESS;
  END default;

```

micro_counter.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY micro_counter IS
  PORT (
    en, clr, clk : IN std_logic;
    output : OUT std_logic_vector(3 DOWNTO 0) := (OTHERS => 'Z')
  );
END micro_counter;

ARCHITECTURE default OF micro_counter IS
BEGIN

  PROCESS (clk, clr)
    VARIABLE temp : integer RANGE 0 TO 3;
  BEGIN
    CASE en IS
      WHEN '1' =>
        CASE clr IS
          WHEN '0' =>
            IF (clk'event AND clk = '1') THEN
              temp := temp + 1;
            END IF;
          WHEN OTHERS =>
            temp := 0;
          END CASE;
        WHEN OTHERS =>
          temp := 0;
        END CASE;
        output <= std_logic_vector(to_unsigned(temp, 4));
      END PROCESS;
    END default;

```

srff.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

-- πάντα enabled
ENTITY srff IS
  PORT (

```

```

    set, rst, clk : IN  std_logic;
    q              : OUT std_logic
  );
END srff;

ARCHITECTURE default OF srff IS

BEGIN

  PROCESS (rst, clk, set)
  BEGIN
    CASE rst IS
      WHEN '1' =>
        IF (set = '0') THEN
          q <= '0';
        ELSIF (set = '1') THEN
          q <= 'Z';
        END IF;
      WHEN '0' =>
        IF (clk'event AND clk = '1' AND set = '1') THEN
          q <= '1';
        END IF;
      WHEN OTHERS =>
        q <= 'Z';
    END CASE;
  END PROCESS;

END default;

```

vbit_fsm.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;
USE ieee.numeric_std.ALL;

ENTITY vbit_fsm IS
  PORT (
    buffer_inv  : IN  std_logic;
    validity_bit : OUT std_logic;

    clk, rst : IN std_logic;

    --testing signal
    curr_state : OUT std_logic_vector(1 DOWNTO 0)
  );
END vbit_fsm;

-- η αρχιτεκτονική βασίζεται στο στυλ σχεδιασμού #1 απτο' βιβλίο του Pedroni
-- ασύγχρονη( έξοδος)

ARCHITECTURE default OF vbit_fsm IS
  TYPE states IS (m0, m1, m2, m3, m4);
  SIGNAL prv_state, nxt_state : states;
BEGIN

  PROCESS (clk, rst)
  BEGIN
    IF (rst = '1') THEN
      prv_state <= m0;
      curr_state <= std_logic_vector(to_unsigned(states'pos(m0), 2));
    ELSIF (clk'event AND clk = '1') THEN
      prv_state <= nxt_state;
      curr_state <= std_logic_vector(to_unsigned(states'pos(nxt_state), 2));
    END IF;
  END PROCESS;

  PROCESS (prv_state, buffer_inv)
  BEGIN
    CASE prv_state IS
      WHEN m0 =>
        validity_bit <= '0';
        CASE buffer_inv IS
          WHEN '0' =>

```

```

        nxt_state <= m1;
        WHEN OTHERS =>
            NULL;
        END CASE;
    WHEN m1 =>
        CASE buffer_inv IS
            WHEN '1' =>
                nxt_state <= m2;
                WHEN OTHERS =>
                    NULL;
            END CASE;
        WHEN m2 =>
            -- wait state
            nxt_state <= m3;
        WHEN m3 =>
            validity_bit <= '1';
            nxt_state <= m4;
        WHEN m4 =>
            validity_bit <= '0';
            CASE buffer_inv IS
                WHEN '0' =>
                    nxt_state <= m1;
                WHEN OTHERS =>
                    NULL;
            END CASE;
        WHEN OTHERS =>
            NULL;
        END CASE;
    END PROCESS;
END default;

```

vbit_fsm_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS
    SIGNAL buffer_inv : std_logic := '0';
    SIGNAL validity_bit : std_logic;
    SIGNAL clk : std_logic := '0';
    SIGNAL rst : std_logic;
    SIGNAL curr_state : std_logic_vector(1 DOWNTO 0);
BEGIN

    vbit_fsm0 : ENTITY work.vbit_fsm PORT MAP (buffer_inv => buffer_inv, validity_bit => validity_bit, clk =>
        clk, rst => rst, curr_state => curr_state);

    clk <= NOT clk AFTER 10 ns;

    PROCESS
    BEGIN
        rst <= '1';
        WAIT FOR 25 ns;

        rst <= '0';
        buffer_inv <= '1';
        WAIT FOR 40 ns;

        buffer_inv <= '0';
        WAIT FOR 25 ns;
    END PROCESS;
END tb;

```

microtick_counter.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY microtick_counter IS
  PORT (
    state           : IN   std_logic_vector(2 DOWNTO 0);
    clk, macro_clock : IN   std_logic;
    global_value     : IN   std_logic_vector(3 DOWNTO 0); --απαιτούμενα macroticks
                                                              --από διατητή
    reset_ff         : IN   std_logic;
    test_output      : OUT  std_logic_vector(3 DOWNTO 0);

    -- output frame
    output : OUT std_logic_vector(44 DOWNTO 0);

    --reset signal for the vbit fsm
    rst : IN std_logic;

    -- test signal for the vbit fsm
    fsm_state : OUT std_logic_vector(1 DOWNTO 0)
  );
END microtick_counter;

ARCHITECTURE default OF microtick_counter IS
  --components
  COMPONENT srff IS
    PORT (
      set, rst, clk : IN   std_logic;
      q             : OUT  std_logic
    );
  END COMPONENT;

  COMPONENT dff IS
    GENERIC (d_len : integer := 1;
             q_len : integer := 1);
    PORT (d           : IN   std_logic_vector(d_len-1 DOWNTO 0);
          q           : OUT  std_logic_vector(q_len-1 DOWNTO 0);
          clk, rst, en : IN   std_logic);
  END COMPONENT;

  COMPONENT micro_comparator IS
    PORT (local_value : IN   std_logic_vector(3 DOWNTO 0); --microticks που μετρήθηκαν
          global_value : IN   std_logic_vector(3 DOWNTO 0); -- απαιτούμενα microticks από διατητή
          en, clk      : IN   std_logic;
          sync_status  : OUT  std_logic);
  END COMPONENT;

  COMPONENT micro_counter IS
    PORT (
      en, clr, clk : IN   std_logic;
      output       : OUT  std_logic_vector(3 DOWNTO 0) := (OTHERS => 'Z')
    );
  END COMPONENT;

  COMPONENT tri_state_buffer IS
    PORT (en      : IN   std_logic;
          input   : IN   std_logic;
          output  : OUT  std_logic
    );
  END COMPONENT;

  COMPONENT vbit_fsm IS
    PORT (
      buffer_inv : IN   std_logic;
      validity_bit : OUT std_logic;

      clk, rst : IN std_logic;

      --testing signal
      curr_state : OUT std_logic_vector(1 DOWNTO 0)
    );
  END COMPONENT;

```

```

-- wires
SIGNAL and0_out  : std_logic;
SIGNAL and1_out  : std_logic;
SIGNAL buffer_en : std_logic;
SIGNAL buffer_out : std_logic;
SIGNAL buffer_inv : std_logic;
SIGNAL cnt_out   : std_logic_vector(3 DOWNTO 0);
SIGNAL dff0_out  : std_logic_vector(3 DOWNTO 0);

SIGNAL local_value : std_logic_vector(3 DOWNTO 0); --microticks που
                                                    --μετρήθηκαν, λαμβάνεται
                                                    --απ' το DFF1
SIGNAL sync_status : std_logic; -- λαμβάνεται απ' το comparator
SIGNAL validity_bit : std_logic; -- λαμβάνεται απ' το vbit_fsm

BEGIN

-- set signal for SR FF
and0_out <= state(2) AND (NOT state(1)) AND (NOT (state(0)));

-- reset signal for SR FF
and1_out <= state(2) AND state(1) AND state(0);

-- sr FF
srff0 : srff PORT MAP (set => and0_out, rst => and1_out, clk => clk, q => buffer_en);

-- buffer
buffer0 : tri_state_buffer PORT MAP (en => buffer_en, input => macro_clock, output => buffer_out);

-- NOT gate
buffer_inv <= NOT buffer_out;

-- counter
counter0 : micro_counter PORT MAP (en => buffer_out, clr => '0', clk => clk, output => cnt_out);

-- D FF 0
dff0 : dff GENERIC MAP (d_len => 4, q_len => 4) PORT MAP (d => cnt_out, q => dff0_out, clk => clk, rst =>
    reset_ff, en => buffer_inv);

test_output <= dff0_out;

-- comparator
comparator0 : micro_comparator PORT MAP (local_value => dff0_out, global_value => global_value, en =>
    buffer_inv, clk => clk, sync_status => sync_status);

-- D FF 1
dff1 : dff GENERIC MAP (d_len => 4, q_len => 4) PORT MAP (d => dff0_out, q => local_value, clk => clk, rst
    => reset_ff, en => buffer_inv);

-- D FF 2
dff2 : dff GENERIC MAP (d_len => 45, q_len => 45) PORT MAP (d(44 DOWNTO 40) => (OTHERS => '0'), d(39 DOWNTO
    35) => "00110", d(34 DOWNTO 33) => "10", d(32 DOWNTO 6) => (OTHERS => '0'), d(5 DOWNTO 2) =>
    local_value, d(1) => sync_status, d(0) => validity_bit, q => output, clk => clk, rst => reset_ff, en =>
    '1');

-- vbit_fsm, χρησιμοποιείται για να θέσει το validity bit σε 1
vbit_fsm0 : vbit_fsm PORT MAP (buffer_inv => buffer_inv, validity_bit => validity_bit, clk => clk, rst =>
    rst, curr_state => fsm_state);

END default;

```

microtick_counter_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS

```

```

SIGNAL state      : std_logic_vector(2 DOWNTO 0);
SIGNAL clk        : std_logic := '0';
SIGNAL macro_clock : std_logic := '0';
SIGNAL global_value : std_logic_vector(3 DOWNTO 0);
SIGNAL reset_ff    : std_logic;
SIGNAL test_output : std_logic_vector(3 DOWNTO 0);
SIGNAL output      : std_logic_vector(44 DOWNTO 0);
-- fsm's signals
SIGNAL rst         : std_logic;
SIGNAL fsm_state   : std_logic_vector(1 DOWNTO 0);
BEGIN

microtick_counter0 : ENTITY work.microtick_counter PORT MAP (state => state, clk => clk, macro_clock =>
    macro_clock, global_value => global_value, reset_ff => reset_ff, test_output => test_output, output =>
    output, rst => rst, fsm_state => fsm_state);

clk          <= NOT clk          AFTER 10 ns;
macro_clock  <= NOT macro_clock AFTER 82 ns;
global_value <= "0100";

PROCESS
BEGIN
    state <= "100";           --set sr ff
    rst   <= '1';             -- reset vbit fsm
    WAIT FOR 15 ns;
    rst   <= '0';
    state <= "000";
    reset_ff <= '1';          -- reset dffs
    WAIT FOR 20 ns;
    reset_ff <= '0';
    -- στα 82 ns ερχεται η πρώτη θετική ακμή του macro_clock
    WAIT FOR 200 ns;
END PROCESS;

END tb;

```

A .7 Communication Controller (Ενοποίηση των components)

main.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY controller IS
    PORT (
        --ΤΥΠΕΣ
        -- θύρες στη μεριά του Host/Microcontroller
        command_in : IN std_logic_vector(1 DOWNTO 0);

        clk        : IN std_logic;
        mil_clk     : IN std_logic;

        status_data : OUT std_logic_vector(49 DOWNTO 0);

        frame_id    : IN std_logic_vector(4 DOWNTO 0);
        data_length : IN std_logic_vector(4 DOWNTO 0);
        type_of_data : IN std_logic_vector(1 DOWNTO 0);
        data        : IN std_logic_vector(31 DOWNTO 0);
        validity_bit : IN std_logic;

        -- θύρες στη μεριά του FlexRay Bus Driver
        data_out : OUT std_logic_vector(54 DOWNTO 0);
        data_in  : IN std_logic_vector(54 DOWNTO 0);

        -----

        -- βοηθητικά εσωτερικά σήματα για τη λειτουργία του testbench

```

```

-- pmm
ext_fsm_rst : IN std_logic;
ext_reg_rst : IN std_logic;

-- cs
ext_macro_clock : IN std_logic;
ext_global_value : IN std_logic_vector(3 DOWNTO 0);
ext_reset_ff : IN std_logic;
ext_test_output : OUT std_logic_vector(3 DOWNTO 0);
ext_rst : IN std_logic;
ext_fsm_state : OUT std_logic_vector(1 DOWNTO 0);

-- mil
ext_test_err : OUT std_logic;
ext_mil_rst : IN std_logic;

-- wupr
ext_wupr_fsm_rst : IN std_logic;
ext_wup_found : OUT std_logic;

-- wupg
ext_sel : IN std_logic;
ext_reset : IN std_logic;
ext_wupg_test_output : OUT std_logic;
ext_wupg_output : OUT std_logic_vector(44 DOWNTO 0)

);
END controller;

ARCHITECTURE default OF controller IS
  COMPONENT chi IS
    PORT (
      -- input, direction: micro => communication
      command_in : IN std_logic_vector(1 DOWNTO 0);
      --corresponding output
      command_out : OUT std_logic_vector(1 DOWNTO 0);
      -- input, origin: micro
      clk : IN std_logic;
      -- inputs, direction: communication => micro
      config_ack : IN std_logic;
      reset_ack : IN std_logic;
      state : IN std_logic_vector(2 DOWNTO 0);
      sync_status : IN std_logic;
      frame_rcv : IN std_logic_vector(43 DOWNTO 0);
      -- corresponding output
      status_data : OUT std_logic_vector(49 DOWNTO 0);
      -- inputs, direction: micro => communication
      frame_id : IN std_logic_vector(4 DOWNTO 0);
      data_length : IN std_logic_vector(4 DOWNTO 0);
      type_of_data : IN std_logic_vector(1 DOWNTO 0);
      data : IN std_logic_vector(31 DOWNTO 0);
      validity_bit : IN std_logic;
      -- corresponding output
      frame_send : OUT std_logic_vector(44 DOWNTO 0));
  END COMPONENT;

  COMPONENT pmm IS
    PORT (command : IN std_logic_vector(1 DOWNTO 0);
          sync_status : IN std_logic;
          clk, fsm_rst, reg_rst : IN std_logic;

          curr_state : OUT std_logic_vector(2 DOWNTO 0);
          reset_ack : OUT std_logic;
          wup_en : OUT std_logic;
          config_ack : OUT std_logic);
  END COMPONENT;

  -- Clock Synchronization Module
  COMPONENT microtick_counter IS
    PORT (
      state : IN std_logic_vector(2 DOWNTO 0);
      clk, macro_clock : IN std_logic;
      global_value : IN std_logic_vector(3 DOWNTO 0); --απαιτούμενα macroticks

```

```

--από διαιτητή
reset_ff      : IN  std_logic;
test_output   : OUT std_logic_vector(3 DOWNTO 0);

-- output frame
output : OUT std_logic_vector(44 DOWNTO 0);

--reset signal for the vbit fsm
rst : IN std_logic;

-- test signal for the vbit fsm
fsm_state : OUT std_logic_vector(1 DOWNTO 0)

);
END COMPONENT;

COMPONENT mil_main IS
PORT (
  clk : IN std_logic;
  rst : IN std_logic := '0'; --θα αλλάξει, γιατί θα ελέγχεται απ' το clock
  --synchronization module

  -- output layer
  chi_out, wupg_out, cs_out : IN  std_logic_vector(44 DOWNTO 0);
  data_out                  : OUT std_logic_vector(54 DOWNTO 0);

  -- input layer
  data_in                  : IN  std_logic_vector(54 DOWNTO 0);
  -- το wup_in οδηγεί στον controller host interface
  -- το chi_in είναι για τα δεδομένα
  chi_in, wup_in, cs_in : OUT std_logic_vector(43 DOWNTO 0);

  -- testing
  test_err : OUT std_logic
);
END COMPONENT;

COMPONENT wupr_main IS
PORT (symbol      : IN  std_logic_vector(3 DOWNTO 0);
      clk, fsm_rst : IN  std_logic;
      wup_found    : OUT std_logic);
END COMPONENT;

COMPONENT wupg_main IS
PORT (
  wupg_en : IN  std_logic;
  wupg_out : OUT std_logic_vector(44 DOWNTO 0);
  clk      : IN  std_logic;

  sel      : IN  std_logic;
  reset    : IN  std_logic;
  test_output : OUT std_logic
);
END COMPONENT;

-- signals για τη διασύνδεση των components
----- CHI -----
SIGNAL command_out : std_logic_vector(1 DOWNTO 0);
SIGNAL config_ack  : std_logic;
SIGNAL reset_ack   : std_logic;
SIGNAL state       : std_logic_vector(2 DOWNTO 0);
SIGNAL sync_status : std_logic;
SIGNAL frame_rev   : std_logic_vector(43 DOWNTO 0);
SIGNAL frame_send  : std_logic_vector(44 DOWNTO 0);

----- FMM -----
SIGNAL fsm_rst : std_logic;      -- external
SIGNAL reg_rst : std_logic;      -- external
SIGNAL wup_en  : std_logic;

```

```

----- CS -----
SIGNAL macro_clock : std_logic;           -- external
SIGNAL global_value : std_logic_vector(3 DOWNTO 0); --external
SIGNAL reset_ff : std_logic;              -- external
SIGNAL test_output : std_logic_vector(3 DOWNTO 0); --external
SIGNAL output : std_logic_vector(44 DOWNTO 0);
SIGNAL rst : std_logic;                   -- external
SIGNAL fsm_state : std_logic_vector(1 DOWNTO 0); -- external

----- MIL -----
SIGNAL mil_rst : std_logic;
SIGNAL wupg_out : std_logic_vector(44 DOWNTO 0);
SIGNAL wup_in : std_logic_vector(43 DOWNTO 0);
SIGNAL cs_in : std_logic_vector(43 DOWNTO 0); -- unused
SIGNAL test_err : std_logic;                -- external

----- WUPR -----
SIGNAL wupr_fsm_rst : std_logic;           -- external
SIGNAL wup_found : std_logic;             -- external, προοριζόμενο για το CHI

----- WUPG -----
SIGNAL sel : std_logic;                   -- external
SIGNAL reset : std_logic;                 -- external
SIGNAL wupg_test_output : std_logic;      -- external

SIGNAL check_validity : std_logic;

BEGIN

-- έλεγχος εκυρότητας του sync_status
PROCESS (output)
BEGIN
CASE output(0) IS -- το validity bit
WHEN '1' =>
sync_status <= output(1); -- τότε ίσο με την έξοδο του clock synchronizion
WHEN OTHERS =>
sync_status <= 'Z';
END CASE;
END PROCESS;

chi0 : chi PORT MAP (command_in => command_in, command_out => command_out, clk => clk, config_ack =>
config_ack, reset_ack => reset_ack, sync_status => sync_status, state => state, frame_rcv => frame_rcv,
status_data => status_data, frame_id => frame_id, data_length => data_length, type_of_data =>
type_of_data, data => data, validity_bit => validity_bit, frame_send => frame_send);

pnm0 : pnm PORT MAP (command => command_out, sync_status => sync_status, clk => clk, fsm_rst => fsm_rst,
reg_rst => reg_rst, curr_state => state, reset_ack => reset_ack, wup_en => wup_en, config_ack =>
config_ack);

cs0 : microtick_counter PORT MAP (state => state, clk => clk, macro_clock => macro_clock, global_value =>
global_value, reset_ff => reset_ff, test_output => test_output, output => output, rst => rst, fsm_state
=> fsm_state);

mil0 : mil_main PORT MAP (clk => mil_clk, rst => mil_rst, chi_out => frame_send, wupg_out => wupg_out,
cs_out => output, data_out => data_out, data_in => data_in, chi_in => frame_rcv, wup_in => wup_in, cs_in
=> cs_in, test_err => test_err);

wupr0 : wupr_main PORT MAP (symbol => wup_in(4 DOWNTO 1), clk => clk, fsm_rst => wupr_fsm_rst, wup_found =>
wup_found);

wupg0 : wupg_main PORT MAP (wupg_en => wup_en, wupg_out => wupg_out, clk => clk, sel => sel, reset => reset
, test_output => wupg_test_output);

-- οδήγηση βοηθητικών σημάτων για το testbench
-- pnm
fsm_rst <= ext_fsm_rst;
reg_rst <= ext_reg_rst;

-- cs

```

```

macro_clock      <= ext_macro_clock;
global_value     <= ext_global_value;
reset_ff         <= ext_reset_ff;
ext_test_output  <= test_output;
rst              <= ext_rst;
ext_fsm_state    <= fsm_state;

-- mil
ext_test_err <= test_err;
mil_rst      <= ext_mil_rst;

-- wupr
wupr_fsm_rst <= ext_wupr_fsm_rst;
ext_wup_found <= wup_found;

--wupg
sel              <= ext_sel;
reset            <= ext_reset;
ext_wupg_test_output <= wupg_test_output;
ext_wupg_output  <= wupg_out;

END default;

```

main_tb.vhd

```

LIBRARY ieee;
USE ieee.std_logic_1164.ALL;

ENTITY testbench IS
END testbench;

ARCHITECTURE tb OF testbench IS

--ΘΥΠΕΣ

-- θύρες στη μεριά του Host/Microcontroller
SIGNAL command_in : std_logic_vector(1 DOWNTO 0);

SIGNAL clk : std_logic := '0';
SIGNAL mil_clk : std_logic := '0'; -- ρολόι για το Medium Interface Layer

SIGNAL status_data : std_logic_vector(49 DOWNTO 0);

SIGNAL frame_id : std_logic_vector(4 DOWNTO 0);
SIGNAL data_length : std_logic_vector(4 DOWNTO 0);
SIGNAL type_of_data : std_logic_vector(1 DOWNTO 0);
SIGNAL data : std_logic_vector(31 DOWNTO 0);
SIGNAL validity_bit : std_logic;

-- θύρες στη μεριά του FlexRay Bus Driver
SIGNAL data_out : std_logic_vector(54 DOWNTO 0);
SIGNAL data_in : std_logic_vector(54 DOWNTO 0);

-----

-- βοηθητικά εσωτερικά σήματα για τη λειτουργία του testbench

-- prnm
SIGNAL fsm_rst : std_logic; -- external
SIGNAL reg_rst : std_logic; -- external

-- cs
SIGNAL macro_clock : std_logic := '0'; -- external
SIGNAL global_value : std_logic_vector(3 DOWNTO 0); --external
SIGNAL reset_ff : std_logic; -- external
SIGNAL test_output : std_logic_vector(3 DOWNTO 0); --external
SIGNAL rst : std_logic; -- external
SIGNAL fsm_state : std_logic_vector(1 DOWNTO 0); -- external

-- mil
SIGNAL test_err : std_logic; -- external
SIGNAL mil_rst : std_logic; --external

```

```

-- wupr
SIGNAL wupr_fsm_rst : std_logic;      -- external
SIGNAL wup_found    : std_logic;      -- external, προοριζόμενο για το CHI

-- wupg
SIGNAL sel          : std_logic;      -- external
SIGNAL reset        : std_logic;      -- external
SIGNAL wupg_test_output : std_logic;  -- external
SIGNAL wupg_output   : std_logic_vector(44 DOWNTO 0); --external

BEGIN
controller : ENTITY work.controller PORT MAP (command_in => command_in, clk => clk, status_data =>
status_data, frame_id => frame_id, data_length => data_length, type_of_data => type_of_data, data =>
data, validity_bit => validity_bit, data_out => data_out, data_in => data_in, ext_fsm_rst => fsm_rst,
ext_reg_rst => reg_rst, ext_macro_clock => macro_clock, ext_global_value => global_value, ext_reset_ff
=> reset_ff, ext_test_output => test_output, ext_rst => rst, ext_fsm_state => fsm_state, ext_test_err =>
test_err, ext_mil_rst => mil_rst, ext_wupr_fsm_rst => wupr_fsm_rst, ext_wup_found => wup_found, ext_sel
=> sel, ext_reset => reset, ext_wupg_test_output => wupg_test_output, ext_wupg_output => wupg_output,
mil_clk => mil_clk);

clk          <= NOT clk          AFTER 10 ns;
mil_clk      <= clk;
macro_clock  <= NOT macro_clock AFTER 100 ns;
-- PROCESS (clk)
-- BEGIN
--   IF (clk'event) THEN
--       mil_clk <= NOT mil_clk AFTER 5 ns;
--   END IF;
-- END PROCESS;

reg_rst      <= '0';          -- για τον καταχωρητή στο pmu
mil_rst      <= '0';          -- για τους καταχωρητές εντός του mil
validity_bit <= '0'; -- ψηφίο εγκυρότητας για πλαίσιο από τον host

global_value <= "0101"; -- επιθυμητός αριθμός microticks για επιτυχή συγχρονισμό

PROCESS
BEGIN
--reset to set state to default_config
fsm_rst <= '1';

-- reset dffs inside cs (to get validity bit = 0/Z in mil)
reset_ff <= '1';
-- reset vbit fsm
rst      <= '1';

command_in <= "ZZ";

WAIT FOR 25 ns;

reset_ff <= '0';
rst      <= '0';

-- command: config
fsm_rst  <= '0';
command_in <= "10";

WAIT FOR 100 ns; -- όταν βρεθεί σε κατάσταση sleep και λάβουμε config_ack=1

-- command: wakeup
command_in <= "00";
WAIT FOR 30 ns;

-- αρχικοποίηση wakeup pattern generator
sel <= '0';
reset <= '1';
WAIT FOR 5 ns;
reset <= '0';
sel <= '1';

WAIT FOR 260 ns; -- μετά την αποστολή των πλαισίων
-- δεν είναι αναγκαία η αναμονή, αλλά το κάνω για
-- τη διευκόλυνση οπτικοποίησης των αποτελεσμάτων

```

```
-- command: startup
command_in <= "01";
WAIT FOR 310 ns;

--αφού είναι σε κατάσταση active
-- command: halt
command_in <= "11";

WAIT FOR 400 ns;

END PROCESS;

END tb;
```

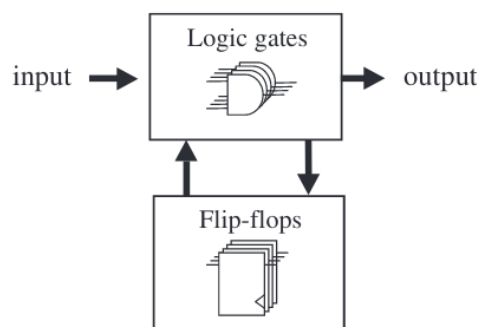
B . Σχόλια σχετικά με τον κώδικα

Σε αυτή την ενότητα αναγράφονται κάποια σχόλια για τον κώδικα, που χρησιμοποιήθηκε στην παρούσα διπλωματική εργασία, και παρουσιάστηκε στο παράρτημα Α.

B .1 Οι μηχανές πεπερασμένων καταστάσεων

Οι μηχανές πεπερασμένων καταστάσεων στα αρχεία `fsm.vhd` (Protocol Management Module) και `wupr_fsm.vhd` (Wakeup Pattern Recognition) βασίζονται στο στυλ σχεδιασμού #2 απ' το βιβλίο του Pedroni [7]. Αντίθετα, η FSM στο αρχείο `vbit_fsm.vhd` (Clock Synchronization) βασίζεται στο στυλ σχεδιασμού #1.

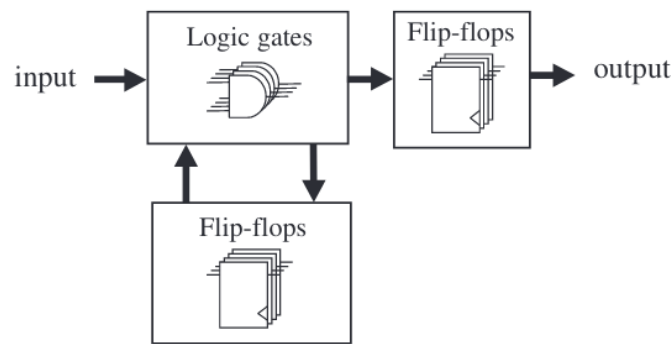
Το στυλ σχεδιασμού #1 αποσκοπεί στην αποθήκευση μόνο της προηγούμενης κατάστασης. Αυτό έχει ως αποτέλεσμα, την άμεση αλλαγή της εξόδου από τη μεταβολή της εισόδου, όταν η έξοδος εξαρτάται από την τρέχουσα είσοδο. Δηλαδή, υπάρχει ασύγχρονη έξοδος (δεν εξαρτάται απ' το ρολόι). Το συνολικό κύκλωμα συνοψίζεται στο παρακάτω σχήμα.



Σχήμα ΒΠ.1: Κυκλωματικό διάγραμμα στυλ σχεδιασμού #1

Αντίθετα, στο στυλ σχεδιασμού #2 αποθηκεύεται και η έξοδος του κυκλώματος, με αποτέλεσμα η έξοδος να είναι σύγχρονη. Έτσι, η μεταβολή της εξόδου θα συμβεί μόνο κατά τη θετική ακμή του ρολογιού.¹ Σε αυτή την περίπτωση, το διάγραμμα του κυκλώματος είναι το παρακάτω.

¹Φυσικά, το είδος της ακμής εξαρτάται απ' το σχεδιασμό, π.χ. θα μπορούσε να 'ναι αρνητική. Απλά στην περίπτωση του συγκεκριμένου κώδικα, χρησιμοποιείται η θετική ακμή, γι' αυτό και αναφέρω αυτή.



Σχήμα Β.2: Κυκλωματικό διάγραμμα στυλ σχεδιασμού #2

B .2 Αντιστοιχίες σημάτων στο σύστημα

Σε αυτή την ενότητα παρουσιάζονται ορισμένες αντιστοιχίες των σημάτων μεταξύ διαφορετικών components, για την καλύτερη κατανόηση του τρόπου διασύνδεσής τους στον κώδικα. Τα υπογραμμισμένα σήματα είναι αυτά, των οποίων τα ονόματα χρησιμοποιήθηκαν στον κώδικα ενοποίησης ([main.vhd](#)).

CHI	PMM
<u>command_out</u>	command
<u>config_ack</u>	config_ack
<u>reset_ack</u>	reset_ack
<u>state</u>	curr_state

Πίνακας Β.1: Αντιστοιχία σημάτων CHI-PMM

CHI	CS
<u>sync_status</u>	output(1)

Πίνακας Β.2: Αντιστοιχία σημάτων CHI-CS

PMM	CS
<u>sync_status</u>	output(1)
curr_state	<u>state</u>

Πίνακας Β.3: Αντιστοιχία σημάτων PMM-CS

CHI	MIL
<u>Frame_rcv</u>	chi_in
<u>Frame_send</u>	chi_out

Πίνακας Β.4: Αντιστοιχία σημάτων CHI-MIL

CS	MIL
<u>output</u>	cs_out
—	cs_in

Πίνακας Β.5: Αντιστοιχία σημάτων CS-MIL

WUPR	MIL
symbol	<u>wup_in(4 downto 1)</u>

Πίνακας Β.6: Αντιστοιχία σημάτων WUPR-MIL

WUPG	MIL
<u>wupg_out</u>	<u>wupg_out</u>

Πίνακας Β.7: Αντιστοιχία σημάτων WUPG-MIL

WUPG	PMM
<u>wupg_en</u>	<u>wupg_en</u>

Πίνακας Β.8: Αντιστοιχία σημάτων WUPG-PMM

Στον πίνακα [Β.5](#) το σήμα cs.in δεν αντιστοιχεί σε κάποιο σήμα του CS. Αυτό συμβαίνει γιατί, δεν υλοποιήθηκε η ανάγνωση των πλαισίων δεδομένων εντός του CS. Όμως, αποτελεί αντικείμενο για μελλοντική δουλειά, αφού ο συγχρονισμός πραγματοποιείται εντός του Clock

Synchronization, και η ανταλλαγή δεδομένων συγχρονισμού απαιτεί την επικοινωνία μεταξύ των διαφορετικών κόμβων.

B.3 Η εγκυρότητα του σήματος sync_status

Είναι γνωστό ότι, το σήμα sync_status σηματοδοτεί το αν ο κόμβος είναι συγχρονισμένος με το δίκτυο. Το συγκεκριμένο σήμα αποτελεί μια απ' τις εξόδους του Clock Synchronization, και αποστέλλεται εντός ενός πλαισίου δεδομένων για την ενημέρωση των υπόλοιπων κόμβων ή/και του διατητή του δικτύου. Εκτός αυτού, πραγματοποιείται συνεχής ανάγνωση του σήματος απ' τις υπομονάδες Controller Host Interface και Protocol Management Module.

Όμως, η ανάγνωση του σήματος θα πρέπει να πραγματοποιείται μόνο όταν αυτό είναι έγκυρο. Το πλαίσιο δεδομένων πάντα περιέχει τη σωστή τιμή του σήματος, χάρη στη μηχανή πεπερασμένων καταστάσεων (Validity Bit FSM), που έχει υλοποιηθεί εντός του Clock Synchronization. Αλλά, η απευθείας ανάγνωσή του από τις υπομονάδες CHI και PMM απαιτεί επιπλέον ψηφιακή λογική.

Αυτή η ψηφιακή λογική δε φαίνεται στις αρχιτεκτονικές, καθώς προστέθηκε κατά την ενοποίηση των συστατικών, και αποτελείται από τον παρακάτω κώδικα:

```
-- έλεγχος εγκυρότητας του sync_status
PROCESS (output)
BEGIN
  CASE output(0) IS -- το validity bit
    WHEN '1' =>
      sync_status <= output(1); -- τότε ίσο με την έξοδο του clock synchronization
    WHEN OTHERS =>
      sync_status <= 'Z';
  END CASE;
END PROCESS;
```

Με αυτόν τον τρόπο, διαβάζεται πάντα η σωστή τιμή του σήματος, ενώ όταν το σήμα βρίσκεται σε υψηλή εμπέδηση (τιμή 'Z'), αγνοείται.

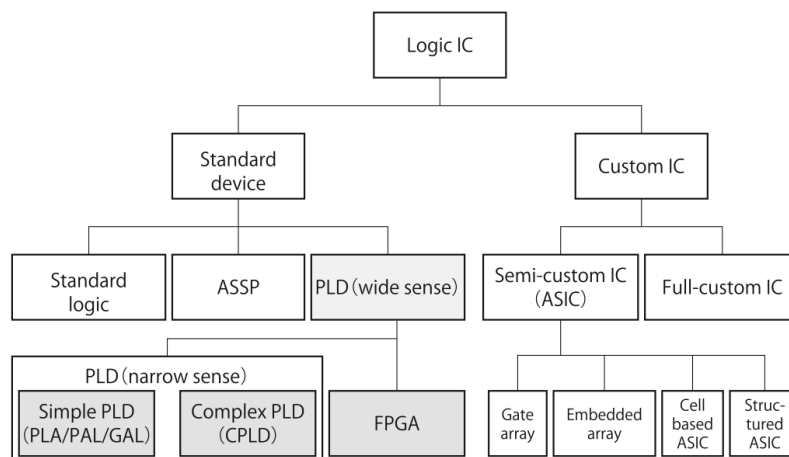
Γ . Τεχνολογία FPGA

Γ .1 Τί είναι FPGA;

Το FPGA (Field-Programmable Gate Array) είναι μια προγραμματιζόμενη λογική συσκευή, και πιο συγκεκριμένα ένας τύπος ολοκληρωμένων κυκλωμάτων, που μπορούν να χρησιμοποιηθούν για να υλοποιήσουν οποιοδήποτε ψηφιακό κύκλωμα. Το όνομα προέρχεται απ' το γεγονός ότι, ο χρήστης χρησιμοποιεί μια συστοιχία πυλών (Gate Array), η οποία είναι προγραμματιζόμενη σε ένα πεδίο (Field) του χώρου εργασίας.

Γ .2 Η κατάταξη των FPGAs

Οι λογικές συσκευές διαιρούνται σε δύο κατηγορίες, στις καθιερωμένες λογικές συσκευές, και στα προσαρμοσμένα ολοκληρωμένα κυκλώματα (ICs), όπως φαίνεται στο σχήμα [Γ0.1](#)



Σχήμα Γ0.1: Η θέση των FPGAs στις συσκευές ημιαγωγών

Στις περισσότερες περιπτώσεις, τα προσαρμοσμένα ολοκληρωμένα κυκλώματα υπερτερούν όσον αφορά την απόδοση (ταχύτητα λειτουργίας), την πυκνότητα (αριθμό πυλών), και την ευελιξία. Παρ' όλα αυτά, το κόστος για μη επαναλαμβανόμενο σχεδιασμό προσαρμοσμένων ολοκληρωμένων κυκλωμάτων συνεχώς αυξάνεται, και ο χρόνος απ' την παραγγελία έως και την παράδοσή τους γίνεται όλο και μεγαλύτερος.

Τα προσαρμοσμένα ολοκληρωμένα κυκλώματα κατατάσσονται σε πλήρως προσαρμοσμένα και ημι-προσαρμοσμένα. Τα πρώτα έχουν σχεδιαστεί εκ του μηδενός, ενώ τα τελευταία χρησιμοποιούν καθιερωμένα στοιχεία¹ (standard cells).

Σε αντίθεση με τα τυποποιημένα κυκλώματα συγκεκριμένης εφαρμογής (ASSP), μια προγραμματιζόμενη λογική συσκευή (PLD) είναι σε θέση να υλοποιήσει διάφορα λογικά κυκλώ-

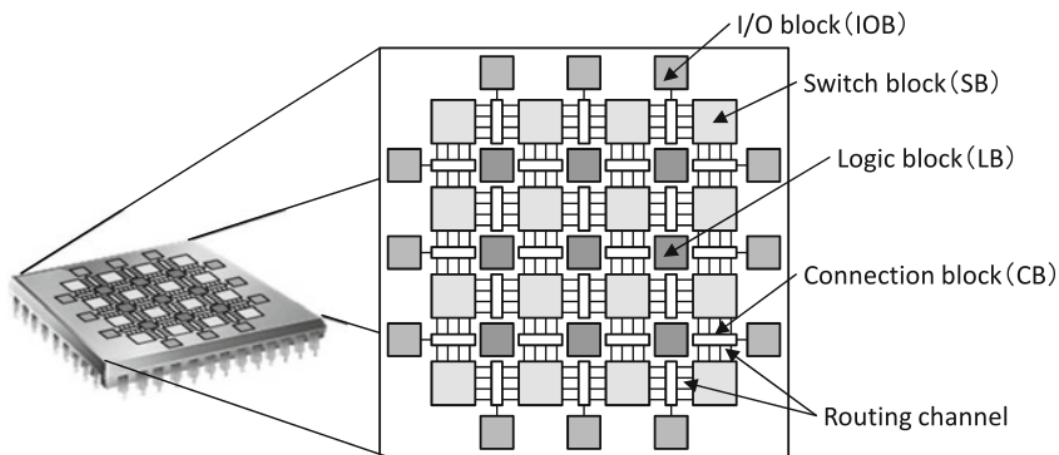
¹Ως καθιερωμένο στοιχείο ορίζεται οποιοδήποτε προσχεδιασμένο λογικό στοιχείο, όπως λογικές πύλες, πολυπλέκτες, flip-flop, κ.λ.π.

ματα, σύμφωνα με την εφαρμογή του χρήστη. Οι προγραμματιζόμενες λογικές συσκευές έχουν αναπτυχθεί ευρέως, καθώς εκμεταλλεύονται τις ιδιότητες του προγραμματισμού πεδίου (field programming) και την ελευθερία αναδιαμόρφωσης (reconfigurability).

Το FPGA είναι μια προγραμματιζόμενη λογική συσκευή, η οποία συνδυάζει λογικά μπλοκ σε μια και μόνο συσκευή για υψηλό βαθμό προγραμματισμού. Μπορεί να προβεί σε μαζική παραγωγή με μια κενή (αρχικοποιημένη) κατάσταση, οπότε καθίσταται μια ιδανικά καθιερωμένη συσκευή για πωλητές ημιαγωγών. Εκτός αυτού, μπορεί να θεωρηθεί ως ένα ολοκληρωμένο κύκλωμα ειδικής εφαρμογής (Application Specific Integrated Circuit) εύκολο στην κατασκευή, αφού έχει μικρό κόστος μη επαναλαμβανόμενου σχεδιασμού και δεν απαιτεί το σχεδιασμό ή/και κατασκευή μάσκας².

Γ.3 Η δομή ενός FPGA

Στο παρακάτω σχήμα απεικονίζεται η δομή ενός FPGA τύπου "νήσου" (island-type).



Σχήμα Γ.2: Δομή παραδοσιακού FPGA

Το βασικό μέρος του FPGA αποτελείται από τρία τμήματα. Το πρώτο τμήμα περιέχει τα λογικά στοιχεία (Logic Blocks ή LBs), τα οποία υλοποιούν λογικά κυκλώματα. Το δεύτερο τμήμα είναι τα στοιχεία εισόδου/εξόδου (Input/Output Blocks ή IOBs), τα οποία οδηγούν σήματα εισόδου και εξόδου. Και το τρίτο τμήμα είναι τα στοιχεία καλωδίωσης, δηλαδή τα μπλοκ μεταγωγής (Switch Blocks ή SBs) και τα μπλοκ σύνδεσης (Connection Blocks ή CBs). Αυτά τα στοιχεία συνδέουν τα λογικά μπλοκ και τα μπλοκ εισόδου/εξόδου. Επίσης, υπάρχουν ένα δίκτυο ρολογιού, μια αλυσίδα ρύθμισης παραμέτρων και ένα κύκλωμα δοκιμών. Τα εμπορικά FPGAs περιλαμβάνουν επιπλέον κυκλώματα για εξειδικευμένες λειτουργίες, όπως επεξεργαστές, μπλοκ μνήμης, και πολυπλέκτες.

Ακολουθεί μια αναλυτικότερη περιγραφή των στοιχείων, που αναφέρθηκαν παραπάνω.

Λογικό Στοιχείο: Υπάρχουν τρία κύρια σχήματα υλοποίησης λογικής προγράμματος, και συγκεκριμένα το lookup table, ο πολυπλέκτης και το γινόμενο όρων (όπως ορίζεται στην άλγεβρα Boole). Οποιαδήποτε μέθοδος αποτελείται από ένα προγραμματιζόμενο μέρος, που μπορεί να χρησιμοποιηθεί για την υλοποίηση οποιουδήποτε λογικού κυκλώματος.

Στοιχείο Εισόδου/Εξόδου: Είναι ένα μπλοκ, το οποίο συνδέει pins εισόδου/εξόδου με στοιχεία καλωδίωσης. Επίσης, περιλαμβάνει ορισμένα κυκλώματα ελέγχου, όπως pull-up, pull-down

²Ως μάσκα ορίζεται η διάταξη υλικών (π.χ. στρώματα μετάλλου, οξειδίου κ.λ.π.) του κυκλώματος, που αποτελούν τα συστατικά του.

και κατευθύνσεις εισόδου/εξόδου. Ακόμη, περιέχει ένα κύκλωμα για την αποθήκευση τιμών, π.χ. flip-flop.

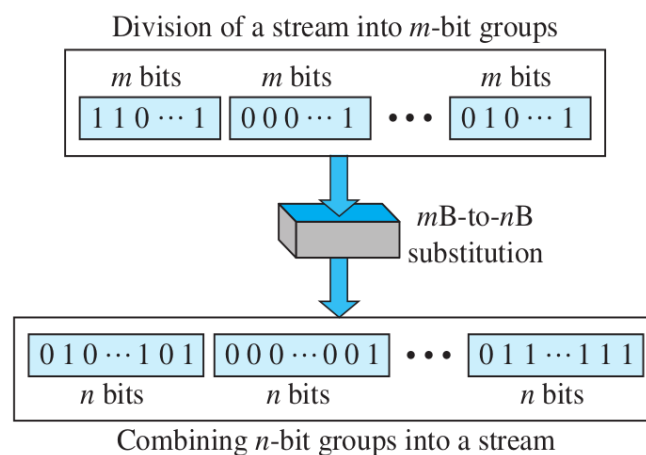
Στοιχείο Καλωδίωσης: Αποτελείται από κανάλια καλωδίωσης, μπλοκ διασύνδεσης, και μπλοκ μεταγωγής στο σημείο σύνδεσης μεταξύ λογικών μπλοκ και μπλοκ εισόδου/εξόδου. Τα μπλοκ μεταγωγής είναι προγραμματιζόμενα και είναι δυνατόν να εγκατασταθεί οποιαδήποτε διαδρομή με τη χρήση των διαθέσιμων πόρων καλωδίωσης.

Άλλα στοιχεία: Οι λογικές λειτουργίες και διασυνδέσεις όλων των μπλοκ αποφασίζονται από τις τιμές στη μνήμη της αλυσίδας ρύθμισης παραμέτρων. Η αλυσίδα ρύθμισης παραμέτρων είναι ένα μονοπάτι, το οποίο γράφει ακολουθιακά τα δεδομένα ρύθμισης σε όλες τις μνήμες των μονάδων διαμόρφωσης.[6]

Δ . Κωδικοποίηση ανά μπλοκ

Η κωδικοποίηση ανά μπλοκ είναι ένα είδος κωδικοποίησης δεδομένων, η οποία μετατρέπει κάθε μπλοκ των m δυαδικών ψηφίων σε ένα μπλοκ των n δυαδικών ψηφίων. Το συγκεκριμένο είδος κωδικοποίησης προσφέρει πλεονασμό για την εγγύηση συγχρονισμού, και παρέχει δυνατότητα εντοπισμού σφαλμάτων.

Η κωδικοποίηση ανά μπλοκ συνήθως περιλαμβάνει τρία στάδια: *διαίρεση*, *αντικατάσταση*, και *συνδυασμός*. Στο στάδιο της διαίρεσης, μια ακολουθία δυαδικών ψηφίων διαιρείται σε ομάδες των m δυαδικών ψηφίων. Στη συνέχεια, ακολουθεί το στάδιο της αντικατάστασης, όπου κάθε ομάδα των m bits αντικαθίσταται από μια ομάδα των n bits. Τέλος, τα μπλοκ των n bits συνδυάζονται, για να σχηματίσουν την τελική ροή δεδομένων. Στο σχήμα [Δ.1](#), απεικονίζεται η διαδικασία της κωδικοποίησης. Φυσικά, για την αποκωδικοποίηση ακολουθείται η ακριβώς αντίστροφη διαδικασία.



Σχήμα Δ.1: Κωδικοποίηση ανά μπλοκ

Στην παρούσα διπλωματική εργασία, χρησιμοποιείται η κωδικοποίηση ανά μπλοκ 4B/5B, η οποία παρουσιάζεται αναλυτικά παρακάτω.

Δ.1 Κωδικοποίηση 4B/5B

Στη συγκεκριμένη κωδικοποίηση έχουμε $m = 4$ και $n = 5$. Αυτό σημαίνει ότι, κάθε μπλοκ των 4 bits αντιστοιχίζεται σε ένα μπλοκ των 5 bits. Οι ομάδες των 5 bits έχουν το πολύ ένα 0 στην αρχή και το πολύ δύο 0 στο τέλος. Έτσι, στη νέα ακολουθία δεδομένων που προκύπτει, δε συναντώνται πάνω από τρία μηδενικά στη σειρά. Με αυτόν τον τρόπο, αποφεύγονται προβλήματα συγχρονισμού, που μπορεί να προκύψουν απ' την κωδικοποίηση NRZ-1 στη συνέχεια.

Στους παρακάτω πίνακες, φαίνονται οι αντιστοιχίες των δυαδικών ακολουθιών στην κωδικοποίηση 4B/5B.

Ακολουθία Δεδομένων	Κωδικοποιημένη Ακολουθία
0000	11110
0001	01001
0010	10100
0011	10101
0100	01010
0101	01011
0110	01110
0111	01111
1000	10010
1001	10011
1010	10110
1011	10111
1100	11010
1101	11011
1110	11100
1111	11101

Πίνακας ΔΠ.1: Κωδικοποίηση 4B/5B:
Αντιστοιχίες ακολουθιών δεδομένων

Ακολουθία Ελέγχου	Κωδικοποιημένη Ακολουθία
Q (Quiet)	00000
I (Idle)	11111
H (Halt)	00100
J (Start delimiter)	11000
K (Start delimiter)	10001
T (End delimiter)	01101
S (Set)	11001
R (Reset)	00111

Πίνακας ΔΠ.2: Κωδικοποίηση 4B/5B:
Αντιστοιχίες ακολουθιών ελέγχου

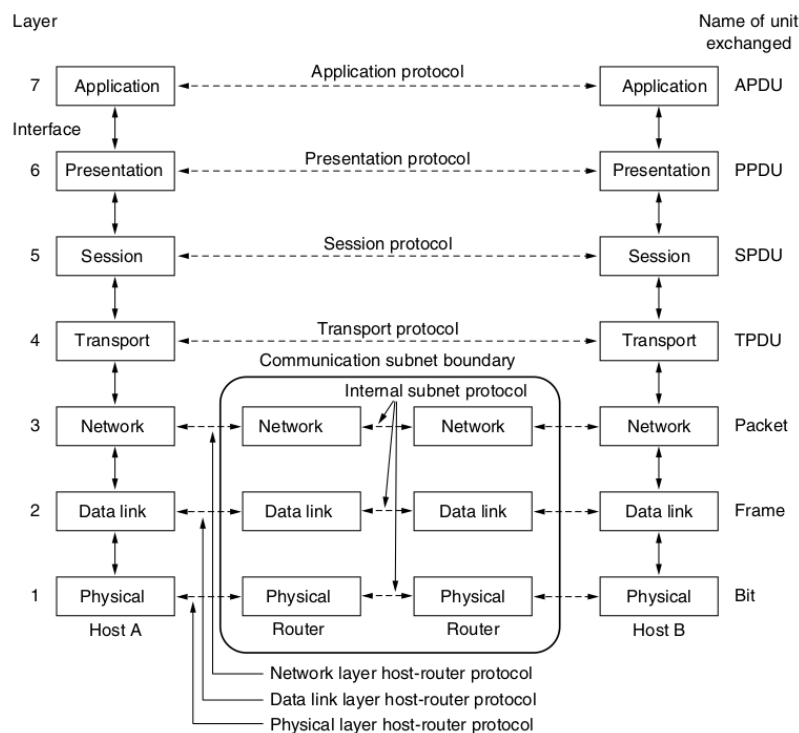
Παρατηρούμε ότι, μια ομάδα των 4 bits μπορεί να έχει 16 διαφορετικούς συνδυασμούς, ενώ μια ομάδα των 5 bits μπορεί να έχει 32 διαφορετικούς συνδυασμούς. Αυτό σημαίνει ότι, υπάρχουν 16 ομάδες των 5 bits, που δε χρησιμοποιούνται στην κωδικοποίηση. Οι 8 απ' αυτές χρησιμοποιούνται για σκοπούς ελέγχου, όπως φαίνεται στον πίνακα ΔΠ.2. Οι υπόλοιπες χρησιμοποιούνται για εντοπισμό σφαλμάτων, αφού όταν γίνει αντιληπτό ότι ένα μπλοκ των 5 ψηφίων αντιστοιχεί στο μη χρησιμοποιούμενο τμήμα του πίνακα, ο παραλήπτης θα γνωρίζει ότι έχει προκύψει κάποιο σφάλμα κατά τη μετάδοση των δεδομένων.[8]

Σημείωση: Ο συγκεκριμένος τρόπος κωδικοποίησης δεν αποτελεί standard για την κωδικοποίηση στο πρωτόκολλο FlexRay. Αλλά τον αναφέρω, γιατί είναι η μέθοδος κωδικοποίησης που χρησιμοποιώ στην υλοποίησή μου.

Ε . Το μοντέλο OSI

Ε .1 Εισαγωγή

Το μοντέλο αναφοράς Open System Interconnection (OSI) είναι μια δομή για τον καθορισμό των συμβάσεων και των διεργασιών, που απαιτούνται απ' τα συστήματα του διαδικτύου, για να επικοινωνήσουν μεταξύ τους. Ο όρος «ανοικτά συστήματα» (open systems) αναφέρεται στο γεγονός ότι, οι προδιαγραφές είναι διαθέσιμες σε όλους. Ο σκοπός του OSI είναι να διευκολύνει τους κατασκευαστές και τους μηχανικούς λογισμικού επικοινωνιών, να παράγουν διαλειτουργικά συστήματα δικτύου. Στο σχήμα ΕΠ.1 απεικονίζεται το μοντέλο OSI (πλην του φυσικού μέσου μετάδοσης δεδομένων).



Σχήμα ΕΠ.1: Το μοντέλο αναφοράς OSI

Το μοντέλο OSI βασίζεται στην ευρέως αποδεκτή τεχνική της διαστρωμάτωσης. Σύμφωνα με αυτήν την προσέγγιση, οι λειτουργίες επικοινωνίας τμηματοποιούνται σε ένα κάθετο σύνολο επιπέδων. Το κάθε επίπεδο σχετίζεται με ένα σύνολο λειτουργιών, το οποίο εμπλουτίζει και χρησιμοποιεί τις υπηρεσίες, που παρέχονται απ' το αμέσως χαμηλότερο επίπεδο. Η προσέγγιση διαστρωμάτωσης αναπτύχθηκε για την επίτευξη των παρακάτω στόχων:

- Η παροχή μιας λογικής αποσύνθεσης του πολύπλοκου δικτύου επικοινωνιών σε μικρότερα, πιο κατανοητά και διαχειρίσιμα μέρη.
- Η παροχή πρότυπων διεπαφών μεταξύ των λειτουργιών του διαδικτύου και των μονάδων.

- Η παροχή μιας πρότυπης γλώσσας για την περιγραφή των λειτουργιών του διαδικτύου, για να χρησιμοποιηθεί από σχεδιαστές, διαχειριστές, κατασκευαστές, και χρήστες του διαδικτύου. [9]

Οι αρχές σύμφωνα με τις οποίες διαμορφώθηκαν τα 7 επίπεδα του μοντέλου OSI, είναι οι ακόλουθες:

1. Κάθε επίπεδο θα πρέπει να δημιουργηθεί, όπου χρειάζεται μια διαφορετική αφαίρεση.
2. Κάθε επίπεδο θα πρέπει να πραγματοποιεί μια καλά ορισμένη λειτουργία.
3. Η λειτουργία κάθε επιπέδου θα πρέπει να επιλεγεί, ώστε να προετοιμάσει το έδαφος για τον καθορισμό διεθνώς πρότυπων πρωτοκόλλων.
4. Τα όρια κάθε επιπέδου θα πρέπει να επιλεγθούν, ώστε να ελαχιστοποιείται η ροή της πληροφορίας μεταξύ των διεπαφών.
5. Ο αριθμός των επιπέδων θα πρέπει να είναι αρκετά μεγάλος, ώστε να μη χρειαστεί διαφορετικές λειτουργίες να καταταχθούν στο ίδιο επίπεδο. Επίσης, τα επίπεδα θα πρέπει να είναι αρκετά μικρά, για να μη γίνουν οι αρχιτεκτονικές τους δυσνόητες.

E .2 Περιγραφή των Επιπέδων

Παρακάτω περιγράφονται όλα τα επίπεδα του μοντέλου OSI, ξεκινώντας απ' το κατώτερο.

E .2.1 Φυσικό Επίπεδο (Physical Layer)

Το **Φυσικό Επίπεδο** ασχολείται με τη μετάδοση ακατέργαστων δυαδικών ψηφίων μέσω ενός καναλιού επικοινωνίας. Τα σχεδιαστικά ζητήματα έχουν να κάνουν με τη διαβεβαίωση ότι, τα δυαδικά ψηφία αποστέλλονται σωστά. Δηλαδή, όταν η μια μεριά στείλει 1, τότε η άλλη λαμβάνει 1, και όχι 0. Τυπικά ερωτήματα υλοποίησης του συγκεκριμένου επιπέδου αποτελούν:

- Τί ηλεκτρικά σήματα θα χρησιμοποιηθούν για την αναπαράσταση των 0 και 1.
- Πόσα nanoseconds θα διαρκέσει ένα δυαδικό ψηφίο.
- Πώς εγκαθιδρύεται η πρώτη επικοινωνία.
- Πώς τερματίζεται η επικοινωνία, όταν και οι δύο πλευρές έχουν ολοκληρώσει τη μετάδοση δεδομένων.

Τα σχεδιαστικά ζητήματα ασχολούνται τόσο με τις μηχανικές, ηλεκτρικές διεπαφές, και διεπαφές χρονισμού, όσο με το φυσικό μέσο μετάδοσης.

E .2.2 Επίπεδο Συνδέσμου Δεδομένων (Data Link Layer)

Ο κύριος στόχος του **Επιπέδου Συνδέσμου Δεδομένων** είναι ο μετασχηματισμός μιας ακατέργαστης μετάδοσης σε μια σειρά δεδομένων, η οποία εμφανίζεται απαλλαγμένη από σφάλματα μετάδοσης. Αυτό επιτυγχάνεται με την απόκρυψη των πραγματικών σφαλμάτων απ' το επίπεδο δικτύου, ώστε να μη γίνουν τα σφάλματα αντιληπτά απ' αυτό.

Αυτή η διεργασία πραγματοποιείται με τη διάσπαση των δεδομένων εισόδου, απ' τη μεριά του αποστολέα, σε *πλαίσια δεδομένων*. Στη συνέχεια, αυτά τα πλαίσια αποστέλλονται ακολουθιακά, και ο παραλήπτης επιβεβαιώνει την επιτυχή λήψη τους, αποστέλλοντας ένα *πλαίσιο αναγνώρισης* (acknowledgement frame) πίσω στον αποστολέα.

Ε .2.3 Επίπεδο Δικτύου (Network Layer)

Το **Επίπεδο Δικτύου** ελέγχει τη λειτουργία του υποδικτύου. Ένα κύριο σχεδιαστικό ζήτημα είναι ο τρόπος δρομολόγησης των πακέτων απ' την πηγή στον προορισμό. Οι διαδρομές βασίζονται σε στατικούς πίνακες, των οποίων τα περιεχόμενα μεταβάλλονται σπάνια ή συχνά για την αποφυγή αποτυχημένων στοιχείων στο δίκτυο. Επίσης, οι διαδρομές μπορούν να αποφασίζονται κατά την έναρξη μιας συνόδου (π.χ. σύνδεση σε απομακρυσμένο μηχάνημα). Τέλος, ένας άλλος ο τρόπος είναι η συνεχής αλλαγή των διαδρομών στον πίνακα, με βάση το τρέχον φόρτο εργασίας στο δίκτυο.

Μια άλλη ευθύνη του επιπέδου δικτύου είναι η διαχείριση της συμφόρησης στη σύνδεση με ανώτερα επίπεδα, τα οποία καθορίζουν το φόρτο εργασίας στο δίκτυο. Με άλλα λόγια, η παρεχόμενη ποιότητα υπηρεσίας (Quality of Service), που αφορά την καθυστέρηση, το χρόνο μετάδοσης κλπ., είναι μια ευθύνη του επιπέδου δικτύου.

Επιπλέον, το επίπεδο δικτύου οφείλει να επιτρέπει τη διασύνδεση μεταξύ ετερογενών δικτύων, καθώς η διευθυνσιοδότηση, το μέγεθος των πακέτων, κλπ. μπορεί να διαφέρουν από δίκτυο σε δίκτυο.

Ε .2.4 Επίπεδο Μεταφοράς (Transport Layer)

Η βασική λειτουργία του **Επιπέδου Μεταφοράς** είναι η αποδοχή των δεδομένων απ' το ανώτερο επίπεδο (Session Layer), η διάσπασή τους σε μικρότερα τμήματα, και η αποστολή τους στο επίπεδο δικτύου. Ακόμη, θα πρέπει να επιβεβαιώνεται η σωστή άφιξη των τμημάτων απ' την άλλη μεριά. Όλα αυτά θα πρέπει να πραγματοποιούνται με αποδοτικό τρόπο, και με τέτοιο ώστε να απομονώνονται τα ανώτερα επίπεδα, και να μην επηρεάζονται από αναπόφευκτες αλλαγές στο υλικό στο πέρασμα του χρόνου.

Επίσης, το επίπεδο μεταφοράς αποφασίζει το είδος της υπηρεσίας (Type of Service), που θα παρέχεται στο επίπεδο συνόδου (Session Layer), και εν τέλει στους χρήστες του δικτύου. Το είδος της υπηρεσίας αποφασίζεται κατά την εγκαθίδρυση της σύνδεσης.

Ε .2.5 Επίπεδο Συνόδου (Session Layer)

Το **Επίπεδο Συνόδου** επιτρέπει στους χρήστες διαφορετικών μηχανημάτων, να εγκαθιδρύσουν συνόδους μεταξύ τους. Αυτές οι σύνοδοι προσφέρουν διάφορες υπηρεσίες, όπως *έλεγχος διαλόγου* (παρακολούθηση της σειράς μετάδοσης), *διαχείριση token* (αποτροπή των δύο μερών να επιχειρήσουν την ίδια κρίσιμη λειτουργία ταυτόχρονα), και *συγχρονισμό* (προσθήκη σημείων ελέγχου για μεταδόσεις μεγάλης χρονικής διάρκειας, ώστε να επιτραπεί η συνέχειά τους σε περίπτωση βλάβης).

Ε .2.6 Επίπεδο Παρουσίασης (Presentation Layer)

Το **Επίπεδο Παρουσίασης** ασχολείται με τη σύνταξη και τη σημασιολογία των δεδομένων, που μεταδίδονται. Προκειμένου να επιτευχθεί η επικοινωνία μεταξύ δύο υπολογιστών με διαφορετικές εσωτερικές αναπαραστάσεις δεδομένων, χρειάζεται η αναπαράσταση των δεδομένων προς συναλλαγή με έναν αφαιρετικό τρόπο. Αυτό μπορεί να επιτευχθεί με τη χρήση ενός καθιερωμένου τρόπου κωδικοποίησης των δεδομένων. Το επίπεδο παρουσίασης είναι υπεύθυνο για τη διαχείριση των αφηρημένων (abstract) δομών δεδομένων και τον καθορισμό των δομών δεδομένων υψηλού επιπέδου προς συναλλαγή.

Ε .2.7 Επίπεδο Εφαρμογής (Application Layer)

Το **Επίπεδο Εφαρμογής** περιέχει το σύνολο των πρωτοκόλλων, που απαιτούνται απ' τον χρήστη. Ένα τέτοιο πρωτόκολλο εφαρμογής είναι το HTTP (HyperText Transfer Protocol), το οποίο αποτελεί τη βάση του παγκόσμιου ιστού (World Wide Web). Άλλα χρησιμοποιούμενα πρωτόκολλα εφαρμογών είναι η μεταφορά αρχείων, το ηλεκτρονικό Mail, και τα διαδικτυακά νέα. [10]

Βιβλιογραφία

- [1] Michael Barr. Programming embedded systems in C and C++. " O'Reilly Media, Inc.", 1999.
- [2] Steve Heath. Embedded systems design. Elsevier, 2002.
- [3] Shanker Shreejith and Suhaib A Fahmy. Extensible flexray communication controller for fpga-based automotive systems. IEEE transactions on vehicular technology, 64(2):453--465, 2014.
- [4] Peter Marwedel. Embedded system design: embedded systems foundations of cyber-physical systems, and the internet of things. Springer Nature, 2021.
- [5] Dominique Paret. FlexRay and its applications: real time multiplexed network. Wiley Online Library, 2012.
- [6] Hideharu Amano. Principles and structures of FPGAs. Springer, 2018.
- [7] Volnei A Pedroni. Circuit design with VHDL. MIT press, 2020.
- [8] B. A. Forouzan. Data communications and networking, fifth edition. McGraw-Hill, 2013.
- [9] Sumit Kumar, Sumit Dalal, and Vivek Dixit. The osi model: Overview on the seven layers of computer networks. International Journal of Computer Science and Information Technology Research, 2(3):461--466, 2014.
- [10] TANENBAUM ANDREW S and WETHERALL DAVID J. Computer networks fifth edition. pearson, 2011.

